

JAVA 17 OCP · YDS STUDY SERIES

Unit 10 · Streams · Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu ana kaynak, [OCP_Java_SE17_Chapter1den_Itibaren.pdf](#) içindeki ilgili chapter gövdesini ve Appendix resmî cevaplarını kaynak sırasını koruyan English → Türkçe paragraf çiftleriyle bir araya getirir. Kod ve terminal çıktıları çevrilmeden, bir kez ve kaynak konumunda gösterilir.

[Vocabulary](#) · [Grammar notes](#) · [Teknik hafıza notu](#)

Kaynak kapsam manifesti

- Kaynak: [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- Chapter: 10 · Streams
- Chapter PDF sayfaları: 531–590
- Appendix cevap sayfaları: 942–945
- Beklenen sayfa marker'ı: 60
- Beklenen resmî cevap: 21
- Eşleme biçimi: English paragraf → Türkçe çeviri → varsa kod

İçindekiler

- 1 [Returning an Optional](#)
- 2 [Using Streams](#)
- 3 [Using Common Terminal Operations](#)
- 4 [Using Common Intermediate Operations](#)
- 5 [Working with Primitive Streams](#)
- 6 [Working with Advanced Stream Pipeline Concepts](#)
- 7 [Summary](#)
- 8 [Exam Essentials](#)
- 9 [Review Questions](#)
- 10 [Official Review Question Answers / Resmî Cevaplar](#)

Chapter 10 · Streams · Eksiksiz çift dilli kaynak

Chapter 10 · Streams

English: OCP exam objectives covered in this chapter: Working with Streams and Lambda expressions.
Türkçe: Bu bölümde kapsanan OCP sınav hedefi: Streams ve lambda expressions ile çalışmak. **English:** Use Java object and primitive Streams, including lambda expressions implementing functional interfaces, to supply, filter, map, consume, and sort data. **Türkçe:** Veriyi sağlamak, filtrelemek, eşlemek ([map](#)), tüketmek ve sıralamak için functional interfaces uygulayan lambda expressions dâhil Java object ve primitive Streams kullanın. **English:** Perform decomposition, concatenation and reduction, and grouping and partitioning on sequential and parallel streams. **Türkçe:** Sequential ve parallel streams üzerinde decomposition, concatenation, reduction, grouping ve partitioning işlemlerini gerçekleştirin.

English: By now, you should be comfortable with the lambda and method reference syntax. Both are used when implementing functional interfaces. If you need more practice, you may want to go back and review Chapter 8, “Lambdas and Functional Interfaces,” and Chapter 9, “Collections and Generics.” In this chapter, we add actual functional programming to that, focusing on the Streams API. **Türkçe:** Şimdiye kadar, lambda ve method reference sözdizimi ile rahat olmalısınız. Her ikisi de functional interfaces uygularken kullanılır. Daha fazla uygulamaya ihtiyacınız varsa, geri dönüp Bölüm 8, “Lambdas ve Functional Interfaces,” ve Bölüm 9, “Collections ve Jenerikleri”ni gözden geçirmek isteyebilirsiniz. Bu bölümde, Streams API'a odaklanarak, buna gerçek işlevsel programlama ekliyoruz. **English:** Note that the Streams API in this chapter is used for functional programming. By contrast, there are also java.io streams, which we talk about in Chapter 14, “I/O.” Despite both using the word stream, they are nothing alike. **Türkçe:** Bu bölümdeki Streams API fonksiyonel programlama için kullanıldığına dikkat edin. Buna karşılık, Bölüm 14'te bahsettiğimiz java.io streams da var, “I/O.” Her ikisi de stream kelimesini kullanmasına rağmen, hiçbir şey birbirine benzemez. **English:** In this chapter, we introduce Optional. Then we introduce the Stream pipeline and tie it all together. You might want to read this chapter twice before doing the review questions so that you really get it. Functional programming tends to have a steep learning curve but can be very exciting once you get the hang of it. **Türkçe:** Bu bölümde Optional'i tanıtıyoruz. Sonra Stream pipeline'u tanıtıyoruz ve hepsini bir araya getiriyoruz. Konuyu gerçekten kavramak için review questions bölümüne geçmeden önce bu chapter'ı iki kez okumak isteyebilirsiniz. Functional programming'in başlangıçta dik bir öğrenme eğrisi vardır; ancak alışınca oldukça heyecan verici olabilir.

Returning an Optional

English: Suppose that you are taking an introductory Java class and receive scores of 90 and 100 on the first two exams. Now, we ask you what your average is. An average is calculated by adding the scores and dividing by the number of scores, so you have $(90+100)/2$. This gives 190/2, so you answer with 95. Great! **Türkçe:** Giriş Java dersini aldığınızı ve ilk iki sınavda 90 ve 100 puan aldığınızı varsayalım. Şimdi size ortalamanızın ne olduğunu soruyoruz. Bir ortalama, puanları ekleyerek ve puan sayısına bölünerek hesaplanır, böylece $(90+100)/2$ 'ye sahip olursunuz. Bu da 190/2 verir, yani 95 ile cevap verirsiniz. Harika! **English:** Now suppose that you are taking your second class on Java, and it is the first day of class. We ask you what your average is in this class that just started. You haven't taken any exams yet, so you don't have anything to average. It wouldn't be accurate to say that your average is zero. That sounds bad and isn't true. There simply isn't any data, so you don't have an average. **Türkçe:** Şimdi ikinci sınıfınızı Java üzerinde aldığınızı varsayalım ve bu sınıfın ilk günü. Bu sınıfta yeni başlayan ortalamanızın ne olduğunu soruyoruz. Henüz herhangi bir sınava girmediniz, bu yüzden ortalama bir şeyiniz yok. Ortalamanızın sıfır olduğunu söylemek doğru olmaz. Bu kulağa kötü geliyor ve doğru değil. Herhangi bir veri yok, bu yüzden bir ortalamanız yok. **English:** How do we express this “we don't know” or “not applicable” answer in Java? We use the Optional type. An Optional is created using a factory. You can either request an empty Optional or pass a value for the Optional to wrap. Think of an Optional as a box that might have something in it or might instead be empty. Figure 10.1 shows both options. **Türkçe:** “Bilmiyoruz” veya “uygulanamaz” yanıtını Java'da nasıl ifade edebiliriz? Optional türünü kullanıyoruz. Bir Optional bir fabrika kullanılarak oluşturulur. Boş bir Optional talep edebilir veya Optional paketi için bir değer geçebilirsiniz. Bir Optional kutusunun içinde bir şey olabileceğini veya bunun yerine boş olabileceğini düşünün. Şekil 10.1 her iki seçeneği de gösterir. **English:** FIGURE 10.1 Optional 95 **Türkçe:** FIGURE 10.1 Optional 95

```
Optional.empty() Optional.of(95)
```

Creating an Optional

English: Here's how to code our average method: **Türkçe:** İşte ortalama yöntemimizi nasıl kodlayacağımız:

```
public static Optional<Double> average(int... scores) {  
    if (scores.length == 0) return Optional.empty();  
    int sum = 0;  
    for (int score: scores) sum += score;  
    return Optional.of((double) sum / scores.length);  
}
```

English: Line 11 returns an empty Optional when we can't calculate an average. Lines 12 and 13 add up the scores. There is a functional programming way of doing this math, but we will get to that later in the chapter. In fact, the entire method could be written in one line, but that wouldn't teach you how Optional works! Line 14 creates an Optional to wrap the average. **Türkçe:** Satır 11, bir ortalama hesaplayamadığımızda boş bir Optional döndürür. 12 ve 13 numaralı satırlar puanları toplar. Bu matematiği yapmanın işlevsel bir programlama yolu var, ancak buna daha sonra bölümde ulaşacağız. Aslında, tüm yöntem tek bir satırda yazılabilir, ancak bu size Optional nasıl çalıştığını öğretmez! Satır 14, ortalama sarmak için bir Optional oluşturur. **English:** Calling the method shows what is in our two boxes: **Türkçe:** Method'u çağırmak iki kutumuzun içinde ne olduğunu gösterir:

```
System.out.println(average(90, 100)); // Optional[95.0]  
System.out.println(average()); // Optional.empty
```

English: You can see that one Optional contains a value and the other is empty. Normally, we want to check whether a value is there and/or get it out of the box. Here's one way to do that: **Türkçe:** Bir Optional değeri içerdiğini ve diğerinin boş olduğunu görebilirsiniz. Normalde, bir değer orada olup olmadığını kontrol etmek ve / veya kutudan çıkarmak istiyoruz. İşte bunu yapmanın bir yolu:

```
Optional<Double> opt = average(90, 100);  
if (opt.isPresent())  
    System.out.println(opt.get()); // 95.0
```

English: First we check whether the Optional contains a value. Then we print it out. What if we didn't do the check, and the Optional was empty? **Türkçe:** Önce Optional'ın bir değer içerip içermediğini kontrol ederiz. Ardından değeri yazdırırız. Peki bu kontrolü yapmasaydık ve Optional boş olsaydı?

```
Optional<Double> opt = average();  
System.out.println(opt.get()); // NoSuchElementException
```

English: We'd get an exception since there is no value inside the Optional. **Türkçe:** Optional içinde bir değer olmadığı için bir istisna elde ederiz.

```
java.util.NoSuchElementException: No value present
```

English: When creating an Optional, it is common to want to use empty() when the value is null. You can do this with an if statement or ternary operator. We use the ternary operator (?:) to simplify the code, which you saw in Chapter 2, “Operators.” **Türkçe:** Optional oluşturulurken, değer null olduğunda empty() kullanmak istemek yaygındır. Bunu if ifadesi veya ternary operatörü ile yapabilirsiniz. Bölüm 2’de gördüğünüz kodu basitleştirmek için üçlü operatörü (?:) kullanıyoruz, “Operatörler”.

```
Optional o = (value == null)? Optional.empty(): Optional.of(value);
```

English: If value is null, o is assigned the empty Optional. Otherwise, we wrap the value. Since this is such a common pattern, Java provides a factory method to do the same thing. **Türkçe:** Eğer değer null ise, o boş Optional olarak atanır. Aksi takdirde, değeri paketleriz. Bu çok yaygın bir kalıp olduğundan, Java aynı şeyi yapmak için bir factory method sağlar.

```
Optional o = Optional.ofNullable(value);
```

English: That covers the static methods you need to know about Optional. Table 10.1 summarizes most of the instance methods on Optional that you need to know for the exam. There are a few others that involve chaining. We cover those later in the chapter. **Türkçe:** Bu, Optional hakkında bilmeniz gereken statik yöntemleri kapsar. Tablo 10.1, sınav için bilmeniz gereken örnek yöntemlerin çoğunu Optional üzerinde özetler. Zincirlemeyi içeren birkaç tane daha var. Bunları daha sonra bölümde ele alacağız.

English: TABLE 10.1 Common Optional instance methods When Optional contains value Method When Optional is empty **Türkçe:** TABLE 10.1 Ortak Optional örnek yöntemleri Optional değer Metodu içerdiğinde Optional boş olduğunda

```
get() Throws exception Returns value
```

English: ifPresent(Consumer c) Does nothing Calls Consumer with value isPresent() Returns false Returns true **Türkçe:** ifPresent(Consumer c) Hiçbir şey Consumer değerini isPresent() ile aramaz Yanlış döndürür Doğru döndürür

```
orElse(T other) Returns other parameter Returns value  
orElseGet(Supplier s) Returns result of calling Supplier Returns value
```

English: orElseThrow() Throws NoSuchElementException Returns value Throws exception created by calling Returns value orElseThrow(Supplier s) **Türkçe:** orElseThrow() NoSuchElementException değerini atar orElseThrow(Supplier s) değerini çağırarak oluşturulan istisnayı atar **English:** Supplier You’ve already seen get() and isPresent(). The other methods allow you to write code that uses an Optional in one line without having to use the ternary operator. This makes the code easier to read. Instead of using an if statement, which we used when checking the average earlier, we can specify a Consumer to be run when there is a value inside the Optional. When there isn’t, the method simply skips running the Consumer. **Türkçe:** Supplier get() ve isPresent()’i zaten gördünüz. Diğer yöntemler, üçlü operatörü kullanmak zorunda kalmadan bir satırda Optional kullanan kod yazmanıza izin verir. Bu, kodun okunmasını kolaylaştırır. Daha önce ortalamayı kontrol ederken kullandığımız bir if ifadesi kullanmak yerine, Optional içinde bir değer olduğunda çalıştırılacak bir Consumer belirtebiliriz. Olmadığında, yöntem sadece Consumer çalıştıran atlar.

```
Optional<Double> opt = average(90, 100);
opt.ifPresent(System.out::println);
```

English: Using `ifPresent()` better expresses our intent. We want something done if a value is present. You can think of it as an `if` statement with no `else`. **Türkçe:** `ifPresent()` kullanmak niyetimizi daha iyi ifade eder. Bir değer varsa bir şeyin yapılmasını istiyoruz. Başkası olmayan bir ifade olarak düşünebilirsiniz.

Dealing with an Empty Optional

English: The remaining methods allow you to specify what to do if a value isn't present. There are a few choices. The first two allow you to specify a return value either directly or using a `Supplier`. **Türkçe:** Kalan yöntemler, bir değer yoksa ne yapmanız gerektiğini belirtmenize izin verir. Birkaç `Optional` var. İlk ikisi, bir dönüş değerini doğrudan veya bir `Supplier` kullanarak belirtmenize izin verir.

```
Optional<Double> opt = average();
System.out.println(opt.orElse(Double.NaN));
System.out.println(opt.orElseGet(() -> Math.random()));
```

English: This prints something like the following: **Türkçe:** Bu, aşağıdaki gibi bir şey yazdırır: **English:** `NaN 0.49775932295380165` Line 31 shows that you can return a specific value or variable. In our case, we print the "not a number" value. Line 32 shows using a `Supplier` to generate a value at runtime to return instead. I'm glad our professors didn't give us a random average, though! **Türkçe:** `NaN 0.49775932295380165` Satır 31, belirli bir değeri veya değişkeni döndürebileceğinizi gösterir. Bizim durumumuzda, "bir sayı değil" değerini yazdırıyoruz. Satır 32, bunun yerine dönmek için çalışma zamanında bir değer oluşturmak için bir `Supplier` kullanarak gösterir. Profesörlerimizin bize rastgele bir ortalama vermemesine sevindim! **English:** Alternatively, we can have the code throw an exception if the `Optional` is empty. **Türkçe:** Alternatif olarak, `Optional` boşsa, kodun bir istisna atmasını sağlayabiliriz.

```
Optional<Double> opt = average();
System.out.println(opt.orElseThrow());
```

English: This prints something like the following: **Türkçe:** Bu, aşağıdaki gibi bir şey yazdırır: **English:** `Exception in thread "main" java.util.NoSuchElementException` **Türkçe:** `Exception` iş parçacığında "ana" `java.util.NoSuchElementException`: **English:** No value present **Türkçe:** Mevcut değer yok

```
at java.base/java.util.Optional.orElseThrow(Optional.java:382)
```

English: Without specifying a `Supplier` for the exception, Java will throw a `NoSuchElementException`. Alternatively, we can have the code throw a custom exception if the `Optional` is empty. Remember that the stack trace looks weird because the lambdas are generated rather than named classes. **Türkçe:** İstisna için `Supplier` belirtmeden, Java bir `NoSuchElementException` atar. Alternatif olarak, `Optional` boşsa, kodun özel bir istisna atmasını sağlayabiliriz. Yığın izinin garip görüldüğünü unutmayın, çünkü lambdas adlandırılmış sınıflar yerine oluşturulur.

```
Optional<Double> opt = average();
System.out.println(opt.orElseThrow(
    () -> new IllegalStateException()));
```

English: This prints something like the following: **Türkçe:** Bu, aşağıdaki gibi bir şey yazdırır: **English:** Exception in thread "main" java.lang.IllegalStateException **Türkçe:** Exception iş parçasığında "ana" java.lang.IllegalStateException

```
at optionals.Methods.lambda$orElse$1(Methods.java:31)
at java.base/java.util.Optional.orElseThrow(Optional.java:408)
```

English: Line 32 shows using a Supplier to create an exception that should be thrown. Notice that we do not write throw new IllegalStateException(). The orElseThrow() method takes care of actually throwing the exception when we run it. **Türkçe:** Satır 32, atılması gereken bir istisna oluşturmak için bir Supplier kullanarak gösterir. Yazmadığımıza dikkat edin, yeni IllegalStateException() atın. orElseThrow() yöntemi, çalıştırdığımızda istisnayı gerçekten atmaya özen gösterir. **English:** The two methods that take a Supplier have different names. Do you see why this code does not compile? **Türkçe:** Supplier alan iki yöntemin farklı isimleri vardır. Bu kodun neden derlenmediğini görüyor musunuz?

```
System.out.println(opt.orElseGet(
    () -> new IllegalStateException()); // DOES NOT COMPILE
```

English: The opt variable is an Optional<Double>. This means the Supplier must return a Double. Since this Supplier returns an exception, the type does not match. **Türkçe:** Opt değişkeni bir Optional<Double> dir. Bu, Supplier bir Double döndürmesi gerektiği anlamına gelir. Bu Supplier bir istisna döndürdüğünden, tür eşleşmez. **English:** The last example with Optional is really easy. What do you think this does? **Türkçe:** Optional ile son örnek gerçekten kolaydır. Sence bu ne işe yarıyor?

```
Optional<Double> opt = average(90, 100);
System.out.println(opt.orElse(Double.NaN));
System.out.println(opt.orElseGet(() -> Math.random()));
System.out.println(opt.orElseThrow());
```

English: It prints out 95.0 three times. Since the value does exist, there is no need to use the "or else" logic. **Türkçe:** 95.0'ı üç kez yazdırıyor. Değer var olduğundan, "yoksa" mantığını kullanmaya gerek yoktur. **English:** Is Optional the Same as null? **Türkçe:** Optional null ile aynı mı? **English:** An alternative to Optional is to return null. There are a few shortcomings with this approach. One is that there isn't a clear way to express that null might be a special value. By contrast, returning an Optional is a clear statement in the API that there might not be a value. **Türkçe:** Optional için bir alternatif null döndürmektir. Bu yaklaşımda bazı eksiklikler vardır. Birincisi, null özel bir değer olabileceğini ifade etmenin net bir yolu olmadığıdır. Buna karşılık, bir Optional döndürmek, API 'da bir değer olmayabileceğini belirten açık bir ifadedir. **English:** Another advantage of Optional is that you can use a functional programming style with **Türkçe:** Optional'ın bir diğer avantajı, fonksiyonel bir programlama stilini kullanabilmenizdir.

```
ifPresent() and the other methods rather than needing an if statement. Finally, you
```

English: see toward the end of the chapter that you can chain Optional calls. **Türkçe:** Optional çağrılarını zincirleyebileceğiniz bölümün sonuna doğru bakın.

Using Streams

English: A stream in Java is a sequence of data. A stream pipeline consists of the operations that run on a stream to produce a result. First, we look at the flow of pipelines conceptually. After that, we get into the code. **Türkçe:** Java içindeki bir stream bir veri dizisidir. Bir stream pipeline, bir sonuç üretmek için bir stream üzerinde çalışan işlemlerden oluşur. İlk olarak, kavramsal olarak pipelines akışına bakıyoruz. Ondan sonra koda gireceğiz.

Understanding the Pipeline Flow

English: Think of a stream pipeline as an assembly line in a factory. Suppose that we are running an assembly line to make signs for the animal exhibits at the zoo. We have a number of jobs. It is one person's job to take the signs out of a box. It is a second person's job to paint the sign. It is a third person's job to stencil the name of the animal on the sign. It's the last person's job to put the completed sign in a box to be carried to the proper exhibit. **Türkçe:** Bir fabrikada bir stream pipeline montaj hattı olarak düşünün. Hayvanat bahçesindeki hayvan sergilerine işaret vermek için bir montaj hattı işlettiğimizi varsayalım. Bir dizi işimiz var. Bir kişinin görevi, işaretleri bir kutudan çıkarmaktır. İşareti boyamak ikinci bir kişinin işidir. İşaretteki hayvanın adını koklamak üçüncü bir kişinin görevidir. Tamamlanan işareti uygun sergiye taşımak için bir kutuya koymak son kişinin işidir. **English:** Notice that the second person can't do anything until one sign has been taken out of the box by the first person. Similarly, the third person can't do anything until one sign has been painted, and the last person can't do anything until it is stenciled.

Türkçe: İkinci kişinin ilk kişi tarafından kutudan bir işaret çıkarılıncaya kadar hiçbir şey yapamayacağına dikkat edin. Benzer şekilde, üçüncü kişi bir işaret boyanıncaya kadar hiçbir şey yapamaz ve son kişi de kalıplanana kadar hiçbir şey yapamaz. **English:** The assembly line for making signs is finite. Once we process the contents of our box of signs, we are finished. Finite streams have a limit. Other assembly lines essentially run forever, like one for food production. Of course, they do stop at some point when the factory closes down, but pretend that doesn't happen. Or think of a sunrise/sunset cycle as infinite, since it doesn't end for an inordinately large period of time. **Türkçe:** İşaretler yapmak için montaj hattı sonludur. İşaret kutumuzun içeriğini işlediğimizde, işimiz biter. Finite streams bir sınırı vardır. Diğer montaj hatları, temel olarak gıda üretimi için olduğu gibi sonsuza kadar çalışır. Tabii ki, fabrika kapandığında bir noktada dururlar, ancak böyle bir şey olmuyormuş gibi davranırlar. Ya da bir gündeğümü/gündeğümü döngüsünü sonsuz olarak düşünün, çünkü aşırı derecede büyük bir zaman dilimi için sona ermez. **English:** Another important feature of an assembly line is that each person touches each element to do their operation, and then that piece of data is gone. It doesn't come back. The next person deals with it at that point. This is different than the lists and queues that you saw in the previous chapter. With a list, you can access any element at any time. With a queue, you are **Türkçe:** Bir montaj hattının bir diğer önemli özelliği, her bir kişinin operasyonlarını yapmak için her bir elemana dokunması ve daha sonra bu veri parçasının ortadan kalkmasıdır. Geri dönmüyor. Bir sonraki kişi bu noktada onunla ilgilenir. Bu, önceki bölümde gördüğümüz lists ve kuyruklardan farklıdır. list ile herhangi bir elemana istediğiniz zaman erişebilirsiniz. Kuyrukla, sen

English: limited in which elements you can access, but all of the elements are there. With streams, the data isn't generated up front— it is created when needed. This is an example of lazy evaluation, which delays execution until necessary. **Türkçe:** Hangi elementlere erişebileceğinizi sınırlayın, ancak tüm elemanlar oradadır. streams ile veriler ön planda oluşturulmaz gerektiğinde oluşturulur. Bu, gerekli olana kadar yürütmeyi geciktiren lazy evaluation örneğidir. **English:** Many things can happen in the assembly line stations along the way. In functional programming, these are called stream operations. Just like with the assembly line, operations occur in a pipeline. Someone has to start and end the work, and there can be any number of stations in between. After all, a job with one person isn't an assembly line! There are three parts to a stream pipeline, as shown in Figure 10.2. **Türkçe:** Yol boyunca montaj hattı istasyonlarında birçok şey olabilir. İşlevsel programlamada bunlara stream işlemler denir. Montaj hattında olduğu gibi, işlemler bir pipeline içinde gerçekleşir. Birinin işi başlatması ve bitirmesi gerekiyor ve aralarında herhangi bir sayıda istasyon olabilir. Sonuçta, bir kişi ile bir iş bir montaj hattı değildir! Şekil 10.2'de gösterildiği gibi, bir stream pipeline için üç bölüm vardır. **English:** ■ ■ Source: Where the stream comes from. ■ ■ Intermediate operations: Transforms the stream into another one. There can be as few or as many intermediate operations as you'd like. Since streams use lazy evaluation, the intermediate operations do not run until the terminal operation runs. **Türkçe:** Kaynak: stream'in nereden geldiği. Intermediate operations: stream'yi başka bir taneye dönüştürür. İstedığınız kadar az veya intermediate operations olabilir. streams lazy evaluation kullandığından, terminal operation çalışana kadar intermediate operations çalışmaz. **English:** ■ ■ Terminal operation: Produces a result. Since streams can be used only once, the stream is no longer valid after a terminal operation completes. **Türkçe:** Terminal operation: Bir sonuç üretir. streams sadece bir kez kullanılabildiğinden, stream bir terminal operation tamamlandıktan sonra artık geçerli değildir. **English:** FIGURE 10.2 Stream pipeline Intermediate operations Source Terminal operation Notice that the operations are unknown to us. When viewing the assembly line from the outside, you care only about what comes in and goes out. What happens in between is an implementation detail. **Türkçe:** FIGURE 10.2 Stream pipeline Intermediate operations Kaynak Terminal operation İşlemlerin bizim için bilinmediğine dikkat edin. Montaj hattını dışarıdan izlerken, sadece içeri giren ve çıkan şeyleri önemlersiniz. Aradaki şey bir uygulama detayıdır. **English:** You will need to know the differences between intermediate and terminal operations well. Make sure you can fill in Table 10.2. **Türkçe:** Ara ve terminal operations arasındaki farkları iyi bilmeniz gerekir. Tablo 10.2'yi doldurabildiğinizden emin olun. **English:** TABLE 10.2 Intermediate vs. terminal operations Scenario Intermediate operation Terminal operation Required part of useful pipeline? No Yes Can exist multiple times in pipeline? Yes No Return type is stream type? Yes No Executed upon method call? No Yes Stream valid after call? Yes No **Türkçe:** TABLE 10.2 Orta vs. terminal operations Senaryo Intermediate operation Terminal operation Kullanışlı pipeline parçası gerekli mi? Hayır Evet pipeline içinde birden fazla kez var olabilir mi? Evet Hayır Return type stream türü mü? Evet Hayır Metodu Çağrısı Üzerine Yürütüldü? Hayır Evet Stream aramadan sonra geçerli mi? Evet Hayır

English: A factory typically has a foreperson who oversees the work. Java serves as the foreperson when working with stream pipelines. This is a really important role, especially when dealing with lazy evaluation and infinite streams. Think of declaring the stream as giving instructions to the foreperson. As the foreperson finds out what needs to be done, they set up the stations and tell the workers what their duties will be. However, the workers do not start until the foreperson tells them to begin. The foreperson waits until they see the terminal operation to kick off the work. They also watch the work and stop the line as soon as work is complete. **Türkçe:** Bir fabrikada tipik olarak işi denetleyen bir öncü bulunur. Java, stream pipelines ile çalışırken ön kişi olarak hizmet eder. Bu, özellikle lazy evaluation ve infinite streams ile uğraşırken gerçekten önemli bir roldür. stream ibaresini öninsana talimat vermek olarak ilan etmeyi düşünün. Öngörülen kişi ne yapılması gerektiğini öğrenirken, istasyonları set yükseltir ve işçilere görevlerinin ne olacağını söyler. Bununla birlikte, işçiler, öngösterici onlara başlamalarını söyleyene kadar başlamazlar. Öngörülen kişi, işi başlatmak için terminal operation 'ı görene kadar bekler. Ayrıca işi izlerler ve iş tamamlanır tamamlanmaz hattı durdururlar. **English:** Let's look at a few examples of this. We aren't using code in these examples because it is really important to understand the stream pipeline concept before starting to write the code. Figure 10.3 shows a stream pipeline with one intermediate operation.

Türkçe: Bunun birkaç örneğine bakalım. Bu örneklerde kod kullanmıyoruz, çünkü kodu yazmaya başlamadan önce stream pipeline kavramını anlamak gerçekten önemlidir. Şekil 10.3, bir intermediate operation ile bir stream pipeline gösterir. **English:** FIGURE 10.3 Steps in running a stream pipeline Intermediate Take sign Put sign operations out of box in pile Paint sign 1 2 3 4 5 6 Let's take a look at what happens from the point of view of the foreperson. First, they see that the source is taking signs out of the box. The foreperson sets up a worker at the table to unpack the box and says to await a signal to start. Then the foreperson sees the intermediate operation to paint the sign. They set up a worker with paint and say to await a signal to start. Finally, the foreperson sees the terminal operation to put the signs into a pile. They set up a worker to do this and yell that all three workers should start. **Türkçe:** FIGURE 10.3 Bir stream pipeline Intermediate Take işareti'nin çalıştırılmasında adımlar İşaret işlemlerini kutunun dışına koyun Kazık içinde boya işareti 1 2 3 4 5 6 Öndeki kişinin bakış açısından neler olduğuna bir göz atalım. İlk olarak, kaynağın kutudan işaretler aldığını görürler. Ön kişi sets kutuyu açmak için masadaki bir işçiye yukarı kaldırır ve başlamak için bir sinyal beklemesini söyler. Daha sonra ön kişi işareti boyamak için intermediate operation değerini görür. Boyalı bir işçi kurdular ve başlamak için bir sinyal beklemelerini söylediler. Son olarak, ön kişi işaretleri bir yığına koymak için terminal operation değerini görür. Bunu yapmak için bir işçi kurdular ve üç işçinin de başlaması gerektiğini haykırdılar. **English:** Suppose that there are two signs in the box. Step 1 is the first worker taking one sign out of the box and handing it to the second worker. Step 2 is the second worker painting it and handing it to the third worker. Step 3 is the third worker putting it in the pile. Steps 4–6 are this same process for the other sign. Then the foreperson sees that there are no signs left and shuts down the entire enterprise. **Türkçe:** Kutuda iki işaret olduğunu varsayalım. Adım 1, kutudan bir işaret alan ve ikinci işçiye veren ilk işçidir. Adım 2, onu resmeden ve üçüncü işçiye veren ikinci işçidir. Adım 3, onu yığının içine koyan üçüncü işçidir. 4-6. adımlar diğer işaret için de aynı işlemdir. O zaman öndeki kişi hiçbir işaretin kalmadığını görür ve tüm işletmeyi kapatır.

English: The foreperson is smart and can make decisions about how to best do the work based on what is needed. As an example, let's explore the stream pipeline in Figure 10.4. **Türkçe:** Öngörülen kişi akıllıdır ve işin en iyi şekilde nasıl yapılacağına dair kararlar alabilir. Örnek olarak, Şekil 10.4'teki stream pipeline'u inceleyelim. **English:** FIGURE 10.4 A stream pipeline with a limit Intermediate Take sign Put sign operations out of box in pile Paint sign Only do 2 signs **Türkçe:** FIGURE 10.4 A stream pipeline ile bir limit Intermediate Take işareti Yığın içinde işaret işlemlerini kutunun dışına koyun Boya işareti Sadece 2 işaret yapın

English: The foreperson still sees a source of taking signs out of the box and assigns a worker to do that on command. They still see an intermediate operation to paint and set up another worker with instructions to wait and then paint. Then they see an intermediate step that we need only two signs. They set up a worker to count the signs that go by and notify the foreperson when the worker has seen two. Finally, they set up a worker for the terminal operation to put the signs in a pile. **Türkçe:** Öngörülen kişi hala işaretlerin kutudan çıkarılmasının bir kaynağını görür ve bunu komutla yapması için bir işçi görevlendirir. Hala boyamak için bir intermediate operation ve beklemek ve daha sonra boyamak için talimatlarla başka bir işçiyi yukarı çıkarmak için set görüyorlar. Sonra sadece iki işarete ihtiyaç duyduğumuz bir ara adım görüyorlar. set bir işçiyi, geçen işaretleri saymak ve işçi iki tane gördüğünde ön kişiyi bilgilendirmek için yukarı çıkarırlar. Sonunda, işaretleri bir yığına koymak için terminal operasyonu için bir işçi kurdular.

English: This time, suppose that there are 10 signs in the box. We start like last time. The first sign makes its way down the pipeline. The second sign also makes its way down the pipeline. When the worker in charge of counting sees the second sign, they tell the foreperson. The foreperson lets the terminal operation worker finish their task and then yells, "Stop the line." It doesn't matter that there are eight more signs in the box. We don't need them, so it would be unnecessary work to paint them. And we all want to avoid unnecessary work! **Türkçe:** Bu sefer kutuda 10 işaret olduğunu varsayalım. Geçen seferki gibi başlıyoruz. İlk işaret pipeline aşağı doğru ilerler. İkinci işaret de pipeline aşağı doğru ilerler. Saymakla görevli işçi ikinci işareti gördüğünde, öndeki kişiye haber verirler. Öngörülen kişi, terminal operation çalışanının görevlerini tamamlamasına izin verir ve ardından "Stop the line" diye bağırır. Kutuda sekiz işaretin daha olması önemli değildir. Onlara ihtiyacımız yok, bu yüzden onları boyamak gereksiz bir iş olurdu. Ve hepimiz gereksiz işlerden kaçınmak istiyoruz! **English:** Similarly, the foreperson would have stopped the line after the first sign if the terminal operation was to find the first sign that gets created.

Türkçe: Benzer şekilde, terminal operation oluşturulan ilk işareti bulmak için olsaydı, ön kişi ilk işareten sonra çizgiyi durdururdu. **English:** In the following sections, we cover the three parts of the pipeline. We also discuss special types of streams for primitives and how to print a stream. **Türkçe:** Aşağıdaki bölümlerde, pipeline 'nin üç bölümünü kapsarız. Ayrıca, ilkeller için özel streams türlerini ve bir stream nasıl yazdırılacağını da tartışıyoruz.

Creating Stream Sources

English: In Java, the streams we have been talking about are represented by the `Stream<T>` interface, defined in the `java.util.stream` package. **Türkçe:** Java'da, bahsettiğimiz streams, `java.util.stream` paketinde tanımlanan `Stream<T>` arayüzü ile temsil edilir.

Creating Finite Streams

English: For simplicity, we start with finite streams. There are a few ways to create them. **Türkçe:** Basitlik için finite streams ile başlıyoruz. Onları yaratmanın birkaç yolu vardır.

```
Stream<String> empty = Stream.empty();
// count = 0
Stream<Integer> singleElement = Stream.of(1); // count = 1
Stream<Integer> fromArray = Stream.of(1, 2, 3); // count = 3
```

English: Line 11 shows how to create an empty stream. Line 12 shows how to create a stream with a single element. Line 13 shows how to create a stream from a varargs. **Türkçe:** Satır 11 boş stream nasıl oluşturulacağını gösterir. 12. satır, tek bir elemanla stream nasıl oluşturulacağını gösterir. Satır 13, bir varargdan stream nasıl oluşturulacağını gösterir. **English:** Java also provides a convenient way of converting a Collection to a stream. **Türkçe:** Java ayrıca bir Collection dosyasını stream konumuna dönüştürmenin uygun bir yolunu sunar.

```
var list = List.of("a", "b", "c");  
Stream<String> fromList = list.stream();
```

English: Line 15 shows that it is a simple method call to create a stream from a list. This is helpful since such conversions are common. **Türkçe:** Satır 15, bir list 'den bir stream oluşturmak için basit bir yöntem çağrısı olduğunu gösterir. Bu tür dönüşümler yaygın olduğu için yararlıdır. **English:** Creating a Parallel Stream It is just as easy to create a parallel stream from a list. **Türkçe:** Bir Parallel Stream oluşturmak, bir list 'den bir parallel stream oluşturmak kadar kolaydır.

```
var list = List.of("a", "b", "c");  
Stream<String> fromListParallel = list.parallelStream();
```

English: This is a great feature because you can write code that uses concurrency before even learning what a thread is. Using parallel streams is like setting up multiple tables of workers who can do the same task. Painting would be a lot faster if we could have five painters painting signs instead of just one. Just keep in mind some tasks cannot be done in parallel, such as putting the signs away in the order that they were created in the stream. Also be aware that there is a cost in coordinating the work, so for smaller streams, it might be faster to do it sequentially. You learn much more about running tasks concurrently in Chapter 13, "Concurrency." **Türkçe:** Bu harika bir özelliktir, çünkü bir ipliğin ne olduğunu öğrenmeden önce eşzamanlılığı kullanan kod yazabilirsiniz. parallel streams kullanmak, aynı görevi yapabilen birden fazla işçi tablosu oluşturmak gibidir. Sadece bir tane yerine beş ressamın tabelalarını boyaması çok daha hızlı olurdu. Sadece bazı görevlerin paralel olarak yapılamayacağını unutmayın, örneğin işaretleri stream içinde oluşturuldukları sıraya koymak gibi. Ayrıca, işin koordine edilmesinde bir maliyet olduğunu unutmayın, bu nedenle daha küçük streams için, sıralı olarak yapmak daha hızlı olabilir. Bölüm 13'te eşzamanlı olarak görevleri yürütme hakkında çok daha fazla bilgi edirsiniz, "Karşılıklık".

Creating Infinite Streams

English: So far, this isn't particularly impressive. We could do all this with lists. We can't create an infinite list, though, which makes streams more powerful. **Türkçe:** Şu ana kadar, bu özellikle etkileyici değil. Tüm bunları lists ile yapabiliriz. Sonsuz bir list yaratamayız, bu da streams'yi daha güçlü yapar.

```
Stream<Double> randomness = Stream.generate(Math::random);  
Stream<Integer> oddNumbers = Stream.iterate(1, n -> n + 2);
```

English: Line 17 generates a stream of random numbers. How many random numbers? However many you need. If you call `randoms.forEach(System.out::println)`, the program will print random numbers until you kill it. Later in the chapter, you learn about operations like `limit()` to turn the infinite stream into a finite stream. **Türkçe:** Satır 17, rastgele sayıların bir stream'ünü oluşturur. Kaç tane rastgele sayı var? Ne kadarına ihtiyacınız olursa. `randoms.forEach(System.out::println)` çağırırsanız, program siz öldürene kadar rastgele numaralar yazdırır. Bölümün ilerleyen bölümlerinde, `limit()` gibi işlemleri infinite stream 'yi finite stream'ye dönüştürmek için öğrenirsiniz. **English:** Line 18 gives you more control. The `iterate()` method takes a seed or starting value as the first parameter. This is the first element that will be part of the stream. The other parameter is a lambda expression that is passed the previous value and generates the next value. As with the random numbers example, it will keep on producing odd numbers as long as you need them. **Türkçe:** 18. hat size daha fazla kontrol sağlar. `iterate()` yöntemi ilk parametre olarak bir tohum veya başlangıç değeri alır. Bu, stream 'in bir parçası olacak ilk elemandır. Diğer parametre, bir önceki değerden geçen ve bir sonraki değeri oluşturan bir lambda ifadesidir. Rastgele sayılar örneğinde olduğu gibi, ihtiyacınız olan tek sayıları long olarak üretmeye devam edecektir. **English:** Printing a Stream Reference **Türkçe:** Stream Referans Yazdırma

If you try to call `System.out.print(stream)`, you'll get something like the following:

English: `java.util.stream.ReferencePipeline$3@4517d9a3` This is different from a Collection, where you see the contents. You don't need to know this for the exam. We mention it so that you aren't caught by surprise when writing code for practice. **Türkçe:** `java.util.stream.ReferencePipeline$3@4517d9a3` Bu, içeriği gördüğünüz bir Collection 'den farklıdır. Sınav için bunu bilmenize gerek yok. Uygulama için kod yazarken sürpriz bir şekilde yakalanmamanız için bundan söz ediyoruz. **English:** What if you wanted just odd numbers less than 100? There's an overloaded version of `iterate()` that helps. **Türkçe:** Ya 100'den daha az tek sayılar isteseydiniz? Yardımcı olan `iterate()` aşırı yüklenmiş bir sürümü var:

```
Stream<Integer> oddNumberUnder100 = Stream.iterate(
1, // seed
```

```
n -> n < 100, // Predicate to specify when done
n -> n + 2); // UnaryOperator to get next value
```

English: This method takes three parameters. Notice how they are separated by commas (,) just like in all other methods. The exam may try to trick you by using semicolons since it is similar to a for loop. Similar to a for loop, you have to take care that you aren't accidentally creating an infinite stream. **Türkçe:** Bu yöntem üç parametre gerektirir. Diğer tüm yöntemlerde olduğu gibi virgüllerle (,) nasıl ayrıldıklarına dikkat edin. Sınav, döngü için bir benzer olduğu için virgül kullanarak sizi kandırmaya çalışabilir. Bir döngü için benzer şekilde, yanlışlıkla bir infinite stream oluşturmadığınıza dikkat etmeniz gerekir.

Reviewing Stream Creation Methods

English: To review, make sure you know all the methods in Table 10.3. These are the ways of creating a source for streams, given a Collection instance `coll`. **Türkçe:** İncelemek için Tablo 10.3'teki tüm yöntemleri bildiğinizden emin olun. Bunlar streams için bir kaynak oluşturma yollarıdır, bir Collection örnek `coll` verilir. **English:** TABLE 10.3 Creating a source Finite Method or infinite? Notes **Türkçe:** TABLE 10.3 Kaynak Sonlu Yöntem mi yoksa sonsuz mu Yaratmak? Kaynakça

```
Stream.empty() Finite Creates Stream with zero elements.
Stream.of(varargs) Finite Creates Stream with elements listed.
```

English: coll.stream() Finite Creates Stream from Collection. **Türkçe:** coll.stream() Sonlu Collection dan Stream oluşturur. **English:** coll.parallelStream() Finite Creates Stream from Collection where the stream can run in parallel. **Türkçe:** coll.parallelStream () Sonlu, akışın paralel olarak çalışabileceği Koleksiyondan Akış Oluşturur. **English:** Stream. **Türkçe:** Stream. **English:** Infinite Creates Stream by calling Supplier for each element upon request. generate(supplier) **Türkçe:** Infinite, istek üzerine her eleman için Supplier diyerek Stream oluşturur. generate(supplier) **English:** Stream.iterate(seed, Infinite Creates Stream by using seed for first element and then calling UnaryOperator for each unaryOperator) **Türkçe:** Stream.iterate(seed, Infinite Creates Stream by using seed for first element and then calling UnaryOperator for each unaryOperator) **English:** subsequent element upon request. **Türkçe:** talep üzerine sonraki eleman. **English:** Finite or Stream.iterate(seed, Creates Stream by using seed for first element infinite and then calling UnaryOperator for each predicate, subsequent element upon request. Stops if unaryOperator) **Türkçe:** Sonlu veya Stream.iterate(seed, Creates Stream by using seed for first element infinite and then calling UnaryOperator for each predicate, subsequent element upon request. Stops if unaryOperator) **English:** Predicate returns false. **Türkçe:** Predicate yanlış döndürür.

Using Common Terminal Operations

English: You can perform a terminal operation without any intermediate operations but not the other way around. This is why we talk about terminal operations first. Reductions are a special type of terminal operation where all of the contents of the stream are combined into a single primitive or Object. For example, you might have an int or a Collection. **Türkçe:** Herhangi bir intermediate operations olmadan bir terminal operation gerçekleştirebilirsiniz, ancak tam tersi olmaz. Bu yüzden önce terminal operations hakkında konuşuyoruz. Reductions, stream içeriğinin tümünün tek bir ilkel veya Nesne olarak bir araya getirildiği özel bir terminal operation türüdür. Örneğin, bir int veya Collection olabilir.

English: Table 10.4 summarizes this section. Feel free to use it as a guide to remember the most important points as we go through each one individually. We explain them from simplest to most complex rather than alphabetically. **Türkçe:** Tablo 10.4 bu bölümü özetler. Her birinden bireysel olarak geçerken en önemli noktaları hatırlamak için bir rehber olarak kullanmaktan çekinmeyin. Alfabetik olarak değil, en basitten en karmaşıka kadar açıklarız. **English:** TABLE 10.4 Terminal stream operations Method What happens for infinite streams Return value Reduction **Türkçe:** TABLE 10.4 Terminal stream işlemleri Yöntem infinite streams Dönüş değeri Reduction için ne olur

```
count() Does not terminate long Yes
```

English: min() **Türkçe:** min() **English:** Does not terminate Optional<T> Yes max() **Türkçe:** Optional<T> Evet max() sonlandırmaz **English:** findAny() Terminates Optional<T> No findFirst() **Türkçe:** findAny() Optional<T> Hayır findFirst() Sonlandırır **English:** allMatch() Sometimes terminates boolean No anyMatch() noneMatch() **Türkçe:** allMatch() Bazen boolean Hayır anyMatch() noneMatch() sona erer

```
forEach() Does not terminate void No
reduce() Does not terminate Varies Yes
collect() Does not terminate Varies Yes
```

Counting

English: The `count()` method determines the number of elements in a finite stream. For an infinite stream, it never terminates. Why? Count from 1 to infinity, and let us know when you are finished. Or rather, don't do that, because we'd rather you study for the exam than spend the rest of your life counting. The `count()` method is a reduction because it looks at each element in the stream and returns a single value. The method signature is as follows: **Türkçe:** `count()` yöntemi, bir finite stream içindeki eleman sayısını belirler. Bir infinite stream için, asla sona ermez. Neden sordun? 1'den sonsuza kadar say ve bitirdiğinde bize haber ver. Daha doğrusu, bunu yapmayın, çünkü hayatınızın geri kalanını saymak yerine sınav için çalışmanızı tercih ederiz. `count()` yöntemi bir reduction yöntemidir, çünkü stream içindeki her elemana bakar ve tek bir değer döndürür. method signature aşağıdaki gibidir:

```
public long count()
```

English: This example shows calling `count()` on a finite stream: **Türkçe:** Bu örnek, finite stream üzerinde `count()` çağrısını gösterir:

```
Stream<String> s = Stream.of("monkey", "gorilla", "bonobo");  
System.out.println(s.count()); // 3
```

Finding the Minimum and Maximum

English: The `min()` and `max()` methods allow you to pass a custom comparator and find the smallest or largest value in a finite stream according to that sort order. Like the `count()` method, **Türkçe:** `min()` ve `max()` yöntemleri, özel bir comparator geçirmenizi ve bu sıralama düzenine göre bir finite stream içindeki en küçük veya en büyük değeri bulmanızı sağlar. `count()` yöntemi gibi,

English: `min()` and `max()` hang on an infinite stream because they cannot be sure that a smaller or larger value isn't coming later in the stream. Both methods are reductions because they return a single value after looking at the entire stream. The method signatures are as follows: **Türkçe:** `min()` ve `max()` bir infinite stream tutun, çünkü daha küçük veya daha büyük bir değer stream içinde daha sonra gelmeyeceğinden emin olamazlar. Her iki yöntem de reductions dir, çünkü stream nın tamamına baktıktan sonra tek bir değer döndürürler. method signatures aşağıdaki gibidir:

```
public Optional<T> min(Comparator<? super T> comparator)  
public Optional<T> max(Comparator<? super T> comparator)
```

English: This example finds the animal with the fewest letters in its name: **Türkçe:** Bu örnek, adında en az harf olan hayvanı bulur:

```
Stream<String> s = Stream.of("monkey", "ape", "bonobo");  
Optional<String> min = s.min((s1, s2) -> s1.length() - s2.length());  
min.ifPresent(System.out::println); // ape
```

English: Notice that the code returns an Optional rather than the value. This allows the method to specify that no minimum or maximum was found. We use the Optional method `ifPresent()` and a method reference to print out the minimum only if one is found. As an example of where there isn't a minimum, let's look at an empty stream: **Türkçe:** Kodun değerden ziyade bir Optional döndürdüğüne dikkat edin. Bu, yöntemin minimum veya maksimum bulunmadığını belirtmesine izin verir. Optional metodunu `ifPresent()` ve method reference metodunu kullanarak sadece bir tane bulunursa minimum çıktığı çıkarıyoruz. Minimumun olmadığı bir yere örnek olarak, boş bir stream kısmına bakalım:

```
Optional<?> minEmpty = Stream.empty().min((s1, s2) -> 0);
System.out.println(minEmpty.isPresent()); // false
```

English: Since the stream is empty, the comparator is never called, and no value is present in the Optional. **Türkçe:** stream boş olduğundan, comparator asla çağrılmaz ve Optional içinde hiçbir değer bulunmaz.

What if you need both the `min()` and `max()` values of the same stream?

English: For now, you can't have both, at least not using these methods. **Türkçe:** Şimdilik her ikisine de sahip olamazsınız, en azından bu yöntemleri kullanamazsınız. **English:** Remember, a stream can have only one terminal operation. Once a terminal operation has been run, the stream cannot be used again. As you see later in this chapter, there are built-in summary methods for some numeric streams that will calculate a set of values for you. **Türkçe:** Unutmayın, bir stream yalnızca bir terminal operation içerebilir. terminal operation çalıştırdıktan sonra stream tekrar kullanılamaz. Bu bölümde daha sonra gördüğünüz gibi, sizin için bir set değer hesaplayacak bazı sayısal streams için yerleşik özet yöntemleri vardır.

Finding a Value

English: The `findAny()` and `findFirst()` methods return an element of the stream unless the stream is empty. If the stream is empty, they return an empty Optional. This is the first method you've seen that can terminate with an infinite stream. Since Java generates only the amount of stream you need, the infinite stream needs to generate only one element. **Türkçe:** `findAny()` ve `findFirst()` yöntemleri, stream boş olmadıkça stream ögesini döndürür. stream boşsa, boş bir Optional döndürürler. Bu, bir infinite stream ile sonlandırılabilen gördüğümüz ilk yöntemdir. Java sadece ihtiyacınız olan stream miktarını ürettiğinden, infinite stream yalnızca bir eleman oluşturması gerekir. **English:** As its name implies, the `findAny()` method can return any element of the stream. When called on the streams you've seen up until now, it commonly returns the first element, although this behavior is not guaranteed. As you see in Chapter 13, the `findAny()` method is more likely to return a random element when working with parallel streams.

Türkçe: Adından da anlaşılacağı gibi, `findAny()` yöntemi stream ögesinin herhangi bir elemanını döndürebilir. Şimdiye kadar gördüğümüz streams üzerinde çağrıldığında, bu davranış garanti edilmese de, genellikle ilk ögeyi döndürür. Bölüm 13'te gördüğümüz gibi, `findAny()` yöntemi parallel streams ile çalışırken rastgele bir ögeyi döndürme olasılığı daha yüksektir. **English:** These methods are terminal operations but not reductions. The reason is that they sometimes return without processing all of the elements. This means that they return a value based on the stream but do not reduce the entire stream into one value. **Türkçe:** Bu yöntemler terminal operations ama reductions değildir. Bunun nedeni, bazen tüm unsurları işlemeyen geri dönmeleridir. Bu, stream tabanlı bir değeri döndürdükleri anlamına gelir, ancak stream nin tamamını tek bir değere indirgemezler. **English:** The method signatures are as follows: **Türkçe:** method signatures aşağıdaki gibidir:

```
public Optional<T> findAny()
public Optional<T> findFirst()
```

English: This example finds an animal: **Türkçe:** Bu örnek bir hayvanı bulur:

```
Stream<String> s = Stream.of("monkey", "gorilla", "bonobo");
Stream<String> infinite = Stream.generate(() -> "chimp");
s.findAny().ifPresent(System.out::println); // monkey (usually)
infinite.findAny().ifPresent(System.out::println); // chimp
```

English: Finding any one match is more useful than it sounds. Sometimes we just want to sample the results and get a representative element, but we don't need to waste the processing generating them all. After all, if we plan to work with only one element, why bother looking at more? **Türkçe:** Herhangi bir eşleşmeyi bulmak, görüldüğünden daha yararlıdır. Bazen sadece sonuçları örneklemek ve temsili bir element elde etmek istiyoruz, ancak hepsini üreten işlemi boşa harcamamıza gerek yok. Sonuçta, eğer tek bir unsurla çalışmayı planlıyorsak, neden daha fazla bakmakla uğraşalım?

Matching

English: The `allMatch()`, `anyMatch()`, and `noneMatch()` methods search a stream and return information about how the stream pertains to the predicate. These may or may not terminate for infinite streams. It depends on the data. Like the find methods, they are not reductions because they do not necessarily look at all of the elements. **Türkçe:** `allMatch()`, `anyMatch()` ve `noneMatch()` yöntemleri bir stream arar ve stream'nin predicate ile nasıl ilişkili olduğu hakkında bilgi döndürür. Bunlar infinite streams için sonlandırılabilir veya sonlandırılmaz. Verilere göre değişir. Bulma yöntemleri gibi, bunlar da reductions değildir, çünkü mutlaka tüm elementlere bakmazlar. **English:** The method signatures are as follows: **Türkçe:** method signatures aşağıdaki gibidir:

```
public boolean anyMatch(Predicate <? super T> predicate)
public boolean allMatch(Predicate <? super T> predicate)
public boolean noneMatch(Predicate <? super T> predicate)
```

English: This example checks whether animal names begin with letters: **Türkçe:** Bu örnek, hayvan adlarının harflerle başlayıp başlamadığını kontrol eder:

```
var list = List.of("monkey", "2", "chimp");
Stream<String> infinite = Stream.generate(() -> "chimp");
Predicate<String> pred = x -> Character.isLetter(x.charAt(0));
System.out.println(list.stream().anyMatch(pred)); // true
System.out.println(list.stream().allMatch(pred)); // false
System.out.println(list.stream().noneMatch(pred)); // false
System.out.println(infinite.anyMatch(pred)); // true
```

English: This shows that we can reuse the same predicate, but we need a different stream each time. The `anyMatch()` method returns true because two of the three elements match. The `allMatch()` method returns false because one doesn't match. The `noneMatch()` method also returns false because at least one matches. On the infinite stream, one match is found, so the call terminates. If we called `allMatch()`, it would run until we killed the program. **Türkçe:** Bu, aynı predicate 'i tekrar kullanabileceğimizi gösterir, ancak her seferinde farklı bir stream gerekir. `anyMatch()` yöntemi doğru olarak döner çünkü üç elementten ikisi eşleşir. `allMatch()` yöntemi yanlış döner, çünkü biri eşleşmez. `noneMatch()` yöntemi de en az bir eşleşme olduğu için yanlış döner. infinite stream üzerinde bir eşleşme bulunur, bu nedenle çağrı sona erer. `allMatch()`'u arasaydık, programı öldürene kadar çalışırdı.

Remember that `allMatch()`, `anyMatch()`, and `noneMatch()` return a

English: boolean. By contrast, the find methods return an Optional because they return an element of the stream. **Türkçe:** boolean. Buna karşılık, bulma yöntemleri bir Optional döndürür, çünkü stream öğesini döndürürler.

Iterating

English: As in the Java Collections Framework, it is common to iterate over the elements of a stream. As expected, calling `forEach()` on an infinite stream does not terminate. Since there is no return value, it is not a reduction. **Türkçe:** Java Collections Framework'te olduğu gibi, bir stream elemanı üzerinde yinelemek yaygındır. Beklendiği gibi, bir infinite stream üzerinde `forEach()` araması sona ermez. Geri dönüş değeri olmadığından, reduction değildir. **English:** Before you use it, consider if another approach would be better. Developers who learned to write loops first tend to use them for everything. For example, a loop with an if statement could be written with a filter. You will learn about filters in the intermediate operations section. **Türkçe:** Kullanmadan önce, başka bir yaklaşımın daha iyi olup olmayacağını düşünün. Döngü yazmayı öğrenen geliştiriciler ilk önce bunları her şey için kullanma eğilimindedirler. Örneğin, bir ifade ile bir döngü bir filtre ile yazılabilir. intermediate operations bölümündeki filtreler hakkında bilgi edineceksiniz. **English:** The method signature is as follows: **Türkçe:** method signature aşağıdaki gibidir:

```
public void forEach(Consumer<? super T> action)
```

English: Notice that this is the only terminal operation with a return type of void. If you want something to happen, you have to make it happen in the Consumer. Here's one way to print the elements in the stream (there are other ways, which we cover later in the chapter): **Türkçe:** Bunun void return type ile tek terminal operation olduğuna dikkat edin. Bir şeyin olmasını istiyorsanız, bunun Consumer içinde gerçekleşmesini sağlamalısınız. İşte stream içindeki öğeleri yazdırmanın bir yolu (bölümün ilerleyen bölümlerinde ele aldığımız başka yollar da var):

```
Stream<String> s = Stream.of("Monkey", "Gorilla", "Bonobo");
s.forEach(System.out::print); // MonkeyGorillaBonobo
Remember that you can call forEach() directly on a Collection or on a
```

English: Stream. Don't get confused on the exam when you see both approaches. **Türkçe:** Stream. Her iki yaklaşımı da gördüğünüzde sınavda kafanızı karıştırmayın. **English:** Notice that you can't use a traditional for loop on a stream. **Türkçe:** Geleneksel bir döngüyü stream üzerinde kullanamayacağınıza dikkat edin.

```
Stream<Integer> s = Stream.of(1);  
for (Integer i: s) {} // DOES NOT COMPILE
```

English: While `forEach()` sounds like a loop, it is really a terminal operator for streams. Streams cannot be used as the source in a for-each loop because they don't implement the `Iterable` interface. **Türkçe:** `forEach()` bir döngü gibi görünse de, gerçekten streams için bir terminal operatörüdür. Streams her döngü için kaynak olarak kullanılamaz çünkü `Iterable` arayüzünü uygulamazlar.

Reducing

English: The `reduce()` method combines a stream into a single object. It is a reduction, which means it processes all elements. The three method signatures are these: **Türkçe:** `reduce()` yöntemi bir stream nesnesini tek bir nesnede birleştirir. Bu bir reduction, yani tüm elementleri işler. Üç method signatures şunlardır:

```
public T reduce(T identity, BinaryOperator<T> accumulator)  
public Optional<T> reduce(BinaryOperator<T> accumulator)
```

English: `public <U> U reduce(U identity, BiFunction<U,? super T,U> accumulator, BinaryOperator<U> combiner)` **Türkçe:** genel `U> U reduce(U identity, BiFunction<U,? super T,U> accumulator, BinaryOperator<U> combiner)` **English:** Let's take them one at a time. The most common way of doing a reduction is to start with an initial value and keep merging it with the next value. Think about how you would concatenate an array of `String` objects into a single `String` without functional programming. **Türkçe:** Onları birer birer ele alalım. Bir reduction yapmanın en yaygın yolu, bir başlangıç değeri ile başlamak ve bir sonraki değerle birleştirmeye devam etmektir. İşlevsel programlama olmadan bir dizi `String` nesnesini tek bir `String` nesnesine nasıl birleştireceğinizi düşünün.

English: It might look something like this: **Türkçe:** Şöyle bir şeye benzeyebilir:

```
var array = new String[] { "w", "o", "l", "f" };  
var result = "";  
for (var s: array) result = result + s;  
System.out.println(result); // wolf
```

English: The identity is the initial value of the reduction, in this case an empty `String`. The accumulator combines the current result with the current value in the stream. With lambdas, we can do the same thing with a stream and reduction: **Türkçe:** identity, reduction nin başlangıç değeridir, bu durumda boş `String` olur. accumulator geçerli sonucu stream içindeki geçerli değerle birleştirir. lambdas ile aynı şeyi stream ve reduction ile yapabiliriz:

```
Stream<String> stream = Stream.of("w", "o", "l", "f");  
String word = stream.reduce("", (s, c) -> s + c);  
System.out.println(word); // wolf
```

English: Notice how we still have the empty String as the identity. We also still concatenate the String objects to get the next value. We can even rewrite this with a method reference: **Türkçe:** identity olarak hala boş String olduğuna dikkat edin. Ayrıca bir sonraki değeri elde etmek için String nesnelerini bir araya getiriyoruz. Bunu bir method reference ile bile yeniden yazabiliriz:

```
Stream<String> stream = Stream.of("w", "o", "l", "f");
String word = stream.reduce("", String::concat);
System.out.println(word); // wolf
```

English: Let's try another one. Can you write a reduction to multiply all of the Integer objects in a stream? Try it. Our solution is shown here: **Türkçe:** Bir tane daha deneyelim. Integer nesnelerinin tümünü stream ile çarpmak için reduction yazabilir misiniz? Dene. Çözümümüz burada gösterilmiştir:

```
Stream<Integer> stream = Stream.of(3, 5, 6);
System.out.println(stream.reduce(1, (a, b) -> a*b)); // 90
```

English: We set the identity to 1 and the accumulator to multiplication. In many cases, the identity isn't really necessary, so Java lets us omit it. When you don't specify an identity, an Optional is returned because there might not be any data. There are three choices for what is in the Optional: ■ ■ If the stream is empty, an empty Optional is returned. ■ ■ If the stream has one element, it is returned. ■ ■ If the stream has multiple elements, the accumulator is applied to combine them. **Türkçe:** Biz set identity ile 1 ve accumulator ile çarpma işlemini yapıyoruz. Birçok durumda, identity gerçekten gerekli değildir, bu nedenle Java bunu atlamamıza izin verir. Bir identity belirtmediğinizde, herhangi bir veri olmadığı için bir Optional döndürülür. Optional içindekiler için üç Optional vardır: stream boşsa, boş Optional döndürülür. stream bir elemana sahipse, geri döndürülür. stream birden fazla elemana sahipse, bunları birleştirmek için accumulator uygulanır. **English:** The following illustrates each of these scenarios: **Türkçe:** Aşağıdakiler, bu senaryoların her birini göstermektedir:

```
BinaryOperator<Integer> op = (a, b) -> a * b;
Stream<Integer> empty = Stream.empty();
Stream<Integer> oneElement = Stream.of(3);
Stream<Integer> threeElements = Stream.of(3, 5, 6);
empty.reduce(op).ifPresent(System.out::println); // no output
oneElement.reduce(op).ifPresent(System.out::println); // 3
threeElements.reduce(op).ifPresent(System.out::println); // 90
```

English: Why are there two similar methods? Why not just always require the identity? Java could have done that. However, sometimes it is nice to differentiate the case where the stream is empty rather than the case where there is a value that happens to match the identity being returned from the calculation. The signature returning an Optional lets us differentiate these cases. For example, we might return `Optional.empty()` when the stream is empty and `Optional.of(3)` when there is a value. **Türkçe:** Neden iki benzer yöntem var? Neden her zaman identity gerekmez? Java bunu yapabiliirdi. Bununla birlikte, bazen hesaplamadan döndürülen identity ile eşleşen bir değerin bulunduğu durumdan ziyade stream boş olduğu durumu ayırt etmek iyidir. Bir Optional döndüren imza, bu durumları ayırt etmemizi sağlar. Örneğin, stream boşken ve `Optional.of(3)` bir değer olduğunda `Optional.empty()` döndürebiliriz. **English:** The third method signature is used when we are dealing with different types. It allows Java to create intermediate reductions and then combine them at the end. Let's take a look at an example that counts the number of characters in each String: **Türkçe:** Üçüncü method signature farklı türlerle uğraşırken kullanılır. Java ara reductions oluşturmaya ve sonunda birleştirmesine izin verir. Her String içindeki karakter sayısını sayan bir örneğe bakalım:

```
Stream<String> stream = Stream.of("w", "o", "l", "f!");
int length = stream.reduce(0, (i, s) -> i+s.length(), (a, b) -> a+b);
System.out.println(length); // 5
```

English: The first parameter (0) is the value for the initializer. If we had an empty stream, this would be the answer. The second parameter is the accumulator. Unlike the accumulators you saw previously, this one handles mixed data types. In this example, the first argument, `i`, is an Integer, while the second argument, `s`, is a String. It adds the length of the current String to our running total. The third parameter is called the combiner, which combines any intermediate totals. In this case, `a` and `b` are both Integer values. **Türkçe:** İlk parametre (0), başlatıcının değeridir. Eğer boş bir stream olsaydı, bu cevap olurdu. İkinci parametre accumulator dir. Daha önce gördüğümüz accumulators 'den farklı olarak, bu karma veri türlerini işler. Bu örnekte, ilk argüman, `i`, bir Integer iken, ikinci argüman, `s`, bir String'dir. Geçerli String uzunluğunu çalışan toplamımıza ekler. Üçüncü parametre, herhangi bir ara toplamı birleştiren combiner olarak adlandırılır. Bu durumda, `a` ve `b` değerlerinin her ikisi de Integer değerleridir. **English:** The three-argument `reduce()` operation is useful when working with parallel streams because it allows the stream to be decomposed and reassembled by separate threads. For example, if we needed to count the length of four 100-character strings, the first two values and the last two values could be computed independently. The intermediate result (200 + 200) would then be combined into the final value. **Türkçe:** Üç argument `reduce()` işlemi parallel streams ile çalışırken kullanışlıdır, çünkü stream ayrı ipliklerle ayrıştırılıp yeniden birleştirilmesini sağlar. Örneğin, dört 100 karakter strings uzunluğunu saymamız gerekirse, ilk iki değer ve son iki değer bağımsız olarak hesaplanabilir. Ara sonuç (200 + 200) daha sonra nihai değere birleştirilecekti.

Collecting

English: The collect() method is a special type of reduction called a mutable reduction. It is more efficient than a regular reduction because we use the same mutable object while accumulating. Common mutable objects include StringBuilder and ArrayList. This is a really useful method, because it lets us get data out of streams and into another form. The method signatures are as follows: **Türkçe:** collect() yöntemi, reduction olarak adlandırılan özel bir reduction türüdür. Normal bir reduction den daha verimlidir, çünkü biriken aynı değişken nesneyi kullanırız. Yaygın olarak değiştirilebilen nesneler arasında StringBuilder ve ArrayList bulunur. Bu gerçekten kullanışlı bir yöntemdir, çünkü streams ve başka bir formda veri elde etmemizi sağlar. method signatures aşağıdaki gibidir: **English:** public <R> R collect(Supplier<R> supplier, BiConsumer<R,? super T> accumulator, BiConsumer<R, R> combiner) **Türkçe:** halka açık R> R collect(Supplier<R> supplier, BiConsumer<R,? super T> accumulator, BiConsumer<R, R> combiner)

```
public <R,A> R collect(Collector<? super T, A,R> collector)
```

English: Let's start with the first signature, which is used when we want to code specifically how collecting should work. Our wolf example from reduce can be converted to use collect(): **Türkçe:** Toplamanın nasıl çalışması gerektiğini özellikle kodlamak istediğimizde kullanılan ilk imza ile başlayalım. İndirgeyici kurt örneğimiz collect() kullanımına dönüştürülebilir:

```
Stream<String> stream = Stream.of("w", "o", "l", "f");
```

```
StringBuilder word = stream.collect(
    StringBuilder::new,
    StringBuilder::append,
    StringBuilder::append);
System.out.println(word); // wolf
```

English: The first parameter is the supplier, which creates the object that will store the results as we collect data. Remember that a Supplier doesn't take any parameters and returns a value. In this case, it constructs a new StringBuilder. **Türkçe:** İlk parametre, verileri toplarken sonuçları depolayacak nesneyi oluşturan supplier dir. Bir Supplier parametre almadığını ve bir değer döndürdüğünü unutmayın. Bu durumda yeni bir StringBuilder oluşturur. **English:** The second parameter is the accumulator, which is a BiConsumer that takes two parameters and doesn't return anything. It is responsible for adding one more element to the data collection. In this example, it appends the next String to the StringBuilder. **Türkçe:** İkinci parametre, iki parametre alan ve hiçbir şey döndürmeyen bir BiConsumer olan accumulator'dir. collection verisine bir öge daha eklemekten sorumludur. Bu örnekte, StringBuilder'a bir sonraki String ekler. **English:** The final parameter is the combiner, which is another BiConsumer. It is responsible for taking two data collections and merging them. This is useful when we are processing in parallel. Two smaller collections are formed and then merged into one. This would work with StringBuilder only if we didn't care about the order of the letters. In this case, the accumulator and combiner have similar logic. **Türkçe:** Son parametre başka bir BiConsumer olan combiner dir. İki veriyi collections alıp birleştirmekten sorumludur. Bu, paralel olarak işlediğimizde yararlıdır. İki küçük collections oluşur ve daha sonra birleştirilir. Bu, StringBuilder ile yalnızca harflerin sırasını umursamazsak çalışırdı. Bu durumda, accumulator ve combiner benzer bir mantığa sahiptir. **English:** Now let's look at an example where the logic is different in the accumulator and combiner: **Türkçe:** Şimdi accumulator ve combiner 'de mantığın farklı olduğu bir örneğe bakalım:

```
Stream<String> stream = Stream.of("w", "o", "l", "f");
TreeSet<String> set = stream.collect(
    TreeSet::new,
    TreeSet::add,
    TreeSet::addAll);
System.out.println(set); // [f, l, o, w]
```

English: The collector has three parts as before. The supplier creates an empty TreeSet. The accumulator adds a single String from the Stream to the TreeSet. The combiner adds all of the elements of one TreeSet to another in case the operations were done in parallel and need to be merged. **Türkçe:** collector daha önce olduğu gibi üç parçaya sahiptir. supplier boş bir TreeSet oluşturur. accumulator, Stream'den TreeSet'e tek bir String ekler. combiner, işlemlerin paralel olarak yapılması ve birleştirilmesi gerektiğinde bir TreeSet ögesinin tüm unsurlarını bir diğerine ekler. **English:** We started with the long signature because that's how you implement your own collector. It is important to know how to do this for the exam and understand how collectors work. In practice, many common collectors come up over and over. Rather than making developers keep reimplementing the same ones, Java provides a class with common collectors cleverly named Collectors. This approach also makes the code easier to read because it is more expressive. For example, we could rewrite the previous example as follows: **Türkçe:** long imzası ile başladık çünkü kendi collector imzanızı bu şekilde uyguluyorsunuz. Sınav için bunun nasıl yapılacağını bilmek ve collectors nasıl çalıştığını anlamak önemlidir. Pratikte, birçok yaygın collectors tekrar tekrar ortaya çıkar. Geliştiricilerin aynı olanları yeniden uygulamaya devam etmesini sağlamak yerine, Java, Collectors adlı ortak collectors ile bir sınıf sağlar. Bu yaklaşım aynı zamanda kodun okunmasını kolaylaştırır çünkü daha etkileycidir. Örneğin, önceki örneği aşağıdaki gibi yeniden yazabiliriz:

```
Stream<String> stream = Stream.of("w", "o", "l", "f");
```

English: TreeSet<String> set = **Türkçe:** TreeSet<String> set =

```
stream.collect(Collectors.toCollection(TreeSet::new));
System.out.println(set); // [f, l, o, w]
```

English: If we didn't need the set to be sorted, we could make the code even shorter: **Türkçe:** Eğer sıralamak için set 'a ihtiyacımız olmasaydı, kodu daha da kısaltabilirdik:

```
Stream<String> stream = Stream.of("w", "o", "l", "f");
Set<String> set = stream.collect(Collectors.toSet());
System.out.println(set); // [f, w, l, o]
```

English: You might get different output for this last one since `toSet()` makes no guarantees as to which implementation of `Set` you'll get. It is likely to be a `HashSet`, but you shouldn't expect or rely on that.

Türkçe: Bu sonuncu için farklı çıktılar alabilirsiniz, çünkü `toSet()` hangi `Set` uygulamasının alacağını garanti etmez. Bir `HashSet` olması muhtemeldir, ancak bunu beklememeli veya güvenmemelisiniz.

English: The exam expects you to know about common predefined collectors in addition to being able to write your own by passing a supplier, accumulator, and combiner.

Türkçe: Sınav, bir supplier, accumulator ve combiner'yi geçerek kendinizinkini yazabilmenin yanı sıra, yaygın önceden tanımlanmış collectors hakkında bilgi sahibi olmanızı bekler. **English:** Later in this chapter, we show many Collectors that are used for grouping data. It's a big topic, so it's best to master how streams work before adding too many Collectors into the mix. **Türkçe:** Daha sonra bu bölümde, grouping verileri için kullanılan birçok Collectors gösteririz. Bu büyük bir konu, bu yüzden karışımına çok fazla Collectors eklemekten önce streams nasıl çalıştığına hakim olmak en iyisidir.

Using Common Intermediate Operations

English: Unlike a terminal operation, an intermediate operation produces a stream as its result. An intermediate operation can also deal with an infinite stream simply by returning another infinite stream. Since elements are produced only as needed, this works fine. The assembly line worker doesn't need to worry about how many more elements are coming through and instead can focus on the current element.

Türkçe: Bir terminal operation'ın aksine, bir intermediate operation sonucu olarak bir stream üretir. Bir intermediate operation, başka bir infinite stream döndürerek bir infinite stream ile de başa çıkabilir. Elementler sadece gerektiği gibi üretildiğinden, bu iyi çalışır. Montaj hattı işçisinin kaç elementin daha geldiği konusunda endişelenmesine gerek yoktur ve bunun yerine mevcut elemana odaklanabilir.

Filtering

English: The `filter()` method returns a `Stream` with elements that match a given expression. Here is the method signature: **Türkçe:** `filter()` yöntemi, belirli bir ifadeyle eşleşen öğelerle bir `Stream` döndürür. İşte method signature:

```
public Stream<T> filter(Predicate<? super T> predicate)
```

English: This operation is easy to remember and powerful because we can pass any `Predicate` to it. For example, this retains all elements that begin with the letter `m`: **Türkçe:** Bu işlemi hatırlamak kolaydır ve güçlüdür, çünkü herhangi bir `Predicate`'yi ona geçebiliriz. Örneğin, bu `m` harfi ile başlayan tüm öğeleri tutar:

```
Stream<String> s = Stream.of("monkey", "gorilla", "bonobo");
s.filter(x -> x.startsWith("m"))
  .forEach(System.out::print); // monkey
```

Removing Duplicates

English: The `distinct()` method returns a stream with duplicate values removed. The duplicates do not need to be adjacent to be removed. As you might imagine, Java calls `equals()` to determine whether the objects are equivalent. The method signature is as follows: **Türkçe:** `distinct()` yöntemi, kopya değerleri kaldırılmış bir stream döndürür. Kopyaların kaldırılması için bitişik olması gerekmez. Tahmin edebileceğiniz gibi, Java nesnelerin eşdeğer olup olmadığını belirlemek için `equals()` çağırır. method signature aşağıdaki gibidir:

```
public Stream<T> distinct()
```

English: Here's an example: **Türkçe:** İşte bir örnek:

```
Stream<String> s = Stream.of("duck", "duck", "duck", "goose");
s.distinct()
  .forEach(System.out::print); // duckgoose
```

Restricting by Position

English: The `limit()` and `skip()` methods can make a Stream smaller, or `limit()` could make a finite stream out of an infinite stream. The method signatures are shown here: **Türkçe:** `limit()` ve `skip()` yöntemleri bir Stream daha küçük yapabilir veya `limit()` bir infinite stream dışında bir finite stream yapabilir. method signatures burada gösterilmiştir:

```
public Stream<T> limit(long maxSize)
public Stream<T> skip(long n)
```

English: The following code creates an infinite stream of numbers counting from 1. The `skip()` operation returns an infinite stream starting with the numbers counting from 6, since it skips the first five elements. The `limit()` call takes the first two of those. Now we have a finite stream with two elements, which we can then print with the `forEach()` method: **Türkçe:** Aşağıdaki kod, 1'den sayan bir infinite stream sayı oluşturur. `skip()` işlemi, ilk beş elementi atladığından, sayıların 6'dan saymasıyla başlayan bir infinite stream döndürür. `limit()` çağırısı bunlardan ilk ikisini alır. Şimdi iki elemanlı bir finite stream var, daha sonra `forEach()` yöntemiyle yazdırabiliriz:

```
Stream<Integer> s = Stream.iterate(1, n -> n + 1);
s.skip(5)
  .limit(2)
  .forEach(System.out::print); // 67
```

Mapping

English: The `map()` method creates a one-to-one mapping from the elements in the stream to the elements of the next step in the stream. The method signature is as follows: **Türkçe:** `map()` yöntemi, stream içindeki elemanlardan stream içindeki bir sonraki adımın elemanlarına bire bir mapping oluşturur. method signature aşağıdaki gibidir:

```
public <R> Stream<R> map(Function<? super T, ? extends R> mapper)
```

English: This one looks more complicated than the others you have seen. It uses the lambda expression to figure out the type passed to that function and the one returned. The return type is the stream that is returned. **Türkçe:** Bu gördüğünüz diğerlerinden daha karmaşık görünüyor. lambda ifadesini, o function e geçen ve geri dönenin türünü bulmak için kullanır. return type döndürülen stream dir.

The map() method on streams is for transforming data. Don't confuse it

English: with the Map interface, which maps keys to values. **Türkçe:** maps değerlerinin anahtarı olan Map arayüzü ile. **English:** As an example, this code converts a list of String objects to a list of Integer objects representing their lengths: **Türkçe:** Örnek olarak, bu kod String nesnelerinin bir listesini uzunluklarını temsil eden Integer nesnelerinin bir listesine dönüştürür:

```
Stream<String> s = Stream.of("monkey", "gorilla", "bonobo");
s.map(String::length)
  .forEach(System.out::print); // 676
```

English: Remember that String::length is shorthand for the lambda x -> x.length(), which clearly shows it is a function that turns a String into an Integer. **Türkçe:** String::length, lambda x> x.length() için kısaltmadır, bu da bir String'u Integer'ye dönüştüren bir function olduğunu açıkça gösterir.

Using flatMap

English: The flatMap() method takes each element in the stream and makes any elements it contains top-level elements in a single stream. This is helpful when you want to remove empty elements from a stream or combine a stream of lists. We are showing you the method signature for consistency with the other methods so you don't think we are hiding anything. You aren't expected to be able to read this:

Türkçe: flatMap() yöntemi, stream içindeki her elemanı alır ve içerdiği herhangi bir elemanı tek bir stream içinde üst düzey elemanlar haline getirir. Bu, boş öğeleri bir stream 'den kaldırmak veya lists 'nın bir stream 'ini birleştirmek istediğinizde yararlıdır. Size diğer yöntemlerle tutarlılık için method signature gösteriyoruz, böylece hiçbir şey sakladığımızı düşünmüyorsunuz. Bunu okuyabilmeniz beklenmiyor:

English: public <R> Stream<R> flatMap(Function<? super T,? extends Stream<? extends R>> mapper)

Türkçe: genel R> Stream<R> flatMap(Function<? super T,? extends Stream<? extends R>> mapper)

English: This gibberish basically says that it returns a Stream of the type that the function contains at a lower level. Don't worry about the signature. It's a headache. **Türkçe:** Bu gevezelik temel olarak, function'nin daha düşük bir seviyede içerdiği türden bir Stream döndürdüğünü söyler. İmzayı merak etmeyin. Bu bir baş ağrısı. **English:** What you should understand is the example. This gets all of the animals into the same level and removes the empty list. **Türkçe:** Anlamanız gereken şey örnek. Bu, tüm hayvanları aynı seviyeye getirir ve boş list değerini kaldırır.

```
List<String> zero = List.of();
var one = List.of("Bonobo");
var two = List.of("Mama Gorilla", "Baby Gorilla");
Stream<List<String>> animals = Stream.of(zero, one, two);
animals.flatMap(m -> m.stream())
  .forEach(System.out::println);
```

English: Here's the output: **Türkçe:** İşte çıkış: **English:** Bonobo Mama Gorilla Baby Gorilla As you can see, it removed the empty list completely and changed all elements of each list to be at the top level of the stream. **Türkçe:** Bonobo Mama Gorilla Baby Gorilla Gördüğünüz gibi, boş list 'yı tamamen kaldırdı ve her list 'nin tüm öğelerini stream 'nin en üst seviyesinde olmak için değiştirdi. **English:** Concatenating Streams **Türkçe:** Streams birleştiriliyor

While `flatMap()` is good for the general case, there is a more convenient way to concatenate two streams:

English: concatenate two streams: **Türkçe:** birleştir iki streams:

```
var one = Stream.of("Bonobo");
var two = Stream.of("Mama Gorilla", "Baby Gorilla");
Stream.concat(one, two)
    .forEach(System.out::println);
```

English: This produces the same three lines as the previous example. The two streams are concatenated, and the terminal operation, `forEach()`, is called. **Türkçe:** Bu, önceki örnekle aynı üç çizgiyi üretir. İki streams vardır

concatenated, and the terminal operation, `forEach()`, is called.

Sorting

English: The `sorted()` method returns a stream with the elements sorted. Just like sorting arrays, Java uses natural ordering unless we specify a comparator. The method signatures are these: **Türkçe:** `sorted()` yöntemi, sıralanan elemanlarla bir stream döndürür. Dizileri sıralamak gibi, Java bir comparator belirtmediğimiz sürece doğal sırayı kullanır. method signatures şöyledir:

```
public Stream<T> sorted()
public Stream<T> sorted(Comparator<? super T> comparator)
```

English: Calling the first signature uses the default sort order. **Türkçe:** İlk imzayı çağırmak varsayılan sıralama düzenini kullanır.

```
Stream<String> s = Stream.of("brown- ", "bear- ");
s.sorted()
    .forEach(System.out::print); // bear-brown-
```

English: We can optionally use a `Comparator` implementation via a method or a lambda. In this example, we are using a method: **Türkçe:** Optional olarak bir `Comparator` uygulamasını bir yöntem veya lambda aracılığıyla kullanabiliriz. Bu örnekte, bir yöntem kullanıyoruz:

```
Stream<String> s = Stream.of("brown bear- ", "grizzly- ");
s.sorted(Comparator.reverseOrder())
    .forEach(System.out::print); // grizzly-brown bear-
```

English: Here we pass a Comparator to specify that we want to sort in the reverse of natural sort order. Ready for a tricky one? Do you see why this doesn't compile? **Türkçe:** Burada doğal sıralama sırasının tersini sıralamak istediğimizi belirtmek için bir Comparator geçiyoruz. Hileli bir şeye hazır mısınız? Bunun neden derlenmediğini anlıyor musunuz?

```
Stream<String> s = Stream.of("brown bear- ", "grizzly- ");
s.sorted(Comparator::reverseOrder); // DOES NOT COMPILE
```

English: Take a look at the second sorted() method signature again. It takes a Comparator, which is a functional interface that takes two parameters and returns an int. However, Comparator::reverseOrder doesn't do that. Because reverseOrder() takes no arguments and returns a value, the method reference is equivalent to () -> Comparator.reverseOrder(), which is really a Supplier<Comparator>. This is not compatible with sorted(). We bring this up to remind you that you really do need to know method references well. **Türkçe:** İkinci sorted() method signature tekrar bakın. İki parametre alan ve bir int döndüren fonksiyonel bir arayüz olan bir Karşılaştırıcı alır. Ancak, Comparator::reverseOrder bunu yapmaz. reverseOrder() hiçbir argüman almadığı ve bir değer döndürdüğü için, method reference () -> Comparator.reverseOrder() ile eşdeğerdir, ki bu gerçekten bir Supplier<Comparator>'dir. Bu sorted() ile uyumlu değildir. Bunu, method references 'yi gerçekten iyi bilmeniz gerektiğini hatırlatmak için gündeme getiriyoruz.

Taking a Peek

English: The peek() method is our final intermediate operation. It is useful for debugging because it allows us to perform a stream operation without changing the stream. The method signature is as follows: **Türkçe:** peek() yöntemi son intermediate operation yöntemimizdir. Hata ayıklama için kullanışlıdır, çünkü stream işlemini stream değiştirmeden gerçekleştirmemizi sağlar. method signature aşağıdaki gibidir:

```
public Stream<T> peek(Consumer<? super T> action)
```

English: You might notice the intermediate peek() operation takes the same argument as the terminal forEach() operation. Think of peek() as an intermediate version of forEach() that returns the original stream to you. **Türkçe:** Ara peek() işleminin terminal forEach() işlemiyle aynı argümanı aldığını fark edebilirsiniz. peek()'yi size orijinal stream döndüren forEach()'ün ara sürümü olarak düşünün. **English:** The most common use for peek() is to output the contents of the stream as it goes by. Suppose that we made a typo and counted bears beginning with the letter g instead of b. We are puzzled why the count is 1 instead of 2. We can add a peek() method to find out why. **Türkçe:** peek() için en yaygın kullanım, stream içeriğini geçtiği gibi çıkarmaktır. Bir yazım hatası yaptığımızı ve b yerine g harfi ile başlayan ayıları saydığımızı varsayalım. Kontun neden 2 yerine 1 olduğu konusunda şaşkınsınız. Nedenini öğrenmek için peek() metodunu ekleyebiliriz.

```
var stream = Stream.of("black bear", "brown bear", "grizzly");
long count = stream.filter(s -> s.startsWith("g"))
    .peek(System.out::println).count(); // grizzly
System.out.println(count); // 1
```

English: In Chapter 9, you saw that `peek()` looks only at the first element when working with a Queue. In a stream, `peek()` looks at each element that goes through that part of the stream pipeline. It's like having a worker take notes on how a particular step of the process is doing. **Türkçe:** Bölüm 9'da, `peek()`'un bir Kuyruk ile çalışırken yalnızca ilk elemana baktığını gördünüz. Bir stream içinde `peek()`, stream pipeline nin o kısmından geçen her elemana bakar. Bu, bir işçinin sürecin belirli bir adımının nasıl gerçekleştiğine dair notlar alması gibidir. **English:** Danger: Changing State with `peek()` **Türkçe:** Tehlike: `peek()` ile Durumu Değiştirme

```
Remember that peek() is intended to perform an operation without changing the result.
Here's a straightforward stream pipeline that doesn't use peek():
var numbers = new ArrayList<>();
var letters = new ArrayList<>();
numbers.add(1);
letters.add('a');
Stream<List<?>> stream = Stream.of(numbers, letters);
stream.map(List::size).forEach(System.out::print); // 11
Now we add a peek() call and note that Java doesn't prevent us from writing bad
```

English: peek code: **Türkçe:** koda göz atın:

```
Stream<List<?>> bad = Stream.of(numbers, letters);
bad.peek(x -> x.remove(0))
.map(List::size)
.forEach(System.out::print); // 00
This example is bad because peek() is modifying the data structure that is used in the
```

English: stream, which causes the result of the stream pipeline to be different than if the peek wasn't present. **Türkçe:** stream, stream pipeline sonucunun, peek'in bulunmamasından farklı olmasına neden olur.

Putting Together the Pipeline

English: Streams allow you to use chaining and express what you want to accomplish rather than how to do so. Let's say that we wanted to get the first two names of our friends alphabetically that are four characters long. Without streams, we'd have to write something like the following: **Türkçe:** Streams zincirleme kullanmanıza ve bunu nasıl yapacağınızdan ziyade başarmak istediğinizi ifade etmenize izin verir. Diyelim ki arkadaşlarımızın ilk iki ismini alfabetik olarak dört karakter long almak istedik. streams olmadan, aşağıdaki gibi bir şey yazmamız gerekir:

```
var list = List.of("Toby", "Anna", "Leroy", "Alex");
List<String> filtered = new ArrayList<>();
```

English: for (String name: list) **Türkçe:** için (String ad: list)

```
if (name.length() == 4) filtered.add(name);
```

```

Collections.sort(filtered);
var iter = filtered.iterator();
if (iter.hasNext()) System.out.println(iter.next());
if (iter.hasNext()) System.out.println(iter.next());

```

English: This works. It takes some reading and thinking to figure out what is going on. The problem we are trying to solve gets lost in the implementation. It is also very focused on the how rather than on the what. With streams, the equivalent code is as follows: **Türkçe:** Bu işe yarıyor. Neler olup bittiğini anlamak için biraz okumak ve düşünmek gerekir. Çözmeye çalıştığımız sorun uygulamada kaybolur. Aynı zamanda neye değil de nasıl olduğuna çok odaklanmıştır. streams ile eşdeğer kod aşağıdaki gibidir:

```

var list = List.of("Toby", "Anna", "Leroy", "Alex");
list.stream().filter(n -> n.length() == 4).sorted()
    .limit(2).forEach(System.out::println);

```

English: Before you say that it is harder to read, we can format it. **Türkçe:** Okumanın daha zor olduğunu söylemeden önce, onu biçimlendirebiliriz.

```

var list = List.of("Toby", "Anna", "Leroy", "Alex");
list.stream()
    .filter(n -> n.length() == 4)
    .sorted()
    .limit(2)
    .forEach(System.out::println);

```

English: The difference is that we express what is going on. We care about String objects of length 4. Then we want them sorted. Then we want the first two. Then we want to print them out. It maps better to the problem that we are trying to solve, and it is simpler. **Türkçe:** Aradaki fark, olup biteni ifade etmemizdir. String uzunluktaki nesnelere önem veriyoruz. O zaman biz onları halletmek istiyoruz. O zaman ilk ikisini istiyoruz. Sonra da çıktılarını almak istiyoruz. Çözmeye çalıştığımız soruna maps daha iyi ve daha basit. **English:** Once you start using streams in your code, you may find yourself using them in many places. Having shorter, briefer, and clearer code is definitely a good thing! **Türkçe:** Kodunuzda streams kullanmaya başladığınızda, kendinizi birçok yerde kullanırken bulabilirsiniz. Daha kısa, daha kısa ve daha net bir koda sahip olmak kesinlikle iyi bir şey! **English:** In this example, you see all three parts of the pipeline. Figure 10.5 shows how each intermediate operation in the pipeline feeds into the next. **Türkçe:** Bu örnekte, pipeline 'in üç bölümünü de görüyorsunuz. Şekil 10.5, pipeline içindeki her intermediate operation nin bir sonrakine nasıl beslendiğini gösterir. **English:** FIGURE 10.5 Stream pipeline with multiple intermediate operations Intermediate operations **Türkçe:** FIGURE 10.5 Stream pipeline ile çoklu intermediate operations Intermediate operations

```

stream()
forEach()
filter() sorted() limit()

```

English: Remember that the assembly line foreperson is figuring out how to best implement the stream pipeline. They set up all of the tables with instructions to wait before starting. They tell the `limit()` worker to inform them when two elements go by. They tell the `sorted()` worker that they should just collect all of the elements as they come in and sort them all at once. After sorting, they should start passing them to the `limit()` worker one at a time. The data flow looks like this: **Türkçe:** Assembly line foreper'in stream pipeline'ü en iyi şekilde nasıl uygulayacağını çözdüğünü unutmayın. Başlamadan önce beklemesi gereken talimatlarla tüm tabloları set yükseltirler. `limit()` işçisine iki eleman geçtiğinde onları bilgilendirmesini söylerler. `sorted()` işçiye, içeri girerken tüm öğeleri toplamaları ve hepsini bir kerede sıralamaları gerektiğini söylerler. Sıralamadan sonra, bunları birer birer `limit()` işçiye aktarmaya başlamalıdır. Veri akışı şöyle görünür:

English: 1.The `stream()` method sends Toby to `filter()`. The `filter()` method sees that the length is good and sends Toby to `sorted()`. The `sorted()` method can't sort yet because it needs all of the data, so it holds Toby. 2.The `stream()` method sends Anna to `filter()`. The `filter()` method sees that the length is good and sends Anna to `sorted()`. The `sorted()` method can't sort yet because it needs all of the data, so it holds Anna. 3.The `stream()` method sends Leroy to `filter()`. The `filter()` method sees that the length is not a match, and it takes Leroy out of the assembly line processing. 4.The `stream()` method sends Alex to `filter()`. The `filter()` method sees that the length is good and sends Alex to `sorted()`. The `sorted()` method can't sort yet because it needs all of the data, so it holds Alex. It turns out `sorted()` does have all of the required data, but it doesn't know it yet. 5.The foreperson lets `sorted()` know that it is time to sort, and the sort occurs. 6.The `sorted()` method sends Alex to `limit()`. The `limit()` method remembers that it has seen one element and sends Alex to `forEach()`, printing Alex. 7.The `sorted()` method sends Anna to `limit()`. The `limit()` method remembers that it has seen two elements and sends Anna to `forEach()`, printing Anna. 8.The `limit()` method has now seen all of the elements that are needed and tells the foreperson. The foreperson stops the line, and no more processing occurs in the pipeline. **Türkçe:** 1.The `stream()` yöntemi Toby'yi `filter()` adresine gönderir. `filter()` yöntemi uzunluğun iyi olduğunu görür ve Toby'yi `sorted()`'e gönderir. `sorted()` yöntemi, tüm verilere ihtiyaç duyduğu için henüz sıralama yapamaz, bu nedenle Toby'yi tutar. 2. `stream()` yöntemi Anna'yı `filter()`'ya gönderir. `filter()` yöntemi uzunluğun iyi olduğunu görür ve Anna'yı `sorted()`'e gönderir. `sorted()` yöntemi, tüm verilere ihtiyaç duyduğu için henüz sıralama yapamaz, bu nedenle Anna'yı tutar. 3.The `stream()` yöntemi Leroy'u `filter()` adresine gönderir. `filter()` yöntemi, uzunluğun bir eşleşme olmadığını görür ve Leroy'u montaj hattı işlemesinden çıkarır. 4. `stream()` yöntemi Alex'i `filter()`'e gönderir. `filter()` yöntemi uzunluğun iyi olduğunu görür ve Alex'i `sorted()` adresine gönderir. `sorted()` yöntemi henüz sıralama yapamaz çünkü tüm verilere ihtiyaç duyar, bu yüzden Alex'i tutar. `sorted()` gerekli tüm verilere sahip olduğu ortaya çıktı, ancak henüz bilmiyor. 5.Ön kişi `sorted()`'un sıralama zamanının geldiğini bilmesini sağlar ve sıralama gerçekleşir. 6. `sorted()` yöntemi Alex'i `limit()`'e gönderir. `limit()` yöntemi bir element gördüğünü hatırlar ve Alex'i `forEach()`'e gönderir, Alex'i yazdırır. 7. `sorted()` yöntemi Anna'yı `limit()`'e gönderir. `limit()` yöntemi iki element gördüğünü hatırlar ve Anna'yı `forEach()`'ye gönderir, Anna'yı yazdırır. 8. `limit()` yöntemi artık ihtiyaç duyulan tüm elementleri gördü ve foreperson'u anlattı. Foreperson çizgiyi durdurur ve pipeline 'da daha fazla işlem gerçekleşmez. **English:** Make sense? Let's try a few more examples to make sure that you understand this well. What do you think the following does? **Türkçe:** Mantıklı mı? Bunu iyi anladığınızdan emin olmak için birkaç örnek daha deneyelim. Sizce aşağıdakiler ne yapıyor?

```
Stream.generate(() -> "Elsa")
    .filter(n -> n.length() == 4)
    .sorted()
    .limit(2)
    .forEach(System.out::println);
```

English: It hangs until you kill the program, or it throws an exception after running out of memory. The foreperson has instructed `sorted()` to wait until everything to sort is present. That never happens because there is an infinite stream. What about this example? **Türkçe:** Programı öldürene kadar asılı kalır, ya da hafıza bittikten sonra bir istisna atar. Önder, `sorted()`'a her şeyin sıralanmasını beklemesi talimatını verdi. Bu asla olmaz çünkü bir infinite stream var. Peki ya bu örnek?

```
Stream.generate(() -> "Elsa")
    .filter(n -> n.length() == 4)
    .limit(2)
    .sorted()
    .forEach(System.out::println);
```

English: This one prints Elsa twice. The filter lets elements through, and `limit()` stops the earlier operations after two elements. Now `sorted()` can sort because we have a finite list. Finally, what do you think this does? **Türkçe:** Bu Elsa'yı iki kez basıyor. Filtre, elemanların geçmesine izin verir ve `limit()` iki elemandan sonra önceki işlemleri durdurur. Şimdi `sorted()` sonlu list var çünkü sıralayabilir. Son olarak, bunun ne işe yaradığını düşünüyorsunuz?

```
Stream.generate(() -> "Olaf Lazisson")
    .filter(n -> n.length() == 4)
```

```
    .limit(2)
    .sorted()
    .forEach(System.out::println);
```

English: This one hangs as well until we kill the program. The filter doesn't allow anything through, so `limit()` never sees two elements. This means we have to keep waiting and hope that they show up.

Türkçe: Bu da programı öldürene kadar asılı kalacak. Filtre hiçbir şeyin geçmesine izin vermez, bu nedenle `limit()` asla iki eleman görmez. Bu, beklemeye devam etmemiz ve ortaya çıkmalarını ummamız gerektiği anlamına gelir. **English:** You can even chain two pipelines together. See if you can identify the two sources and two terminal operations in this code. **Türkçe:** Hatta birlikte iki pipelines zincirleyebilirsiniz. Bu koddaki iki kaynağı ve iki terminal operations tanımlayıp tanımlayamadığınıza bakın.

```
long count = Stream.of("goldfish", "finch")
    .filter(s -> s.length() > 5)
    .collect(Collectors.toList())
    .stream()
    .count();
System.out.println(count); // 1
```

English: Lines 30–32 are one pipeline, and lines 33 and 34 are another. For the first pipeline, line 30 is the source, and line 32 is the terminal operation. For the second pipeline, line 33 is the source, and line 34 is the terminal operation. Now that's a complicated way of outputting the number 1! **Türkçe:** 3032 satırları bir pipeline ve 33 ve 34 satırları bir diğeridir. İlk pipeline için, satır 30 kaynaktır ve satır 32 terminal operation'dir. İkinci pipeline için, 33. satır kaynaktır ve 34. satır terminal operation'dir. Şimdi bu 1 sayısını çıkarmak için karmaşık bir yol! **English:** On the exam, you might see long or complex pipelines as answer choices. If this happens, focus on the differences between the answers. **Türkçe:** Sınavda, long veya karmaşık pipelines yanıt seçenekleri olarak görebilirsiniz. Bu gerçekleşirse, cevaplar arasındaki farklılıklara odaklanın. **English:** Those will be your clues to the correct answer. This approach will also save you time by not having to study the whole pipeline on each option. **Türkçe:** Bunlar doğru cevap için ipuçlarınız olacaktır. Bu yaklaşım aynı zamanda her seçenekte tüm pipeline üzerinde çalışmak zorunda kalmadan size zaman kazandıracaktır. **English:** When you see chained pipelines, note where the source and terminal operations are. This will help you keep track of what is going on. You can even rewrite the code in your head to have a variable in between so it isn't as long and complicated. Our prior example can be written as follows: **Türkçe:** Zincirli pipelines gördüğünüzde, kaynağın ve terminal operations'un nerede olduğunu not edin. Bu, neler olup bittiğini takip etmenize yardımcı olacaktır. Hatta kafanızdaki kodu long ve karmaşık olmayan bir değışkene sahip olmak için yeniden yazabilirsiniz. Önceki örneğimiz aşağıdaki gibi yazılabilir:

```
List<String> helper = Stream.of("goldfish", "finch")
    .filter(s -> s.length() > 5)
    .collect(Collectors.toList());
long count = helper.stream()
    .count();
System.out.println(count);
```

English: Which style you use is up to you. However, you need to be able to read both styles before you take the exam. **Türkçe:** Hangi stili kullanacağınız size kalmış. Ancak, sınava girmeden önce her iki stili de okuyabilmeniz gerekir.

Working with Primitive Streams

English: Up until now, all of the streams we've created used the Stream interface with a generic type, like Stream<String>, Stream<Integer>, and so on. For numeric values, we have been using wrapper classes. We did this with the Collections API in Chapter 9, so it should feel natural. **Türkçe:** Şimdiye kadar, yarattığımız tüm streams, Stream<String>, Stream<Integer> ve benzeri genel bir türe sahip Stream arayüzünü kullandı. Sayısal değerler için, ambalaj sınıflarını kullanıyoruz. Bunu 9. bölümde Collections API ile yaptık, bu yüzden doğal hissetmelidir. **English:** Java actually includes other stream classes besides Stream that you can use to work with select primitives: int, double, and long. Let's take a look at why this is needed. Suppose that we want to calculate the sum of numbers in a finite stream: **Türkçe:** Java, seçili ilkellerle çalışmak için kullanabileceğiniz Stream dışında diğer stream sınıflarını da içerir: int, double ve long. Bunun neden gerekli olduğuna bir göz atalım. Bir finite stream içindeki sayıların toplamını hesaplamak istediğimizi varsayalım:

```
Stream<Integer> stream = Stream.of(1, 2, 3);
System.out.println(stream.reduce(0, (s, n) -> s + n)); // 6
```

English: Not bad. It wasn't hard to write a reduction. We started the accumulator with zero. We then added each number to that running total as it came up in the stream. There is another way of doing that, shown here: **Türkçe:** Fena değil. Bir reduction yazmak zor olmadı. accumulator'u sıfır ile başlattık. Daha sonra her sayıyı stream 'de olduğu gibi bu çalışan toplamına ekledik. Bunu yapmanın başka bir yolu daha var, burada gösterilmiştir:

```
Stream<Integer> stream = Stream.of(1, 2, 3);
System.out.println(stream.mapToInt(x -> x).sum()); // 6
```

English: This time, we converted our Stream<Integer> to an IntStream and asked the IntStream to calculate the sum for us. An IntStream has many of the same intermediate and terminal methods as a Stream but includes specialized methods for working with numeric data. The primitive streams know how to perform certain common operations automatically. **Türkçe:** Bu kez, Stream<Integer>'mizi IntStream'ye dönüştürdük ve IntStream'den bizim için toplamı hesaplamasını istedik. Bir IntStream, Stream ile aynı ara ve terminal yöntemlerinin birçoğuna sahiptir, ancak sayısal verilerle çalışmak için özel yöntemler içerir. primitive streams bazı ortak işlemlerin otomatik olarak nasıl yapılacağını bilir. **English:** So far, this seems like a nice convenience but not terribly important. Now think about how you would compute an average. You need to divide the sum by the number of elements. The problem is that streams allow only one pass. Java recognizes that calculating an average is a common thing to do, and it provides a method to calculate the average on the stream classes for primitives. **Türkçe:** Şimdiye kadar, bu iyi bir kolaylık gibi görünüyor ama korkunç derecede önemli değil. Şimdi bir ortalama nasıl hesaplayacağınızı düşünün. Toplamı element sayısına bölmeniz gerekir. Sorun şu ki streams sadece bir geçişe izin veriyor. Java, bir ortalama hesaplamasının yapılması gereken ortak bir şey olduğunu kabul eder ve ilkeler için stream sınıflarındaki ortalama hesaplamak için bir yöntem sağlar.

```
IntStream intStream = IntStream.of(1, 2, 3);
OptionalDouble avg = intStream.average();
System.out.println(avg.getAsDouble()); // 2.0
```

English: Not only is it possible to calculate the average, but it is also easy to do so. Clearly, primitive streams are important. We look at creating and using such streams, including optionals and functional interfaces. **Türkçe:** Sadece ortalama hesaplamak mümkün değil, aynı zamanda bunu yapmak da kolaydır. primitive streams çok önemlidir. optionals ve functional interfaces de dahil olmak üzere bu tür streams oluşturmaya ve kullanmaya bakıyoruz.

Creating Primitive Streams

English: Here are the three types of primitive streams: ■ ■ IntStream: Used for the primitive types int, short, byte, and char ■ ■ LongStream: Used for the primitive type long ■ ■ DoubleStream: Used for the primitive types double and float **Türkçe:** İlkel akışların üç türü şunlardır: IntStream: İlkel tipler için kullanılan int, kısa, bayt ve char LongStream: İlkel tip için kullanılan uzun DoubleStream: İlkel tipler için kullanılan çift ve float

English: Why doesn't each primitive type have its own primitive stream? These three are the most common, so the API designers went with them. **Türkçe:** Neden her ilkel türün kendi primitive stream'i yok? Bu üçü en yaygın olanıdır, bu yüzden API tasarımcıları da onlarla birlikte gitti. **English:** When you see the word stream on the exam, pay attention to the case. **Türkçe:** Sınavda stream kelimesini gördüğünüzde, davadaya dikkat edin. **English:** With a capital S or in code, Stream is the name of a class that contains an Object type. With a lowercase s, a stream is a concept that might be a Stream, DoubleStream, IntStream, or LongStream. **Türkçe:** Bir büyük S veya kod ile, Stream bir Object türü içeren bir sınıfın adıdır. Bir küçük harf s ile, bir stream bir Stream, DoubleStream, IntStream veya LongStream olabilecek bir kavramdır. **English:** Table 10.5 shows some of the methods that are unique to primitive streams. Notice that we don't include methods in the table like empty() that you already know from the Stream interface. **Türkçe:** Tablo 10.5, primitive streams ye özgü yöntemlerden bazılarını gösterir. Stream arayüzünden zaten bildiğiniz empty() gibi yöntemleri tabloya dahil etmediğimize dikkat edin. **English:** TABLE 10.5 Common primitive stream methods

Method	Primitive stream	Description
OptionalDouble average()	IntStream LongStream DoubleStream Stream<T> boxed()	IntStream Stream<T> where T is wrapper class associated with LongStream primitive value DoubleStream OptionalInt max()
IntStream LongStream DoubleStream Stream<T> where T is wrapper class associated with LongStream primitive value DoubleStream OptionalInt max()	IntStream LongStream DoubleStream Stream<T> where T is wrapper class associated with LongStream primitive value DoubleStream OptionalInt max()	IntStream LongStream DoubleStream Stream<T> where T is wrapper class associated with LongStream primitive value DoubleStream OptionalInt max()
OptionalLong max()	LongStream OptionalDouble max()	DoubleStream OptionalInt min()
OptionalInt min()	IntStream Minimum element of stream OptionalLong min()	LongStream OptionalDouble min()
OptionalDouble min()	DoubleStream IntStream range(int a, int b)	IntStream Returns primitive stream from a (inclusive) to b (exclusive) LongStream range(long a, LongStream long b)

Türkçe: TABLE 10.5 Ortak primitive Streams yöntemleri

Method	Primitive stream	Açıklama
OptionalDouble average()	IntStream LongStream DoubleStream StreamT> T'nin LongStream ilkel değer DoubleStream OptionalInt max()	IntStream Maksimum akış elemanı OptionalLong max()
OptionalLong max()	DoubleStream OptionalInt min()	OptionalOptionalOptionalLong natürelD

English: IntStream rangeClosed(int a, IntStream Returns primitive stream from int b) **Türkçe:** IntStream rangeClosed(int a, IntStream Returns primitive stream from int b) **English:** a (inclusive) to b (inclusive) **Türkçe:** a (dahil) to b (dahil) **English:** LongStream rangeClosed(long a, LongStream long b) **Türkçe:** LongStream rangeClosed(long a, LongStream long b)

English: Method Primitive stream Description **Türkçe:** Yöntem Primitive stream Açıklama

```
int sum() IntStream Returns sum of elements in
```

English: stream long sum() LongStream double sum() DoubleStream IntSummaryStatistics IntStream Returns object containing numerous stream statistics summaryStatistics() **Türkçe:** stream long sum() LongStream double sum() DoubleStream IntSummaryStatistics IntStream Sayısız stream istatistik içeren nesneyi döndürür summaryStatistics() **English:** such as average, min, max, etc. LongSummaryStatistics LongStream summaryStatistics() **Türkçe:** gibi ortalama, min, max, vb. LongSummaryStatistics LongStream summaryStatistics() **English:** DoubleSummaryStatistics DoubleStream summaryStatistics() **Türkçe:** DoubleSummaryStatistics DoubleStream summaryStatistics() **English:** Some of the methods for creating a primitive stream are equivalent to how we created the source for a regular Stream. You can create an empty stream with this: **Türkçe:** primitive stream oluşturma yöntemlerinden bazıları, düzenli bir Stream için kaynağı nasıl oluşturduğumuza eşdeğerdir. Bununla boş bir stream oluşturabilirsiniz:

```
DoubleStream empty = DoubleStream.empty();
```

English: Another way is to use the of() factory method from a single value or by using the varargs overload. **Türkçe:** Başka bir yol, of() factory method değerini tek bir değerden veya varargs aşırı yüklenmesini kullanarak kullanmaktır.

```
DoubleStream oneValue = DoubleStream.of(3.14);
oneValue.forEach(System.out::println);
DoubleStream varargs = DoubleStream.of(1.0, 1.1, 1.2);
varargs.forEach(System.out::println);
```

English: This code outputs the following: **Türkçe:** Bu kod aşağıdaki çıktıları verir: **English:** 3.14 1.0 1.1 1.2 You can also use the two methods for creating infinite streams, just like we did with Stream. **Türkçe:** 3.14 1.0 1.1 1.2 Stream ile yaptığımız gibi infinite streams oluşturmak için iki yöntemi de kullanabilirsiniz.

```
var random = DoubleStream.generate(Math::random);
var fractions = DoubleStream.iterate(.5, d -> d / 2);
random.limit(3).forEach(System.out::println);
fractions.limit(3).forEach(System.out::println);
```

English: Since the streams are infinite, we added a limit intermediate operation so that the output doesn't print values forever. The first stream calls a static method on Math to get a random double. Since the numbers are random, your output will obviously be different. The second stream keeps creating smaller numbers, dividing the previous value by two each time. The output from when we ran this code was as follows: **Türkçe:** streams sonsuz olduğundan, çıktının değerleri sonsuza kadar yazdırmaması için bir sınır intermediate operation ekledik. İlk stream rastgele bir double elde etmek için Matematik üzerinde statik bir yöntem çağırır. Sayılar rastgele olduğundan, çıktınız açıkça farklı olacaktır. İkinci stream, önceki değeri her seferinde ikiye bölerek daha küçük sayılar oluşturmaya devam ediyor. Bu kodu çalıştırdığımızda çıkan çıktı şu şekildeydi: **English:** 0.07890654781186413 0.28564363465842346 0.6311403511266134 0.5 0.25 0.125 You don't need to know this for the exam, but the Random class provides a method to get primitives streams of random numbers directly. Fun fact! For example, ints() generates an infinite IntStream of primitives. **Türkçe:** 0.07890654781186413 0.2856436344658423446 0.6311403511266134 0.5 0.25 0.125 Sınav için bunu bilmenize gerek yok, ancak Rastgele sınıf, doğrudan rastgele sayıların streams ilkelerini elde etmek için bir yöntem sunar. Eğlenceli bir gerçek! Örneğin, ints() sonsuz IntStream ilkeleri üretir. **English:** It works the same way for each type of primitive stream. When dealing with int or long primitives, it is common to count. Suppose that we wanted a stream with the numbers from 1 through 5. We could write this using what we've explained so far: **Türkçe:** Her primitive stream türü için aynı şekilde çalışır. int veya long ilkeleriyle uğraşırken, saymak yaygındır. 1'den 5'e kadar olan sayılarla bir stream istediğimizi varsayalım. Bunu şu ana kadar açıkladıklarımızı kullanarak yazabiliriz:

```
IntStream count = IntStream.iterate(1, n -> n+1).limit(5);
count.forEach(System.out::print); // 12345
```

English: This code does print out the numbers 1–5. However, it is a lot of code to do something so simple. Java provides a method that can generate a range of numbers. **Türkçe:** Bu kod 15 numaralarını yazdırır. Ancak, bu kadar basit bir şey yapmak çok fazla kod. Java bir dizi sayı üretebilecek bir yöntem sağlar.

```
IntStream range = IntStream.range(1, 6);
range.forEach(System.out::print); // 12345
```

English: This is better. If we wanted numbers 1–5, why did we pass 1–6? The first parameter to the `range()` method is inclusive, which means it includes the number. The second parameter to the `range()` method is exclusive, which means it stops right before that number. However, it still could be clearer. We want the numbers 1–5 inclusive. Luckily, there's another method, `rangeClosed()`, which is inclusive on both parameters. **Türkçe:** Bu daha iyi. Eğer 15 numaralar istediyseniz, neden 16'yı geçtik? `range()` yönteminin ilk parametresi kapsayıcıdır, yani sayıyı içerir. `range()` yönteminin ikinci parametresi özeldir, yani bu sayıdan hemen önce durur. Ancak, yine de daha açık olabilir. 15 sayılarının dahil edilmesini istiyoruz. Neyse ki, her iki parametrede de kapsayıcı olan `rangeClosed()` başka bir yöntem var.

```
IntStream rangeClosed = IntStream.rangeClosed(1, 5);
rangeClosed.forEach(System.out::print); // 12345
```

English: Even better. This time we expressed that we want a closed range or an inclusive range. This method better matches how we express a range of numbers in plain English. **Türkçe:** Daha da iyi. Bu sefer kapalı bir aralık ya da kapsayıcı bir aralık istediğimizi ifade ettik. Bu yöntem düz İngilizcede bir dizi sayıyı nasıl ifade ettiğimizle daha iyi eşleşir.

Mapping Streams

English: Another way to create a primitive stream is by mapping from another stream type. Table 10.6 shows that there is a method for mapping between any stream types. **Türkçe:** Bir primitive stream oluşturmanın başka bir yolu, başka bir stream türünden mapping ile. Tablo 10.6, herhangi bir stream türü arasında mapping için bir yöntem olduğunu gösterir.

English: TABLE 10.6 Mapping methods between types of streams
 Source To create To create To create
 To create stream class Stream DoubleStream IntStream LongStream Stream<T> map() mapToDouble()
 mapToInt() mapToLong() **Türkçe:** TABLE 10.6 Akış türleri arasındaki haritalama yöntemleri
 Kaynak Yaratmak için Oluşturun Akış sınıfı oluşturmak için Stream DoubleStream IntStream LongStream
 Stream<T> map() mapToDouble() mapToInt() mapToLong() **English:** DoubleStream mapToObj() map()
 mapToInt() mapToLong() **Türkçe:** DoubleStream mapToObj() mapToInt() mapToLong() **English:**
 IntStream mapToObj() mapToDouble() map() mapToLong() **Türkçe:** IntStream mapToObj()
 mapToDouble() map() mapToLong() **English:** LongStream mapToObj() mapToDouble() mapToInt()
 map() **Türkçe:** LongStream mapToObj() mapToDouble() mapToInt() map **English:** Obviously, they have
 to be compatible types for this to work. Java requires a mapping function to be provided as a parameter,
 for example: **Türkçe:** Açıkçası, bunun çalışması için uyumlu tipler olmaları gerekir. Java parametre olarak
 sağlanacak bir mapping function gerektirir, örneğin:

```
Stream<String> objStream = Stream.of("penguin", "fish");
IntStream intStream = objStream.mapToInt(s -> s.length());
```

English: This function takes an Object, which is a String in this case. The function returns an int. The function mappings are intuitive here. They take the source type and return the target type. In this example, the actual function type is ToIntFunction. Table 10.7 shows the mapping function names. As you can see, they do what you might expect. **Türkçe:** Bu function, bu durumda bir String olan bir Nesne alır. function bir int döndürür. function mappings burada sezgiseldir. Kaynak türünü alır ve hedef türünü döndürürler. Bu örnekte, gerçek function türü ToIntFunction'dır. Tablo 10.7 mapping function isimlerini gösterir. Gördüğünüz gibi, beklediğiniz şeyi yapıyorlar. **English:** You do have to memorize Table 10.6 and Table 10.7. It's not as hard as it might seem. There are patterns in the names if you remember a few rules. For Table 10.6, mapping to the same type you started with is just called map(). When returning an object stream, the method is mapToObj(). Beyond that, it's the name of the primitive type in the map method name. **Türkçe:** Tablo 10.6 ve Tablo 10.7'yi ezberlemeniz gerekir. Görüldüğü kadar zor değil. Birkaç kural hatırlarsanız isimlerde desenler vardır. Tablo 10.6 için, mapping ile başladığınız aynı türe sadece map() denir. Bir nesneyi stream döndürürken, yöntem mapToObj()'dir. Bunun ötesinde, map yöntem ismindeki ilkel türün adıdır. **English:** For Table 10.7, you can start by thinking about the source and target types. When the target type is an object, you drop the To from the name. When the mapping is to the same type you started with, you use a unary operator instead of a function for the primitive streams. **Türkçe:** Tablo 10.7 için kaynak ve hedef tiplerini düşünerek başlayabilirsiniz. Hedef türü bir nesne olduğunda, To'yu isimden bırakırsınız. mapping ile başladığınız aynı türe geldiğinde, primitive streams için function yerine bir unary operatörü kullanırsınız. **English:** Using flatMap() **Türkçe:** flatMap() kullanılarak **English:** We can use this approach on primitive streams as well. It works the same way as on a regular Stream, except the method name is different. Here's an example: **Türkçe:** Bu yaklaşımı primitive streams üzerinde de kullanabiliriz. Normal bir Stream üzerinde olduğu gibi çalışır, ancak yöntem adı farklıdır. İşte bir örnek:

```
var integerList = new ArrayList<Integer>();
IntStream ints = integerList.stream()
    .flatMapToInt(x -> IntStream.of(x));
DoubleStream doubles = integerList.stream()
    .flatMapToDouble(x -> DoubleStream.of(x));
LongStream longs = integerList.stream()
    .flatMapToLong(x -> LongStream.of(x));
```

English: TABLE 10.7 Function parameters when mapping between types of streams To create To create To create Source stream class Stream DoubleStream IntStream LongStream ToInt ToLongStream<T> Function<T,R> ToDouble Function<T> Function<T> Function<T> DoubleStream Double DoubleUnary DoubleToInt DoubleToLong Function<R> Operator Function Function IntStream IntFunction<R> IntToDouble IntUnary IntToLong Function Operator Function LongStream Long LongToDouble LongToInt LongUnary Function<R> Function Function Operator Additionally, you can create a Stream from a primitive stream. These methods show two ways of accomplishing this: **Türkçe:** TABLE 10.7 Akış türleri arasında eşleme yaparken fonksiyon parametreleri Oluşturulmak için oluşturmak için Kaynak akışı sınıfı oluşturmak için Stream DoubleStream IntStream LongStream ToInt ToLongStream<T> İşlevi<T> İşlevi<T> ÇiftStream DoubleUnary ÇiftStream UzunStream İşlevi<R> Operatör İşlevi IntStream İşlevi IntDoubleTot İşlevi Bu yöntemler bunu başarmanın iki yolunu göstermektedir:

```
private static Stream<Integer> mapping(IntStream stream) {
    return stream.mapToObj(x -> x);
}
private static Stream<Integer> boxing(IntStream stream) {
    return stream.boxed();
}
```

English: The first one uses the `mapToObj()` method we saw earlier. The second one is more succinct. It does not require a mapping function because all it does is autobox each primitive to the corresponding wrapper object. The `boxed()` method exists on all three types of primitive streams. **Türkçe:** İlki, daha önce gördüğümüz `mapToObj()` yöntemini kullanıyor. İkincisi ise daha özdür. mapping function gerektirmez, çünkü tek yaptığı her bir ilkeli ilgili ambalaj nesnesine otomatik kutulamaktır. `boxed()` yöntemi her üç primitive streams türünde de mevcuttur.

Using Optional with Primitive Streams

English: Earlier in the chapter, we wrote a method to calculate the average of an `int[]` and promised a better way later. Now that you know about primitive streams, you can calculate the average in one line.

Türkçe: Bölümün başlarında, bir `int` ortalamasını hesaplamak için bir yöntem yazdık ve daha sonra daha iyi bir yol vaat ettik. Artık primitive streams hakkında bilgi sahibi olduğunuza göre, bir satırdaki ortalamayı hesaplayabilirsiniz.

```
var stream = IntStream.rangeClosed(1,10);  
OptionalDouble optional = stream.average();
```

English: The return type is not the `Optional` you have become accustomed to using. It is a new type called `OptionalDouble`. Why do we have a separate type, you might wonder? Why not just use `Optional<Double>`? The difference is that `OptionalDouble` is for a primitive and `Optional<Double>` is for the `Double` wrapper class. Working with the primitive optional class looks similar to working with the `Optional` class itself. **Türkçe:** return type kullanmaya alıştığınız `Optional` değildir. `OptionalDouble` olarak adlandırılan yeni bir türdür. Neden ayrı bir tipimiz var, merak edebilirsiniz. Neden sadece `Optional<Double>` kullanmıyorsunuz? Aradaki fark, `OptionalDouble` ilkel ve `Optional<Double>` ise `Double` ambalaj sınıfı içindir. İlkel optional sınıfıyla çalışmak, `Optional` sınıfıyla çalışmaya benzer.

```
optional.ifPresent(System.out::println); // 5.5  
System.out.println(optional.getAsDouble()); // 5.5  
System.out.println(optional.orElseGet(() -> Double.NaN)); // 5.5
```

English: The only noticeable difference is that we called `getAsDouble()` rather than `get()`. This makes it clear that we are working with a primitive. Also, `orElseGet()` takes a `DoubleSupplier` instead of a `Supplier`.

Türkçe: Fark edilen tek fark, `get()` yerine `getAsDouble()` olarak adlandırdığımızdır. Bu, ilkel bir şeyle çalıştığımızı açıkça ortaya koymaktadır. Ayrıca, `orElseGet()` bir `Supplier` yerine bir Çift Tedarikçi alır.

English: As with the primitive streams, there are three type-specific classes for primitives. Table 10.8 shows the minor differences among the three. You probably won't be surprised that you have to memorize this table as well. This is really easy to remember since the primitive name is the only change. As you should remember from the terminal operations section, a number of stream methods return an optional such as `min()` or `findAny()`. These each return the corresponding optional type. The primitive stream implementations also add two new methods that you need to know. The `sum()` method does not return an optional. If you try to add up an empty stream, you simply get zero. The `average()` method always returns an `OptionalDouble` since an average can potentially have fractional data for any type. **Türkçe:** primitive streams'te olduğu gibi, ilkeler için üç tipe özgü sınıf vardır. Tablo 10.8, üçü arasındaki küçük farklılıkları göstermektedir. Muhtemelen bu masayı da ezberlemek zorunda olduğunuza şaşırırmazsınız. Bunu hatırlamak çok kolaydır, çünkü ilkel isim tek değişikliktir. terminal operations bölümünden hatırlamanız gereken gibi, bir dizi stream yöntemi `min()` veya `findAny()` gibi bir optional döndürür. Bunların her biri karşılık gelen optional türünü döndürür. primitive stream uygulamaları ayrıca bilmeniz gereken iki yeni yöntem ekler. `sum()` yöntemi bir optional döndürmez. Boş bir stream eklemeye çalışırsanız, sadece sıfır elde edersiniz. `average()` yöntemi her zaman bir `OptionalDouble` döndürür, çünkü bir ortalama herhangi bir tür için potansiyel olarak kesirli verilere sahip olabilir. **English:** TABLE 10.8 Optional types for primitives
OptionalDouble OptionalInt OptionalLong
Getting as primitive `getAsDouble()` `getAsInt()` `getAsLong()`
Türkçe: TABLE 10.8 Optional ilkeler için OptionalDouble OptionalInt OptionalLong İlkel `getAsDouble()` `getAsInt()` `getAsLong()`
English: `orElseGet()` parameter type `DoubleSupplier` `IntSupplier` `LongSupplier`
Return type of `max()` and `min()` `OptionalDouble` `OptionalInt` `OptionalLong`
Return type of `sum()` `double` `int` `long`
Return type of `average()` `OptionalDouble` `OptionalDouble` `OptionalDouble`
Let's try an example to make sure that you understand this: **Türkçe:** `orElseGet()` parametre türü Çift Tedarikçi Int Tedarikçi Uzun Tedarikçi `Max()` ve `min()` `OptionalDouble` `OptionalInt` `OptionalInt` `OptionalLong`
Return type of `sum()` `double` `int` `long`
Return type of `average()` `OptionalDouble` `OptionalDouble` `OptionalDouble`
Bunu anladığınızdan emin olmak için bir örnek deneyelim:

```
LongStream longs = LongStream.of(5, 10);
long sum = longs.sum();
System.out.println(sum); // 15
DoubleStream doubles = DoubleStream.generate(() -> Math.PI);
OptionalDouble min = doubles.min(); // runs infinitely
```

English: Line 5 creates a stream of long primitives with two elements. Line 6 shows that we don't use an optional to calculate a sum. Line 8 creates an infinite stream of double primitives. Line 9 is there to remind you that a question about code that runs infinitely can appear with primitive streams as well. **Türkçe:** Line 5, iki elementli long ilkelerden oluşan bir stream oluşturur. Satır 6, bir toplamı hesaplamak için bir optional kullanmadığımızı gösterir. Line 8, double ilkelerinden oluşan bir infinite stream oluşturur. 9. satır, sonsuz çalışan kodla ilgili bir sorunun primitive streams ile de görünebileceğini hatırlatmak için oradadır.

Summarizing Statistics

English: You've learned enough to be able to get the maximum value from a stream of int primitives. If the stream is empty, we want to throw an exception. **Türkçe:** int ilkelerinin bir stream değerinden maksimum değeri elde edebilecek kadar şey öğrendiniz. stream boşsa, bir istisna atmak istiyoruz.

```
private static int max(IntStream ints) {
    OptionalInt optional = ints.max();
    return optional.orElseThrow(RuntimeException::new);
}
```

English: This should be old hat by now. We got an OptionalInt because we have an IntStream. If the optional contains a value, we return it. Otherwise, we throw a new RuntimeException. **Türkçe:** Bu şimdiye kadar eski bir şapka olmalıydı. OptionalInt var çünkü IntStream var. optional bir değer içeriyorsa, onu iade ederiz. Aksi takdirde yeni bir RuntimeException atarız. **English:** Now we want to change the method to take an IntStream and return a range. The range is the minimum value subtracted from the maximum value. Uh-oh. Both min() and max() are terminal operations, which means that they use up the stream when they are run. We can't run two terminal operations against the same stream. Luckily, this is a common problem, and the primitive streams solve it for us with summary statistics. Statistic is just a big word for a number that was calculated from data. **Türkçe:** Şimdi bir IntStream almak ve bir aralık döndürmek için yöntemi değiştirmek istiyoruz. Menzil, maksimum değerden çıkarılan minimum değerdir. Uh-oh. Hem min() hem de max() terminal operations'dir, bu da çalıştırdıklarında stream'ı kullandıkları anlamına gelir. Aynı stream karşısında iki terminal operations çalıştıramayız. Neyse ki, bu yaygın bir sorundur ve primitive streams özet istatistiklerle bizim için çözer. İstatistik, verilerden hesaplanan bir sayı için sadece büyük bir kelimedir.

```
private static int range(IntStream ints) {
    IntSummaryStatistics stats = ints.summaryStatistics();
    if (stats.getCount() == 0) throw new RuntimeException();
    return stats.getMax() - stats.getMin();
}
```

English: Here we asked Java to perform many calculations about the stream. Summary statistics include the following: ■ ■ getCount(): Returns a long representing the number of values. ■ ■ getAverage(): Returns a double representing the average. If the stream is empty, returns 0. ■ ■ getSum(): Returns the sum as a double for DoubleSummaryStream and long for IntSummaryStream and LongSummaryStream. ■ ■ getMin(): Returns the smallest number (minimum) as a double, int, or long, depending on the type of the stream. If the stream is empty, returns the largest numeric value based on the type. ■ ■ getMax(): Returns the largest number (maximum) as a double, int, or long depending on the type of the stream. If the stream is empty, returns the smallest numeric value based on the type. **Türkçe:** Burada Java 'den stream ile ilgili birçok hesaplama yapmasını istedik. Özet istatistikler şunları içerir: getCount(): Değer sayısını temsil eden bir long döndürür. getAverage(): Ortalamayı temsil eden bir double döndürür. Akış boşsa, 0 döndürür. getSum(): Toplamı DoubleSummaryStream için çift olarak ve IntSummaryStream ve LongSummaryStream için uzun olarak döndürür. getMin(): Akışın türüne bağlı olarak en küçük sayıyı (minimum) çift, int veya uzun olarak döndürür. Akış boşsa, türüne göre en büyük sayısal değeri döndürür. getMax(): Akışın türüne bağlı olarak en büyük sayıyı (maksimum) çift, int veya uzun olarak döndürür. stream boşsa, türüne göre en küçük sayısal değeri döndürür.

Editor note: The source uses DoubleSummaryStream, IntSummaryStream, and LongSummaryStream in this paragraph. The Java 17 API types are DoubleSummaryStatistics, IntSummaryStatistics, and LongSummaryStatistics. **Editör notu:** Kaynak bu paragrafta DoubleSummaryStream, IntSummaryStream ve LongSummaryStream adlarını kullanıyor. Java 17 API'deki doğru type adları DoubleSummaryStatistics, IntSummaryStatistics ve LongSummaryStatistics'tir.

Working with Advanced Stream

Pipeline Concepts

English: Congrats, you only have a few more topics left! In this last stream section, we learn about the relationship between streams and the underlying data, chaining Optional, and grouping collectors. After this, you should be a pro with streams! **Türkçe:** Tebrikler, sadece birkaç konu kaldı! Bu son stream bölümünde, streams ve altta yatan veriler, Optional ve grouping collectors arasındaki ilişkiyi öğreniyoruz. Bundan sonra, streams ile bir profesyonel olmalısınız!

Linking Streams to the Underlying Data

English: What do you think this outputs? **Türkçe:** Sence bu ne çıktı?

```
var cats = new ArrayList<String>();
cats.add("Annie");
cats.add("Ripley");
var stream = cats.stream();
cats.add("KC");
System.out.println(stream.count());
```

English: The correct answer is 3. Lines 25–27 create a List with two elements. Line 28 requests that a stream be created from that List. Remember that streams are lazily evaluated. This means that the stream isn't created on line 28. An object is created that knows where to look for the data when it is needed. On line 29, the List gets a new element. On line 30, the stream pipeline runs. First, it looks at the source and seeing three elements. **Türkçe:** Doğru cevap 3'tür. 25-27 satırları iki elemanlı bir List oluşturur. 28. satır, o List dan bir stream oluşturulmasını ister. streams'nin tembelce değerlendirildiğini unutmayın. Bu, stream'un 28. satırda oluşturulmadığı anlamına gelir. Gerektiğinde veriyi nerede arayacağını bilen bir nesne oluşturulur. 29. hatta List yeni bir eleman alır. 30. satırda stream pipeline çalışır. İlk olarak kaynağa bakar ve üç element görür.

Chaining Optionals

English: By now, you are familiar with the benefits of chaining operations in a stream pipeline. A few of the intermediate operations for streams are available for Optional. **Türkçe:** Şimdiye kadar, bir stream pipeline zincirleme işlemlerinin faydalarını biliyorsunuz. streams için intermediate operations 'dan birkaçı Optional için kullanılabilir. **English:** Suppose that you are given an Optional<Integer> and asked to print the value, but only if it is a three-digit number. Without functional programming, you could write the following: **Türkçe:** Size bir Optional<Integer> verildiğini ve değeri yazdırmanız istendiğini, ancak yalnızca üç basamaklı bir sayı olduğunu varsayalım. İşlevsel programlama olmadan şunları yazabilirsiniz:

```
private static void threeDigit(Optional<Integer> optional) {
    if (optional.isPresent()) { // outer if
        var num = optional.get();
        var string = "" + num;
        if (string.length() == 3) // inner if
            System.out.println(string);
    }
}
```

English: It works, but it contains nested if statements. That's extra complexity. Let's try this again with functional programming: **Türkçe:** İşe yarıyor, ancak ifadeler varsa yuvalanmış içeriyor. Bu ekstra bir karmaşıklık. Bunu fonksiyonel programlama ile tekrar deneyelim:

```
private static void threeDigit(Optional<Integer> optional) {
    optional.map(n -> "" + n)
    // part 1
    .filter(s -> s.length() == 3) // part 2
    .ifPresent(System.out::println); // part 3
}
```

English: This is much shorter and more expressive. With lambdas, the exam is fond of carving up a single statement and identifying the pieces with a comment. We've done that here to show what happens with both the functional programming and nonfunctional programming approaches. **Türkçe:** Bu çok daha kısa ve daha etkileyicidir. lambdas ile sınav, tek bir ifadeyi oymaya ve parçaları bir yorumla tanımlamaya düşkündür. Bunu burada hem işlevsel programlama hem de işlevsiz programlama yaklaşımlarında neler olduğunu göstermek için yaptık. **English:** Suppose that we are given an empty Optional. The first approach returns false for the outer if statement. The second approach sees an empty Optional and has both map() and filter() pass it through. Then ifPresent() sees an empty Optional and doesn't call the Consumer parameter. **Türkçe:** Bize boş bir Optional verildiğini varsayalım. İlk yaklaşım, dış ifade için yanlış döner. İkinci yaklaşım boş bir Optional görür ve hem map() hem de filter() geçer. Ardından ifPresent() boş bir Optional görür ve Consumer parametresini çağırır. **English:** The next case is where we are given an Optional.of(4). The first approach returns false for the inner if statement. The second approach maps the number 4 to "4". The filter() then returns an empty Optional since the filter doesn't match, and ifPresent() doesn't call the Consumer parameter. **Türkçe:** Bir sonraki durum bize bir Optional.of(4) verildiği yerdir. İlk yaklaşım iç ifade için yanlış döner. İkinci yaklaşım maps 4'ten "4"e kadar olan sayı. filter() daha sonra filtre eşleşmediği için boş bir Optional döndürür ve ifPresent() Consumer parametresini çağırır. **English:** The final case is where we are given an Optional.of(123). The first approach returns true for both if statements. The second approach maps the number 123 to "123". The filter() then returns the same Optional, and ifPresent() now does call the Consumer parameter. **Türkçe:** Son durum, bize bir Optional.of(123) verildiği yerdir. İlk yaklaşım, her iki ifade için de geçerlidir. İkinci yaklaşım maps "123" için 123 sayıdır. filter() daha sonra aynı Optional değerini gönderir ve ifPresent() şimdi Consumer parametresini çağırır. **English:** Now suppose that we wanted to get an Optional<Integer> representing the length of the String contained in another Optional. Easy enough: **Türkçe:** Şimdi başka bir Optional içinde bulunan String uzunluğunu temsil eden bir Optional<Integer> almak istediğimizi varsayalım. Yeterince kolay:

```
Optional<Integer> result = optional.map(String::length);
```

English: What if we had a helper method that did the logic of calculating something for us that returns Optional<Integer>? Using map doesn't work: **Türkçe:** Ya bizim için Optional<Integer> döndüren bir şeyi hesaplama mantığını yapan bir yardımcı yöntemimiz olsaydı? map kullanımı çalışmıyor:

```
Optional<Integer> result = optional
    .map(ChainingOptionals::calculator); // DOES NOT COMPILE
```

English: The problem is that calculator returns `Optional<Integer>`. The `map()` method adds another `Optional`, giving us `Optional<Optional<Integer>>`. Well, that's no good. The solution is to call `flatMap()`, instead: **Türkçe:** Sorun, hesap makinesinin `Optional<Integer>` döndürmesidir. `map()` yöntemi başka bir `Optional` ekleyerek bize `Optional<Optional<Integer>` verir. Bu hiç iyi değil. Çözüm, bunun yerine `flatMap()` aramaktır:

```
Optional<Integer> result = optional
    .flatMap(ChainingOptionals::calculator);
```

English: This one works because `flatMap` removes the unnecessary layer. In other words, it flattens the result. Chaining calls to `flatMap()` is useful when you want to transform one `Optional` type to another.

Türkçe: Bu çalışır çünkü `flatMap` gereksiz katmanı kaldırır. Başka bir deyişle, sonucu düzleştirir. `flatMap()` çağrılarını zincirlemek, bir `Optional` türünü diğerine dönüştürmek istediğinizde kullanışlıdır.

English: Checked Exceptions and Functional Interfaces You might have noticed by now that most functional interfaces do not declare checked exceptions. This is normally okay. However, it is a problem when working with methods that declare checked exceptions. Suppose that we have a class with a method that throws a checked exception: **Türkçe:** Checked Exceptions ve Functional Interfaces Şimdiye kadar çoğu functional interfaces checked exceptions beyan etmediğini fark etmiş olabilirsiniz. Bu normalde sorun değil. Bununla birlikte, checked exceptions beyan eden yöntemlerle çalışırken bir sorundur. checked exception atan bir yönteme sahip bir sınıfımız olduğunu varsayalım:

```
import java.io.*;
import java.util.*;
public class ExceptionCaseStudy {
    private static List<String> create() throws IOException {
        throw new IOException();
    }
}
```

English: Now we use it in a stream: **Türkçe:** Şimdi stream içinde kullanıyoruz:

```
public void good() throws IOException {
    ExceptionCaseStudy.create().stream().count();
}
Nothing new here. The create() method throws a checked exception. The calling method
```

English: handles or declares it. Now, what about this one? **Türkçe:** tutar veya ilan eder. Peki ya bu?

```
public void bad() throws IOException {
    Supplier<List<String>> s = ExceptionCaseStudy::create; // DOES NOT COMPILE
}
```

English: The actual compiler error is as follows: **Türkçe:** Gerçek derleyici hatası aşağıdaki gibidir:

English: unhandled exception type IOException Say what now? The problem is that the lambda to which this method reference expands does not declare an exception. The Supplier interface does not allow checked exceptions. There are two approaches to get around this problem. One is to catch the exception and turn it into an unchecked exception. **Türkçe:** unhandled exception type IOException Şimdi ne diyeceksin? Sorun şu ki, bu method reference genişlettiği lambda bir istisna beyan etmiyor. Supplier arayüzü checked exceptions'ye izin vermez. Bu sorunun üstesinden gelmek için iki yaklaşım vardır. Biri istisnayı yakalamak ve bir unchecked exception haline getirmektir.

```
public void ugly() {
    Supplier<List<String>> s = () -> {
        try {
            return ExceptionCaseStudy.create();
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    };
}
```

English: This works. But the code is ugly. One of the benefits of functional programming is that the code is supposed to be easy to read and concise. Another alternative is to create a wrapper method with try/catch. **Türkçe:** Bu işe yarıyor. Ama kod çirkin. İşlevsel programlamanın faydalarından biri, kodun okunması kolay ve özlü olması gerektiğidir. Başka bir alternatif, deneme/yakalama ile bir ambalajlama yöntemi oluşturmaktır.

```
private static List<String> createSafe() {
    try {
        return ExceptionCaseStudy.create();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

English: Now we can use the safe wrapper in our Supplier without issue. **Türkçe:** Artık Supplier güvenli ambalajı sorunsuz bir şekilde kullanabiliriz.

```
public void wrapped() {
    Supplier<List<String>> s2 = ExceptionCaseStudy::createSafe;
}
```

Using a Splitterator

English: Suppose you buy a bag of food so two children can feed the animals at the petting zoo. To avoid arguments, you have come prepared with an extra empty bag. You take roughly half the food out of the main bag and put it into the bag you brought from home. The original bag still exists with the other half of the food. **Türkçe:** Diyelim ki iki çocuk evcil hayvan hayvanat bahçesindeki hayvanları besleyebilsin diye bir çanta yiyecek alıyorsunuz. Tartışmalardan kaçınmak için, ekstra boş bir çanta ile hazırlandınız.

Yiyeceklerin kabaca yarısını ana çantadan alıp evden getirdiğiniz torbaya koyarsınız. Orijinal çanta hala yemeğin diğer yarısıyla birlikte var. **English:** A Splitterator provides this level of control over processing. It starts with a Collection or a stream— that is your bag of food. You call `trySplit()` to take some food out of the bag. The rest of the food stays in the original Splitterator object. **Türkçe:** Bir Splitterator işleme üzerinde bu düzeyde kontrol sağlar. Collection veya stream ile başlar, bu sizin yiyecek çantanızdır. Çantadan biraz yiyecek almak için `trySplit()`'u ararsınız. Yiyeceklerin geri kalanı orijinal Splitterator nesnesinde kalır.

English: The characteristics of a Splitterator depend on the underlying data source. A Collection data source is a basic Splitterator. By contrast, when using a Stream data source, the Splitterator can be parallel or even infinite. The Stream itself is executed lazily rather than when the Splitterator is created. **Türkçe:** Bir Splitterator karakteristiği altta yatan veri kaynağına bağlıdır. Bir Collection veri kaynağı temel bir Splitterator kaynağıdır. Buna karşılık, bir Stream veri kaynağı kullanıldığında, Splitterator paralel hatta sonsuz olabilir. Stream'nin kendisi, Splitterator oluşturulduğu zaman yerine tembelce yürütülür. **English:** Implementing your own Splitterator can get complicated and is conveniently not on the exam. You do need to know how to work with some of the common methods declared on this interface. The simplified methods you need to know are in Table 10.9. **Türkçe:** Kendi Splitterator uygulamanız karmaşıklaşabilir ve sınavda uygun bir şekilde değildir. Bu arayüzde ilan edilen bazı yaygın yöntemlerle nasıl çalışılacağını bilmeniz gerekir. Bilmeniz gereken basitleştirilmiş yöntemler Tablo 10.9'dadır.

English: TABLE 10.9 Splitterator methods Method Description Splitterator<T> `trySplit()` **Türkçe:** TABLE 10.9 Splitterator yöntemleri Yöntem Açıklaması Splitterator<T> `trySplit()` **English:** Returns Splitterator containing ideally half of the data, which is removed from current Splitterator. This method can be called multiple times and will eventually return null when data is no longer splittable. **Türkçe:** Geçerli Splitterator 'den kaldırılan ideal olarak verilerin yarısını içeren Splitterator döndürür. Bu yöntem birden fazla kez çağrılabilir ve sonunda veriler artık bölünemez olduğunda null döndürür. **English:** Processes remaining elements in void `forEachRemaining(Consumer<T> c)` **Türkçe:** void `forEachRemaining(Consumer<T> c)` içinde kalan öğeleri işler **English:** Splitterator. **Türkçe:** Splitterator. **English:** boolean `tryAdvance(Consumer<T> c)` Processes single element from Splitterator if any remain. Returns whether element was processed. **Türkçe:** boolean `tryAdvance(Consumer<T> c)` Kalan varsa Splitterator tek elemanı işler. Elementin işlenip işlenmediğini döndürür. **English:** Now let's look at an example where we divide the bag into three: **Türkçe:** Şimdi çantayı üçe böldüğümüz bir örneğe bakalım:

```
var stream = List.of("bird- ", "bunny- ", "cat- ", "dog- ", "fish- ", "lamb- ",
"mouse- ");
Splitterator<String> originalBagOfFood = stream.splitterator();
Splitterator<String> emmasBag = originalBagOfFood.trySplit();
emmasBag.forEachRemaining(System.out::print); // bird-bunny-cat-
```

English: 17: **Türkçe:** 17:

```
Splitterator<String> jillsBag = originalBagOfFood.trySplit();
jillsBag.tryAdvance(System.out::print); // dog-
jillsBag.forEachRemaining(System.out::print); // fish-
```

English: 21: **Türkçe:** 21:

```
originalBagOfFood.forEachRemaining(System.out::print); // lamb-mouse-
```

English: On lines 12 and 13, we define a List. Lines 14 and 15 create two Spliterator references. The first is the original bag, which contains all seven elements. The second is our split of the original bag, putting roughly half of the elements at the front into Emma's bag. We then print the three contents of Emma's bag on line 16. **Türkçe:** 12 ve 13 satırlarında bir List tanımlarız. 14. ve 15. satırlar iki Spliterator referans oluşturur. Birincisi, yedi elementin tümünü içeren orijinal çantadır. İkincisi, orijinal çantanın yaklaşık yarısını Emma'nın çantasına koyarak, orijinal çantanın bölünmesidir. Daha sonra Emma'nın çantasının üç içeriğini 16. hatta basıyoruz. **English:** Our original bag of food now contains four elements. We create a new Spliterator on line 18 and put the first two elements into Jill's bag. We use tryAdvance() on line 19 to output a single element, and then line 20 prints all remaining elements (just one left!). **Türkçe:** Orijinal gıda torbamız şu anda dört element içeriyor. 18. hatta yeni bir Spliterator oluşturuyoruz ve ilk iki elemanı Jill'in çantasına koyuyoruz. Tek bir eleman çıkarmak için tryAdvance() satırını kullanıyoruz ve daha sonra kalan tüm elemanları (sadece bir tane kaldı!) satır 20 yazdırıyoruz. **English:** We started with seven elements, removed three, and then removed two more. This leaves us with two elements in the original bag created on line 14. These two items are output on line 22. **Türkçe:** Yedi elementle başladık, üç elementi kaldırdık ve iki elementi daha kaldırdık. Bu da bize 14. hatta oluşturulan orijinal çantada iki eleman bırakıyor. Bu iki parça 22. hatta çıktı.

English: Now let's try an example with a Stream. This is a complicated way to print out 123: **Türkçe:** Şimdi Stream ile bir örnek deneyelim. Bu, 123'ü basmanın karmaşık bir yoludur:

```
var originalBag = Stream.iterate(1, n -> ++n)
    .spliterator();
Spliterator<Integer> newBag = originalBag.trySplit();
newBag.tryAdvance(System.out::print); // 1
newBag.tryAdvance(System.out::print); // 2
newBag.tryAdvance(System.out::print); // 3
```

English: You might have noticed that this is an infinite stream. No problem! The Spliterator recognizes that the stream is infinite and doesn't attempt to give you half. Instead, newBag contains a large number of elements. We get the first three since we call tryAdvance() three times. It would be a bad idea to call forEachRemaining() on an infinite stream! **Türkçe:** Bunun bir infinite stream olduğunu fark etmiş olabilirsiniz. Önemli değil! Spliterator, stream nın sonsuz olduğunu ve size yarı vermeye çalışmadığını kabul eder. Bunun yerine, newBag çok sayıda eleman içerir. tryAdvance()'i üç kez çağırdığımızdan beri ilk üçünü alıyoruz. Bir infinite stream üzerinde forEachRemaining() aramak kötü bir fikir olurdu! **English:** Note that a Spliterator can have a number of characteristics such as CONCURRENT, ORDERED, SIZED, and SORTED. You will only see a straightforward Spliterator on the exam. For example, our infinite stream was not SIZED. **Türkçe:** Bir Spliterator CONCURRENT, ORDERED, SIZED ve SORTED gibi bir dizi özelliğe sahip olabilir. Sınavda sadece basit bir Spliterator göreceksiniz. Örneğin, infinite stream SIZED değildi.

Collecting Results

English: You're almost finished learning about streams. The last topic builds on what you've learned so far to group the results. Early in the chapter, you saw the `collect()` terminal operation. There are many predefined collectors, including those shown in Table 10.10. These collectors are available via static methods on the `Collectors` class. We look at the different types of collectors in the following sections. We left out the generic types for simplicity. **Türkçe:** streams hakkında bilgi edinmeyi neredeyse bitirdiniz. Son konu, sonuçları gruplamak için şimdiye kadar öğrendiklerinize dayanır. Bölümün başlarında, `collect()` terminal operation 'i gördünüz. Tablo 10.10'da gösterilenler de dahil olmak üzere birçok önceden tanımlanmış collectors vardır. Bu collectors, `Collectors` sınıfı üzerinde statik yöntemlerle kullanılabilir. Aşağıdaki bölümlerde farklı collectors türlerine bakıyoruz. Basitlik için jenerik tipleri dışarıda bıraktık.

There is one more collector called `reducing()`. You don't need to know it

English: for the exam. It is a general reduction in case all of the previous collectors don't meet your needs. **Türkçe:** Sınav için. Önceki collectors tüm gereksinimlerinizi karşılamaması durumunda genel bir reduction'dir.

Using Basic Collectors

English: Luckily, many of these collectors work the same way. Let's look at an example: **Türkçe:** Neyse ki, bu collectors ların çoğu aynı şekilde çalışır. Bir örneğe bakalım:

```
var ohMy = Stream.of("lions", "tigers", "bears");
String result = ohMy.collect(Collectors.joining(", "));
System.out.println(result); // lions, tigers, bears
```

English: Notice how the predefined collectors are in the `Collectors` class rather than the `Collector` interface. This is a common theme, which you saw with `Collection` versus `Collections`. In fact, you see this pattern again in Chapter 14 when working with `Paths` and `Path` and other related types. **Türkçe:** Collector arayüzü yerine `Collectors` sınıfında önceden tanımlanmış collectors öğesinin nasıl olduğuna dikkat edin. Bu, `Collection` ve `Collections` ile gördüğünüz ortak bir temadır. Aslında, Bölüm 14'te `Paths` and `Path` ve diğer ilgili türlerle çalışırken bu deseni tekrar görüyorsunuz.

English: TABLE 10.10 Examples of grouping/partitioning collectors Return value when passed Collector Description to collect Calculates average for averagingDouble(Double three core primitive ToDoubleFunction f) **Türkçe:** TABLE 10.10 grouping/partitioning collectors Örnekleri Collector Tanımlandığında geri dönüş değeri averagingDouble(Double three core primitive ToDoubleFunction f) için ortalama hesaplar **English:** types averagingInt(ToIntFunction f) averagingLong(ToLongFunction f) **Türkçe:** türleri averagingInt(ToIntFunction f) averagingLong(ToLongFunction f) **English:** counting() Counts number of ele-Long ments Applies filter before filtering(Predicate p, R calling downstream Collector c) **Türkçe:** counting() Ele-Long ments sayısını sayar filtering(Predicate p, R calling downstream Collector c) önce filtre uygular **English:** collector Creates map grouping groupingBy(Function f) **Türkçe:** collector map grouping groupingBy(Function f) oluşturur **English:** Map<K, List<T>> by specified function groupingBy(Function f, with optional map Collector dc) **Türkçe:** Map<K, List<T> belirtilen function groupingBy(Function f, with optional map Collector dc) **English:** type supplier and groupingBy(Function f, optional downstream Supplier s, Collector dc) **Türkçe:** supplier ve groupingBy(Function f, optional downstream Supplier s, Collector dc) yazın **English:** collector joining(CharSequence cs) Creates String single String using cs as delimiter bet-ween elements if one is specified Finds largest/smallest maxBy(Comparator c) **Türkçe:** collector joining(CharSequence cs) Cs'yi sınırlayıcı olarak kullanarak String tek String oluşturur Eğer belirtilmişse en büyük/en küçük maxBy(Comparator c) bulur **English:** Optional<T> elements minBy(Comparator c) **Türkçe:** Optional<T> elemanları minBy(Comparator c) **English:** Adds another level of mapping(Function f, Collector collectors Collector dc) **Türkçe:** Başka bir mapping(Function f, Collector collectors Collector dc) seviyesi ekler **English:** Creates map grouping partitioningBy(Predicate p) **Türkçe:** map grouping partitioningBy(Predicate p) oluşturur **English:** Map<Boolean, List<T>> by specified predicate partitioningBy(Predicate p, with optional further Collector dc) **Türkçe:** Map<Boolean, List<T> belirtilen predicate partitioningBy(Predicate p, with optional further Collector dc) **English:** downstream collector Calculates average, summarizingDouble(DoubleSummaryStatistics min, max, etc. ToDoubleFunction f) **Türkçe:** aşağı akış collector Ortalamayı hesaplar, summarizingDouble(DoubleSummaryStatistics min, max, etc. ToDoubleFunction f) **English:** IntSummaryStatistics LongSummaryStatistics summarizingInt(ToIntFunction f) summarizingLong(ToLongFunction f) **Türkçe:** IntSummaryStatistics LongSummaryStatistics summarizingInt(ToIntFunction f) summarizingLong(ToLongFunction f) **English:** (continued) **Türkçe:** (devam)

English: TABLE 10.10 Examples of grouping/partitioning collectors Return value when passed Collector Description to collect Calculates sum for our summingDouble(Double three core primitive ToDoubleFunction f) **Türkçe:** TABLE 10.10 grouping/partitioning collectors Örnekleri Collector Çıktığında geri dönüş değeri summingDouble(Double three core primitive ToDoubleFunction f) için toplam hesaplama açıklaması **English:** Integer types summingInt(TolntFunction f) **Türkçe:** Integer türleri summingInt(TolntFunction f) **English:** Long summingLong(ToLongFunction f) **Türkçe:** Long summingLong(ToLongFunction f) **English:** Works with results of teeing(Collector c1, R two collectors to cre-Collector c2, BiFunction f) **Türkçe:** teeing(Collector c1, R two collectors to cre-Collector c2, BiFunction f) sonuçlarıyla çalışır **English:** ate new type Creates arbitrary type toList() **Türkçe:** yeni tip yedi Rastgele tip oluşturur toList() **English:** List of list or set toSet() **Türkçe:** List list veya set toSet() **English:** Set toCollection(Supplier s) Creates Collection Collection of specified type Creates map using toMap(Function k, Function v) **Türkçe:** Set toCollection(Supplier s) Belirtilen türün Collection Collection oluşturur toMap(Function k, Function v) kullanarak map oluşturur **English:** Map functions to map toMap(Function k, Function v, keys, values, optional BinaryOperator m) **Türkçe:** Map functions için map toMap(Function k, Function v, keys, values, optional BinaryOperator m) **English:** merge function, and toMap(Function k, Function v, optional map type BinaryOperator m, Supplier s) **Türkçe:** function ve toMap(Function k, Function v, optional map type BinaryOperator m, Supplier s) birleştirir **English:** supplier We pass the predefined joining() collector to the collect() method. All elements of the stream are then merged into a String with the specified delimiter between each element. It is important to pass the Collector to the collect method. It exists to help collect elements. A Collector doesn't do anything on its own. **Türkçe:** supplier Önceden tanımlanmış joining() collector metodunu collect() metoduna geçiririz. stream 'nın tüm elemanları daha sonra her eleman arasında belirtilen sınırlayıcı ile bir String olarak birleştirilir. Collector'u toplama yöntemine geçirmek önemlidir. Elementleri toplamaya yardımcı olmak için vardır. Bir Collector kendi başına hiçbir şey yapmaz. **English:** Let's try another one. What is the average length of the three animal names? **Türkçe:** Bir tane daha deneyelim. Üç hayvan adının ortalama uzunluğu nedir?

```
var ohMy = Stream.of("lions", "tigers", "bears");
Double result = ohMy.collect(Collectors.averagingInt(String::length));
System.out.println(result); // 5.333333333333333
```

English: The pattern is the same. We pass a collector to collect(), and it performs the average for us. This time, we needed to pass a function to tell the collector what to average. We used a method reference, which returns an int upon execution. With primitive streams, the result of an average was always a double, regardless of what type is being averaged. For collectors, it is a Double since those need an Object.

Türkçe: Desen aynı. collector'den collect()'a geçiyoruz ve bizim için ortalamayı gerçekleştiriyor. Bu sefer, collector ortalamasını söylemek için bir function geçmemiz gerekiyordu. method reference kullandık, bu da yürütme üzerine int döndürür. primitive streams ile, bir ortalamanın sonucu, hangi tür ortalama sahip olursa olsun, her zaman bir double idi. collectors için, bir Nesneye ihtiyaç duydukları için bir Double 'dir.

English: Often, you'll find yourself interacting with code that was written without streams. This means that it will expect a Collection type rather than a Stream type. No problem. You can still express yourself using a Stream and then convert to a Collection at the end. For example: **Türkçe:** Sık sık, kendinizi streams olmadan yazılmış kodlarla etkileşimde bulacaksınız. Bu, bir Stream türü yerine bir Collection türü bekleyeceği anlamına gelir. Önemli değil. Stream kullanarak kendinizi ifade edebilir ve daha sonra sonunda Collection 'e dönüştürebilirsiniz. Örneğin:

```
var ohMy = Stream.of("lions", "tigers", "bears");
TreeSet<String> result = ohMy
    .filter(s -> s.startsWith("t"))
    .collect(Collectors.toCollection(TreeSet::new));
System.out.println(result); // [tigers]
```

English: This time we have all three parts of the stream pipeline. `Stream.of()` is the source for the stream. The intermediate operation is `filter()`. Finally, the terminal operation is `collect()`, which creates a `TreeSet`. If we didn't care which implementation of `Set` we got, we could have written `Collectors.toSet()`, instead.

Türkçe: Bu sefer stream pipeline 'nın üç parçasına da sahibiz. `Stream.of()` stream için kaynaktır. intermediate operation ise `filter()` dir. Son olarak, terminal operation bir `TreeSet` oluşturan `collect()` 'dir. `Set` uygulamasının hangisini aldığımızı önemsemeseydik, bunun yerine `Collectors.toSet()` yazabilirdik.

English: At this point, you should be able to use all of the Collectors in Table 10.10 except `groupingBy()`, `mapping()`, `partitioningBy()`, `toMap()`, and `teeing()`. **Türkçe:** Bu noktada, `groupingBy()`, `mapping()`, `partitioningBy()`, `toMap()` ve `teeing()` dışındaki Tablo 10.10'daki Collectors'in tümünü kullanabilmeniz gerekir.

Collecting into Maps

English: Code using Collectors involving maps can get quite long. We will build it up slowly. Make sure that you understand each example before going on to the next one. Let's start with a straightforward example to create a map from a stream: **Türkçe:** maps içeren Collectors kodu oldukça long alabilir. Yavaş yavaş inşa edeceğiz. Bir sonrakine geçmeden önce her örneği anladığınızdan emin olun. Bir stream 'den bir map oluşturmak için basit bir örnekle başlayalım:

```
var ohMy = Stream.of("lions", "tigers", "bears");
Map<String, Integer> map = ohMy.collect(
    Collectors.toMap(s -> s, String::length));
System.out.println(map); // {lions=5, bears=5, tigers=6}
```

English: When creating a map, you need to specify two functions. The first function tells the collector how to create the key. In our example, we use the provided `String` as the key. The second function tells the collector how to create the value. In our example, we use the length of the `String` as the value. **Türkçe:** Bir map oluştururken, iki functions belirtmeniz gerekir. İlk function, collector anahtarın nasıl oluşturulacağını söyler. Örneğimizde, sağlanan `String` anahtar olarak kullanılır. İkinci function collector değerini nasıl oluşturacağını söyler. Örneğimizde, `String` uzunluğunu değer olarak kullanırız. **English:** Returning the same value passed into a lambda is a common operation, so Java provides a method for it. You can rewrite `s -> s` as **Türkçe:** lambda'a aktarılan aynı değeri döndürmek yaygın bir operasyondur, bu nedenle Java bunun için bir yöntem sağlar. `s`'yi yeniden yazabilirsiniz `-> s` olarak

```
Function.identity(). It is not shorter and may or may not be clearer,
```

English: so use your judgment about whether to use it. **Türkçe:** Bu yüzden onu kullanıp kullanmayacağınıza dair kararınızı kullanın. **English:** Now we want to do the reverse and map the length of the animal name to the name itself. Our first incorrect attempt is shown here: **Türkçe:** Şimdi tam tersini yapmak istiyoruz ve map hayvan adının kendi adına olan uzunluğunu. İlk yanlış girişimiz burada gösterilmiştir:

```
var ohMy = Stream.of("lions", "tigers", "bears");
Map<Integer, String> map = ohMy.collect(Collectors.toMap(
String::length,
k -> k)); // BAD
```

English: Running this gives an exception similar to the following: **Türkçe:** Bunu çalıştırmak aşağıdakilere benzer bir istisna verir: **English:** Exception in thread "main" **Türkçe:** Exception iş parçacığında "ana"

```
java.lang.IllegalStateException: Duplicate key 5
```

English: What's wrong? Two of the animal names are the same length. We didn't tell Java what to do. Should the collector choose the first one it encounters? The last one it encounters? Concatenate the two? Since the collector has no idea what to do, it "solves" the problem by throwing an exception and making it our problem. How thoughtful. Let's suppose that our requirement is to create a comma-separated String with the animal names. We could write this: **Türkçe:** Sorun ne? Hayvan adlarından ikisi aynı uzunluktadır. Java ne yapacağımızı söylemedik. collector karşılaştığı ilk kişiyi Optional mı? Karşılaştığı son kişi? İkisini bir araya getirmek mi? collector ne yapacağını bilmediğinden, bir istisna atarak ve onu bizim problemimiz yaparak sorunu "çözer". Ne kadar düşüncelisin. Diyelim ki, gereksinimimiz hayvan isimleriyle virgülle ayrılmış bir String oluşturmak. Bunu yazabiliriz:

```
var ohMy = Stream.of("lions", "tigers", "bears");
Map<Integer, String> map = ohMy.collect(Collectors.toMap(
String::length,
k -> k,
(s1, s2) -> s1 + "," + s2));
System.out.println(map); // {5=lions,bears, 6=tigers}
System.out.println(map.getClass()); // class java.util.HashMap
```

English: It so happens that the Map returned is a HashMap. This behavior is not guaranteed. Suppose that we want to mandate that the code return a TreeMap instead. No problem. We would just add a constructor reference as a parameter: **Türkçe:** Böylece Map döndürülen bir HashMap olur. Bu davranışın garantisi yoktur. Diyelim ki, kodun yerine bir TreeMap döndürmesi için yetki vermek istiyoruz. Önemli değil. Bir parametre olarak bir constructor referansı ekleriz:

```
var ohMy = Stream.of("lions", "tigers", "bears");
TreeMap<Integer, String> map = ohMy.collect(Collectors.toMap(
String::length,
k -> k,
(s1, s2) -> s1 + "," + s2,
TreeMap::new));
System.out.println(map); // // {5=lions,bears, 6=tigers}
System.out.println(map.getClass()); // class java.util.TreeMap
```

English: This time we get the type that we specified. With us so far? This code is long but not particularly complicated. We did promise you that the code would be long! **Türkçe:** Bu sefer belirttiğimiz türe ulaşıyoruz. Şimdiye kadar bizimle mi? Bu kod long ama özellikle karmaşık değil. Kodun long olacağına söz verdik!

Grouping, Partitioning, and Mapping

English: Great job getting this far. The exam creators like asking about `groupingBy()` and `partitioningBy()`, so make sure you understand these sections very well. Now suppose that we want to get groups of names by their length. We can do that by saying that we want to group by length. **Türkçe:** Buraya kadar gelmen çok iyi oldu. Sınav yaratıcıları `groupingBy()` ve `partitioningBy()` hakkında soru sormaktan hoşlanırlar, bu yüzden bu bölümleri çok iyi anladığınızdan emin olun. Şimdi diyelim ki isim gruplarını uzunluklarına göre elde etmek istiyoruz. Bunu uzunluğuna göre gruplamak istediğimizi söyleyerek yapabiliriz.

```
var ohMy = Stream.of("lions", "tigers", "bears");
Map<Integer, List<String>> map = ohMy.collect(
    Collectors.groupingBy(String::length));
System.out.println(map); // {5=[lions, bears], 6=[tigers]}
```

English: The `groupingBy()` collector tells `collect()` that it should group all of the elements of the stream into a Map. The function determines the keys in the Map. Each value in the Map is a List of all entries that match that key. **Türkçe:** `groupingBy()` toplayıcısı `collection()`'a, akışın tüm öğelerini bir Haritaya gruplaması gerektiğini söyler. function, Map içindeki anahtarları belirler. Map içindeki her değer, bu anahtarla eşleşen tüm girdilerin bir List değeridir.

Note that the function you call in `groupingBy()` cannot return null. It

English: does not allow null keys. **Türkçe:** null tuşlarına izin vermiyor. **English:** Suppose that we don't want a List as the value in the map and prefer a Set instead. No problem. There's another method signature that lets us pass a downstream collector. This is a second collector that does something special with the values. **Türkçe:** map değeri olarak List istemediğimizi ve bunun yerine Set tercih ettiğimizi varsayalım. Önemli değil. Başka bir method signature daha var, bu da collector akışını geçmemizi sağlıyor. Bu değerlerle özel bir şey yapan ikinci bir collector dir.

```
var ohMy = Stream.of("lions", "tigers", "bears");
Map<Integer, Set<String>> map = ohMy.collect(
    Collectors.groupingBy(
        String::length,
        Collectors.toSet()));
System.out.println(map); // {5=[lions, bears], 6=[tigers]}
```

English: We can even change the type of Map returned through yet another parameter. **Türkçe:** Başka bir parametre ile döndürülen Map türünü bile değiştirebiliriz.

```
var ohMy = Stream.of("lions", "tigers", "bears");
TreeMap<Integer, Set<String>> map = ohMy.collect(
    Collectors.groupingBy(
        String::length,
        TreeMap::new,
        Collectors.toSet()));
System.out.println(map); // {5=[lions, bears], 6=[tigers]}
```

English: This is very flexible. What if we want to change the type of Map returned but leave the type of values alone as a List? There isn't a method for this specifically because it is easy enough to write with the existing ones. **Türkçe:** Bu çok esnek. Map döndürülen türünü değiştirmek istesek de List olarak değer türünü yalnız bıraksak ne olur? Bunun için özel bir yöntem yoktur, çünkü mevcut olanlarla yazmak yeterince kolaydır.

```
var ohMy = Stream.of("lions", "tigers", "bears");
TreeMap<Integer, List<String>> map = ohMy.collect(
    Collectors.groupingBy(
        String::length,
        TreeMap::new,
        Collectors.toList()));
System.out.println(map);
```

English: Partitioning is a special case of grouping. With partitioning, there are only two possible groups: true and false. Partitioning is like splitting a list into two parts. **Türkçe:** Partitioning özel bir grouping durumudur. partitioning ile sadece iki olası grup vardır: doğru ve yanlış. Partitioning bir list'ü iki parçaya bölmek gibidir. **English:** Suppose that we are making a sign to put outside each animal's exhibit. We have two sizes of signs. One can accommodate names with five or fewer characters. The other is needed for longer names. We can partition the list according to which sign we need. **Türkçe:** Farz edelim ki, her hayvanın sergisinin dışına çıkmak için bir işaret yapıyoruz. İki boyutlu işaretlerimiz var. Beş veya daha az karakterli isimleri barındırabilirsiniz. Diğerleri daha uzun isimler için gereklidir. list işaretini hangi işarete ihtiyaç duyduğumuza göre bölebiliriz.

```
var ohMy = Stream.of("lions", "tigers", "bears");
Map<Boolean, List<String>> map = ohMy.collect(
    Collectors.partitioningBy(s -> s.length() <= 5));
System.out.println(map); // {false=[tigers], true=[lions, bears]}
```

English: Here we pass a Predicate with the logic for which group each animal name belongs in. Now suppose that we've figured out how to use a different font, and seven characters can now fit on the smaller sign. No worries. We just change the Predicate. **Türkçe:** Burada her hayvan adının hangi gruba ait olduğu mantığıyla bir Predicate geçiyoruz. Şimdi farklı bir yazı tipini nasıl kullanacağımızı anladığımızı varsayalım ve yedi karakter şimdi daha küçük işarete sığabilir. Merak etme. Sadece Predicate'i değiştiriyoruz.

```
var ohMy = Stream.of("lions", "tigers", "bears");
Map<Boolean, List<String>> map = ohMy.collect(
    Collectors.partitioningBy(s -> s.length() <= 7));
System.out.println(map); // {false=[], true=[lions, tigers, bears]}
```

English: Notice that there are still two keys in the map—one for each boolean value. It so happens that one of the values is an empty list, but it is still there. As with groupingBy(), we can change the type of List to something else. **Türkçe:** map boolean değeri için hala iki anahtar olduğunu fark edin. Bu değerlerden birinin boş bir list olduğu böyle olur, ancak hala oradadır. groupingBy() ile olduğu gibi, List türünü başka bir şey olarak değiştirebiliriz.

```
var ohMy = Stream.of("lions", "tigers", "bears");
Map<Boolean, Set<String>> map = ohMy.collect(
    Collectors.partitioningBy(
        s -> s.length() <= 7,
        Collectors.toSet()));
System.out.println(map); // {false=[], true=[lions, tigers, bears]}
```

English: Unlike groupingBy(), we cannot change the type of Map that is returned. However, there are only two keys in the map, so does it really matter which Map type we use? **Türkçe:** groupingBy() 'dan farklı olarak, döndürülen Map türünü değiştiremeyiz. Ancak, map içinde sadece iki anahtar var, bu yüzden hangi Map türünü kullandığımız gerçekten önemli mi? **English:** Instead of using the downstream collector to specify the type, we can use any of the collectors that we've already shown. For example, we can group by the length of the animal name to see how many of each length we have. **Türkçe:** Türü belirtmek için collector alt akımını kullanmak yerine, daha önce gösterdiğimiz collectors herhangi birini kullanabiliriz. Örneğin, her bir uzunluğun kaç tane olduğunu görmek için hayvan adı uzunluğuna göre gruplayabiliriz.

```
var ohMy = Stream.of("lions", "tigers", "bears");
Map<Integer, Long> map = ohMy.collect(
    Collectors.groupingBy(
        String::length,
        Collectors.counting()));
System.out.println(map); // {5=2, 6=1}
```

English: Debugging Complicated Generics **Türkçe:** Hata Ayıklama Karmaşık Jenerikler

When working with collect(), there are often many levels of generics, making compiler

English: errors unreadable. Here are three useful techniques for dealing with this situation: **Türkçe:** Hatalar okunamaz. İşte bu durumla başa çıkmak için üç yararlı teknik: **English:** ■ ■ Start over with a simple statement, and keep adding to it. By making one tiny change at a time, you will know which code introduced the error. **Türkçe:** Basit bir ifadeyle baştan başlayın ve eklemeye devam edin. Bir seferde küçük bir değişiklik yaparak, hangi kodun hatayı başlattığını bileceksiniz. **English:** ■ ■ Extract parts of the statement into separate statements. For example, try writing Collectors.groupingBy(String::length, Collectors.counting());. If it compiles, you know that the problem lies elsewhere. If it doesn't compile, you have a much shorter statement to troubleshoot. **Türkçe:** İfadenin parçalarını ayrı ifadelerle ayırın. Örneğin, Collectors.groupingBy(String::length, Collectors.counting()) yazmayı deneyin; Eğer derlenirse, sorunun başka bir yerde olduğunu bilirsiniz. Derlemiyorsa, sorun gidermek için çok daha kısa bir ifadeniz vardır. **English:** ■ ■ Use generic wildcards for the return type of the final statement: for example, Map<?,?>. If that change alone allows the code to compile, you'll know that the problem lies with the return type not being what you expect. **Türkçe:** Son ifadenin return type için genel jokerleri kullanın: örneğin, Map<?,?>. Bu değişiklik tek başına kodun derlenmesine izin veriyorsa, sorunun return type beklediğiniz gibi olmamasıyla ilgili olduğunu bileceksiniz. **English:** Finally, there is a mapping() collector that lets us go down a level and add another collector. Suppose that we wanted to get the first letter of the first animal alphabetically of each length. Why? Perhaps for random sampling. The examples on this part of the exam are fairly contrived as well. We'd write the following: **Türkçe:** Son olarak, bir seviye aşağı inmemize ve başka bir collector eklememize izin veren bir mapping() collector var. Farz edelim ki ilk hayvanın ilk harfini alfabetik olarak her uzunlukta almak istedik. Neden sordun? Belki de rastgele örnekleme için. Sınavın bu bölümündeki örnekler de oldukça uydurmadır. Aşağıdakileri yazacağız:

```
var ohMy = Stream.of("lions", "tigers", "bears");
Map<Integer, Optional<Character>> map = ohMy.collect(
    Collectors.groupingBy(
        String::length,
        Collectors.mapping(
            s -> s.charAt(0),
            Collectors.minBy((a, b) -> a - b))));
System.out.println(map); // {5=Optional[b], 6=Optional[t]}
```

English: We aren't going to tell you that this code is easy to read. We will tell you that it is the most complicated thing you need to understand for the exam. Comparing it to the previous example, you can see that we replaced counting() with mapping(). It so happens that mapping() takes two parameters: the function for the value and how to group it further. **Türkçe:** Bu kodun okunmasının kolay olduğunu size söylemeyeceğiz. Sınav için anlamanız gereken en karmaşık şey olduğunu size söyleyeceğiz. Bir önceki örneğe kıyasla counting() ile mapping() değiştirdiğimizi görebilirsiniz. mapping() iki parametre alır: değer için function ve nasıl daha fazla gruplandırılacağı. **English:** You might see collectors used with a static import to make the code shorter. The exam might even use var for the return value and less indentation than we used. This means that you might see something like this: **Türkçe:** Kodu daha kısa yapmak için statik bir içe aktarma ile kullanılan collectors 'yi görebilirsiniz. Sınav, dönüş değeri için var ve kullandığımızdan daha az girinti kullanabilir. Bu şu şekilde bir şey görebileceğiniz anlamına gelir:

```
var ohMy = Stream.of("lions", "tigers", "bears");
var map = ohMy.collect(groupingBy(String::length,
    mapping(s -> s.charAt(0), minBy((a, b) -> a - b))));
System.out.println(map); // {5=Optional[b], 6=Optional[t]}
```

English: The code does the same thing as in the previous example. This means that it is important to recognize the collector names because you might not have the Collectors class name to call your attention to it. **Türkçe:** Kod, önceki örnekte olduğu gibi aynı şeyi yapar. Bu, collector adlarını tanımanın önemli olduğu anlamına gelir, çünkü Collectors sınıf adına sahip olmayabilirsiniz.

Teeing Collectors

English: Suppose you want to return two things. As we've learned, this is problematic with streams because you only get one pass. The summary statistics are good when you want those operations. Luckily, you can use teeing() to return multiple values of your own. **Türkçe:** Diyelim ki iki şeyi iade etmek istiyorsunuz. Öğrendiğimiz gibi, bu streams ile sorunludur, çünkü sadece bir geçiş alırsınız. Özet istatistikler, bu işlemleri istediğinizde iyidir. Neyse ki, teeing() kullanarak kendinize ait birden fazla değer döndürebilirsiniz. **English:** First, define the return type. We use a record here: **Türkçe:** İlk olarak, return type değerini belirleyin. Burada bir kayıt kullanıyoruz:

```
record Separations(String spaceSeparated, String commaSeparated) {}
```

English: Now we write the stream. As you read, pay attention to the number of Collectors: **Türkçe:** Şimdi stream yazacağız. Okuduğunuz gibi, Collectors sayısına dikkat edin:

```
var list = List.of("x", "y", "z");
Separations result = list.stream()
    .collect(Collectors.teeing(
        Collectors.joining(" "),
        Collectors.joining(","),
        (s, c) -> new Separations(s, c)));
System.out.println(result);
```

English: When executed, the code prints the following: **Türkçe:** Çalıştırıldığında, kod aşağıdakileri yazdırır: **English:** Separations[spaceSeparated=x y z, commaSeparated=x,y,z] There are three Collectors in this code. Two of them are for joining() and produce the values we want to return. The third is teeing(), which combines the results into the single object we want to return. This way, Java is happy because only one object is returned, and we are happy because we don't have to go through the stream twice. **Türkçe:** Ayrılıklar[spaceSeparated=x y z, commaSeparated=x,y,z] Bu kodda üç Collectors vardır. Bunlardan ikisi joining() içindir ve geri dönmek istediğimiz değerleri üretir. Üçüncüsü, sonuçları dönmek istediğimiz tek nesneye birleştiren teeing(). Bu şekilde, Java mutludur çünkü sadece bir nesne döndürülür ve biz mutluyuz çünkü stream üzerinden iki kez geçmek zorunda değiliz.

Summary

English: An Optional<T> can be empty or store a value. You can check whether it contains a value with isPresent() and get() the value inside. You can return a different value with orElse(T t) or throw an exception with orElseThrow(). There are even three methods that take functional interfaces as parameters: ifPresent(Consumer c), orElseGet(Supplier s), and orElseThrow(Supplier s). There are three optional types for primitives: OptionalDouble, OptionalInt, and OptionalLong. These have the methods getAsDouble(), getAsInt(), and getAsLong(), respectively. **Türkçe:** Bir Optional<T> boş olabilir veya bir değer saklayabilir. İçinde isPresent() ve get() değeri olan bir değer olup olmadığını kontrol edebilirsiniz. orElse(T t) ile farklı bir değer döndürebilir veya orElseThrow() ile bir istisna atabilirsiniz. Parametre olarak functional interfaces alan üç yöntem bile vardır: ifPresent(Consumer c), orElseGet(Supplier s) ve orElseThrow(Supplier s). İlkeler için üç optional türü vardır: OptionalDouble, OptionalInt ve OptionalLong. Bunlar sırasıyla getAsDouble(), getAsInt() ve getAsLong() yöntemlerine sahiptir. **English:** A stream pipeline has three parts. The source is required, and it creates the data in the stream. There can be zero or more intermediate operations, which aren't executed until the **Türkçe:** stream pipeline üç bölüme sahiptir. Kaynak gereklidir ve verileri stream içinde oluşturur. Sıfır veya daha fazla intermediate operations olabilir, bunlar çalıştırılana kadar çalıştırılmaz.

English: terminal operation runs. The first stream class we covered was `Stream<T>`, which takes a generic argument `T`. The `Stream<T>` class includes many useful intermediate operations including `filter()`, `map()`, `flatMap()`, and `sorted()`. Examples of terminal operations include `allMatch()`, `count()`, and `forEach()`.

Türkçe: terminal operation çalışır. Kaptığımız ilk stream sınıfı, genel bir argüman `T` alan `Stream<T>` idi. `Stream<T>` sınıfı, `filter()`, `map()`, `flatMap()` ve `sorted()` dahil olmak üzere birçok yararlı intermediate operations içerir. terminal operations örnekleri arasında `allMatch()`, `count()` ve `forEach()` bulunur.

English: Besides the `Stream<T>` class, there are three primitive streams: `DoubleStream`, `IntStream`, and `LongStream`. In addition to the usual `Stream<T>` methods, `IntStream` and `LongStream` have `range()` and `rangeClosed()`. The call `range(1, 10)` on `IntStream` and `LongStream` creates a stream of the primitives from 1 to 9. By contrast, `rangeClosed(1, 10)` creates a stream of the primitives from 1 to 10. The primitive streams have math operations including `average()`, `max()`, and `sum()`. They also have `summaryStatistics()` to get many statistics in one call.

Türkçe: `Stream<T>` sınıfının yanı sıra üç primitive streams vardır: `DoubleStream`, `IntStream` ve `LongStream`. Her zamanki `Stream<T>` yöntemlerine ek olarak, `IntStream` ve `LongStream` `range()` ve `rangeClosed()` var. `IntStream` ve `LongStream` üzerindeki `range(1, 10)` çağrısı, 1'den 9'a kadar ilkelerin stream çağrısını oluşturur. Buna karşılık, `rangeClosed(1, 10)` ilkelerin 1 ile 10 arasında bir stream oluşturur. primitive streams `average()`, `max()` ve `sum()` dahil olmak üzere matematik işlemlerine sahiptir. Ayrıca, bir çağrıda birçok istatistik elde etmek için `summaryStatistics()` vardır.

English: You can use a `Collector` to transform a stream into a traditional collection. You can even group fields to create a complex map in one line. Partitioning works the same way as grouping, except that the keys are always true and false. A partitioned map always has two keys, even if the value is empty for the key. A `teeing` collector allows you to combine the results of two other collectors. **Türkçe:** Bir stream'ı geleneksel bir collection'ye dönüştürmek için `Collector` kullanabilirsiniz. Bir satırda karmaşık bir map oluşturmak için alanları bile gruplayabilirsiniz. Partitioning tuşların her zaman doğru ve yanlış olması dışında grouping ile aynı şekilde çalışır. Bölümlü bir map, anahtar için değer boş olsa bile her zaman iki tuşa sahiptir. Bir `teeing` collector, diğer iki collectors sonucunu birleştirmenizi sağlar. **English:** You should memorize Table 10.6 and Table 10.7. At the least, be able to spot incompatibilities, such as type differences. Finally, remember that streams are lazily evaluated. They take lambdas or method references as parameters, which execute later when the method is run. **Türkçe:** Tablo 10.6 ve Tablo 10.7'yi ezberlemelisiniz. En azından, tip farklılıkları gibi uyumsuzlukları fark edebilme. Son olarak, streams'in tembel bir şekilde değerlendirildiğini unutmayın. lambdas veya method references parametrelerini alır, bu da yöntem çalıştırıldığında daha sonra çalışır.

Exam Essentials

English: Write code that uses `Optional`. Creating an `Optional` uses `Optional.empty()` or `Optional.of()`. Retrieval frequently uses `isPresent()` and `get()`. Alternatively, there are the functional `ifPresent()` and `orElseGet()` methods. **Türkçe:** `Optional` oluşturmak için `Optional.empty()` veya `Optional.of()` kodlarını yazın. Retrieval sık sık `isPresent()` ve `get()` kullanır. Alternatif olarak, fonksiyonel `ifPresent()` ve `orElseGet()` yöntemleri vardır. **English:** Recognize which operations cause a stream pipeline to execute. Intermediate operations do not run until the terminal operation is encountered. If no terminal operation is in the pipeline, a `Stream` is returned but not executed. Examples of terminal operations include `collect()`, `forEach()`, `min()`, and `reduce()`. **Türkçe:** stream pipeline ile execute. Intermediate arasındaki işlemlerin terminal operation ile karşılaşılıncaya kadar çalışmadığını kabul edin. pipeline içinde terminal operation yoksa, bir `Stream` döndürülür ancak çalıştırılmaz. terminal operations örnekleri arasında `collect()`, `forEach()`, `min()` ve `reduce()` bulunur. **English:** Determine which terminal operations are reductions. Reductions use all elements of the stream in determining the result. The reductions that you need to know are `collect()`, `count()`, `max()`, `min()`, and `reduce()`. A mutable reduction collects into the same object as it goes. The `collect()` method is a mutable reduction. **Türkçe:** terminal operations sonucunun hangi reductions. Reductions olduğunu belirlemek için stream ögesinin tüm elemanlarını kullanın. Bilmeniz gereken reductions `collect()`, `count()`, `max()`, `min()` ve `reduce()` dir. Değiştirilebilir bir reduction, gittiği gibi aynı nesneye toplanır. `collect()` yöntemi reduction değişkenidir. **English:** Write code for common intermediate operations. The `filter()` method returns a `Stream<T>` filtering on a `Predicate<T>`. The `map()` method returns a `Stream`, transforming each element of type `T` to another type `R` through a `Function <T,R>`. The `flatMap()` method flattens nested streams into a single level and removes empty streams. **Türkçe:** Ortak ara operations. The `filter()` yöntemi için kod yazın, bir `Predicate<T>` üzerinde bir `Stream<T>` filtreleme döndürür. `map()` yöntemi bir `Stream` döndürür ve her `T` tipi elemanını bir `Function T,R>` aracılığıyla başka bir `R` türüne dönüştürür. `flatMap()` yöntemi, streams yuvalanmış streams'u tek bir seviyeye düzleştirir ve boş streams'u kaldırır.

English: Compare primitive streams to `Stream<T>`. Primitive streams are useful for performing common operations on numeric types, including statistics like `average()`, `sum()`, and so on. There are three primitive stream classes: `DoubleStream`, `IntStream`, and `LongStream`. There are also three primitive Optional classes: `OptionalDouble`, `OptionalInt`, and `OptionalLong`. Aside from `BooleanSupplier`, they all involve the double, int, or long primitives. **Türkçe:** primitive streams ile `Stream<T>` karşılaştırması. Primitive streams, `average()`, `sum()` gibi istatistikler de dahil olmak üzere sayısal türlerde ortak işlemler yapmak için kullanışlıdır. Üç primitive stream sınıfı vardır: `DoubleStream`, `IntStream` ve `LongStream`. Ayrıca üç ilkel Optional sınıfı vardır: `OptionalDouble`, `OptionalInt` ve `OptionalLong`. Boolean Supplier dışında, hepsi double, int veya long ilkelerini içerir. **English:** Convert primitive stream types to other primitive stream types. Normally, when mapping, you just call the `map()` method. When changing the class used for the stream, a different method is needed. To convert to `Stream`, you use `mapToObj()`. To convert to `DoubleStream`, you use `mapToDouble()`. To convert to `IntStream`, you use `mapToInt()`. To convert to `LongStream`, you use `mapToLong()`. **Türkçe:** primitive stream türlerini diğer primitive stream types. Normally türlerine dönüştürün, mapping olduğunda, sadece `map()` yöntemini çağırırsınız. stream için kullanılan sınıfı değiştirirken, farklı bir yöntem gereklidir. Stream konumuna dönüştürmek için `mapToObj()` kullanılır. `DoubleStream` 'e dönüştürmek için `mapToDouble()` kullanılır. `IntStream` konumuna dönüştürmek için `mapToInt()` kullanılır. `LongStream` 'a dönüştürmek için `mapToLong()` kullanılır. **English:** Use `peek()` to inspect the stream. The `peek()` method is an intermediate operation often used for debugging purposes. It executes a lambda or method reference on the input and passes that same input through the pipeline to the next operator. It is useful for printing out what passes through a certain point in a stream. **Türkçe:** stream. The `peek()` yöntemini incelemek için `peek()` kullanın, genellikle hata ayıklama amacıyla kullanılan bir intermediate operation yöntemidir. Girdi üzerinde bir lambda veya method reference çalıştırır ve bir sonraki operatöre pipeline üzerinden aynı girdiyi geçirir. Bir stream içinde belirli bir noktadan geçenleri yazdırmak için kullanışlıdır. **English:** Search a stream. The `findFirst()` and `findAny()` methods return a single element from a stream in an Optional. The `anyMatch()`, `allMatch()`, and `noneMatch()` methods return a boolean. Be careful, because these three can hang if called on an infinite stream with some data. All of these methods are terminal operations. **Türkçe:** stream. The `findFirst()` ve `findAny()` yöntemlerinde tek bir elemanı stream içinde Optional döndürün. `anyMatch()`, `allMatch()` ve `noneMatch()` yöntemleri bir boolean döndürür. Dikkatli olun, çünkü bu üçü bazı verilerle bir infinite stream üzerinde çağrıldıklarında asılabilir. Tüm bu yöntemler terminal operations dir. **English:** Sort a stream. The `sorted()` method is an intermediate operation that sorts a stream. There are two versions: the signature with zero parameters that sorts using the natural sort order, and the signature with one parameter that sorts using that Comparator as the sort order. **Türkçe:** Bir stream. The `sorted()` metodu, bir stream'i sıralayan bir ara işlemdir. İki sürüm vardır: doğal sıralama sırasını kullanarak sıralayan sıfır parametrelili imza ve sıralama sırası olarak bu Comparator kullanarak sıralayan bir parametrelili imza. **English:** Compare `groupingBy()` and `partitioningBy()`. The `groupingBy()` method is a terminal operation that creates a Map. The keys and return types are determined by the parameters you pass. The values in the Map are a Collection for all the entries that map to that key. The `partitioningBy()` method also returns a Map. This time, the keys are true and false. The values are again a Collection of matches. If there are no matches for that boolean, the Collection is empty. **Türkçe:** `groupingBy()` ve `partitioningBy()` ile karşılaştırın. `groupingBy()` yöntemi, Map oluşturan bir terminal operation yöntemidir. Anahtarlar ve return types geçtiğiniz parametrelerle belirlenir. Map içindeki değerler, o anahtara map yazan tüm girdiler için Collection değeridir. `partitioningBy()` yöntemi de Map döndürür. Bu sefer anahtarlar doğru ve yanlış. Değerler yine maçların Collection değeridir. Bu boolean için eşleşme yoksa, Collection boştur.

Review Questions

English: The answers to the chapter review questions can be found in the Appendix. **Türkçe:** Bölüm inceleme sorularının cevapları Ek'te bulunabilir. **English:** 1.What could be the output of the following? **Türkçe:** 1. Aşağıdakilerin çıktısı ne olabilir?

```
var stream = Stream.iterate("", (s) -> s + "1");
System.out.println(stream.limit(2).map(x -> x + "2"));
```

English: A. 12112 **Türkçe:** A. 12112 **English:** B. 212 **Türkçe:** B. 212 **English:** C. 212112 **Türkçe:** C. 212112 **English:** D. java.util.stream.ReferencePipeline\$3@4517d9a3 **Türkçe:** D. java.util.stream.ReferencePipeline\$3@4517d9a3 **English:** E.The code does not compile. **Türkçe:** E.The kodu derlemez. **English:** F. An exception is thrown. **Türkçe:** F. Bir istisna atılır. **English:** G. The code hangs. **Türkçe:** G. Kod asılı. **English:** 2.What could be the output of the following? **Türkçe:** 2. Aşağıdakilerin çıktısı ne olabilir?

```
Predicate<String> predicate = s -> s.startsWith("g");
var stream1 = Stream.generate(() -> "growl!");
var stream2 = Stream.generate(() -> "growl!");
var b1 = stream1.anyMatch(predicate);
var b2 = stream2.allMatch(predicate);
System.out.println(b1 + " " + b2);
```

English: A. true false **Türkçe:** A. gerçek yanlış **English:** B. true true **Türkçe:** B. gerçek **English:** C. java.util.stream.ReferencePipeline\$3@4517d9a3 **Türkçe:** C. java.util.stream.ReferencePipeline\$3@4517d9a3 **English:** D.The code does not compile. **Türkçe:** D.The kodu derlemez. **English:** E. An exception is thrown. **Türkçe:** E. Bir istisna atılır. **English:** F. The code hangs. **Türkçe:** F. Kod asılı. **English:** 3.What could be the output of the following? **Türkçe:** 3. Aşağıdakilerin çıktısı ne olabilir?

```
Predicate<String> predicate = s -> s.length() > 3;
var stream = Stream.iterate("- ",
s -> ! s.isEmpty(), (s) -> s + s);
var b1 = stream.noneMatch(predicate);
var b2 = stream.anyMatch(predicate);
System.out.println(b1 + " " + b2);
```

English: A. false false **Türkçe:** A. yanlış yanlış **English:** B. false true **Türkçe:** B. yanlış doğru **English:** C. java.util.stream.ReferencePipeline\$3@4517d9a3 **Türkçe:** C. java.util.stream.ReferencePipeline\$3@4517d9a3 **English:** D.The code does not compile. **Türkçe:** D.The kodu derlemez. **English:** E. An exception is thrown. **Türkçe:** E. Bir istisna atılır. **English:** F. The code hangs. **Türkçe:** F. Kod asılı. **English:** 4.Which are true statements about terminal operations in a stream that runs successfully? **Türkçe:** 4. Başarılı çalışan bir stream içindeki terminal operations ile ilgili doğru ifadeler nelerdir? **English:** (Choose all that apply.) **Türkçe:** (Tüm geçerli olanları seçin.) **English:** A.At most one terminal operation can exist in a stream pipeline. **Türkçe:** A.At çoğu terminal operation bir stream pipeline içinde bulunabilir. **English:** B.Terminal operations are a required part of the stream pipeline in order to get a result. **Türkçe:** B.Terminal işlemleri, bir sonuç elde etmek için stream pipeline 'in gerekli bir parçasıdır. **English:** C.Terminal operations have Stream as the return type. **Türkçe:** C.Terminal işlemlerinde return type olarak Stream bulunur. **English:** D.The peek() method is an example of a terminal operation. **Türkçe:** D.The peek() yöntemi bir terminal operation örneğidir. **English:** E.The referenced Stream may be used after calling a terminal operation. **Türkçe:** E.The referanslı Stream bir terminal operation çağırıldıktan sonra kullanılabilir. **English:** 5.Which of the following sets result to 8.0? (Choose all that apply.) **Türkçe:** 5. Aşağıdaki sets sonuçlarından hangisi 8.0? (Tüm geçerli olanları seçin.) **English:** A. **Türkçe:** A.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .collect(Collectors.groupingBy(x -> x))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

English: B. **Türkçe:** B.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

English: C. **Türkçe:** C.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

English: D. **Türkçe:** D.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .collect(Collectors.groupingBy(x -> x, Collectors.toSet()))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

English: E. **Türkçe:** E.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x, Collectors.toSet()))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

English: F. **Türkçe:** F.

```
double result = LongStream.of(6L, 8L, 10L)
    .mapToInt(x -> (int) x)
    .boxed()
    .collect(Collectors.groupingBy(x -> x, Collectors.toSet()))
    .keySet()
    .stream()
    .collect(Collectors.averagingInt(x -> x));
```

English: 6. Which of the following can fill in the blank so that the code prints out false? (Choose all that apply.) **Türkçe:** 6. Aşağıdakilerden hangisi boşluğu doldurabilir, böylece kod yanlış yazdırır? (Tüm geçerli olanları seçin.)

```
var s = Stream.generate(() -> "meow");
var match = s. (String::isEmpty);
System.out.println(match);
```

English: A. allMatch **Türkçe:** A. allMatch **English:** B. anyMatch **Türkçe:** B. anyMatch **English:** C. findAny **Türkçe:** C. findAny **English:** D. findFirst **Türkçe:** D. findFirst **English:** E. noneMatch **Türkçe:** E. noneMatch **English:** F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbiri

English: 7. We have a method that returns a sorted list without changing the original. Which of the following can replace the method implementation to do the same with streams? **Türkçe:** 7. Orijinalini değiştirmeden sıralanmış bir list döndüren bir yöntemimiz var. streams ile aynı şeyi yapmak için yöntem uygulamasının yerine hangi following kullanılabilir?

```
private static List<String> sort(List<String> list) {
    var copy = new ArrayList<String>(list);
    Collections.sort(copy, (a, b) -> b.compareTo(a));
    return copy;
}
```

English: A. Türkçe: A.

```
return list.stream()
    .compare((a, b) -> b.compareTo(a))
    .collect(Collectors.toList());
```

English: B. Türkçe: B.

```
return list.stream()
    .compare((a, b) -> b.compareTo(a))
    .sort();
```

English: C. Türkçe: C.

```
return list.stream()
    .compareTo((a, b) -> b.compareTo(a))
    .collect(Collectors.toList());
```

English: D. Türkçe: D.

```
return list.stream()
    .compareTo((a, b) -> b.compareTo(a))
    .sort();
```

English: E. Türkçe: E.

```
return list.stream()
    .sorted((a, b) -> b.compareTo(a))
    .collect();
```

English: F. Türkçe: F.

```
return list.stream()
    .sorted((a, b) -> b.compareTo(a))
    .collect(Collectors.toList());
```

English: 8. Which of the following are true given this declaration? (Choose all that apply.) **Türkçe: 8.** Bu beyan göz önüne alındığında aşağıdakilerden hangisi doğrudur? (Tüm geçerli olanları seçin.)

```
var is = IntStream.empty();
```

English: A.is.average() returns the type int. **Türkçe:** A.is.average() int türünü döndürür. **English:** B.is.average() returns the type OptionalInt. **Türkçe:** B.is.average() OptionalInt türünü döndürür. **English:** C.is.findAny() returns the type int. **Türkçe:** C.is.findAny() int türünü döndürür.

English: D.is.findAny() returns the type OptionalInt. **Türkçe:** D.is.findAny() OptionalInt türünü döndürür.

English: E.is.sum() returns the type int. **Türkçe:** E.is.sum() int türünü döndürür. **English:** F.is.sum() returns the type OptionalInt. **Türkçe:** F.is.sum() OptionalInt türünü döndürür. **English:** 9. Which of the following can we add after line 6 for the code to run without error and not produce any output? (Choose all that apply.) **Türkçe:** 9. Kodun hatasız çalışması ve herhangi bir çıktıyı üretmemesi için aşağıdakilerden hangisini ekleyebiliriz? (Tüm geçerli olanları seçin.)

```
var stream = LongStream.of(1, 2, 3);
var opt = stream.map(n -> n * 10)
                .filter(n -> n < 5).findFirst();
```

English: A. **Türkçe:** A.

```
if (opt.isPresent())
    System.out.println(opt.get());
```

English: B. **Türkçe:** B.

```
if (opt.isPresent())
    System.out.println(opt.getAsLong());
```

English: C. **Türkçe:** C.

```
opt.ifPresent(System.out.println);
```

English: D. **Türkçe:** D.

```
opt.ifPresent(System.out::println);
```

English: E. None of these; the code does not compile. **Türkçe:** Bunlardan E. None; kod derlemez.

English: F. None of these; line 6 throws an exception at runtime. **Türkçe:** Bunlardan F. None; hat 6 çalışma zamanında bir istisna atar. **English:** 10. Given the four statements (L, M, N, O), select and order the ones that would complete the expression and cause the code to output 10 lines. (Choose all that apply.) **Türkçe:** 10. Dört ifade (L, M, N, O) göz önüne alındığında, ifadeyi tamamlayacak ve kodun 10 satır çıkmasına neden olacak olanları seçin ve sipariş edin. (Tüm geçerli olanları seçin.)

```
Stream.generate(() -> "1")
L:.filter(x -> x.length() > 1)
M:.forEach(System.out::println)
N:.limit(10)
O:.peek(System.out::println)
;
```

English: A. L, N **Türkçe:** A. L, N **English:** B. L, N, O **Türkçe:** B. L, N, O **English:** C. L, N, M **Türkçe:** C. L, N, M **English:** D. L, N, M, O **Türkçe:** D. L, N, M, O **English:** E. L, O, M **Türkçe:** E. L, O, M **English:** F. N, M **Türkçe:** F. N, M **English:** G. N, O **Türkçe:** G. N, O

English: 11. What changes need to be made together for this code to print the string 12345? (Choose all that apply.) **Türkçe:** 11. string 12345'i yazdırmak için bu kod için birlikte hangi değişikliklerin yapılması gerekir? (Tüm geçerli olanları seçin.)

```
Stream.iterate(1, x -> x++)
    .limit(5).map(x -> x)
    .collect(Collectors.joining());
```

English: A. Change Collectors.joining() to Collectors.joining(", "). **Türkçe:** A. Collectors.joining() ile Collectors.joining(", ") arasında değişiklik yapın. **English:** B. Change map(x -> x) to map(x -> "" + x). **Türkçe:** B. map(x -> x) ile map(x -> "" + x) arasında değişiklik yapın. **English:** C. Change x -> x++ to x -> ++x. **Türkçe:** C. x -> x++'ı x -> ++x'e değiştirin. **English:** D. Add.forEach(System.out::print) after the call to collect(). **Türkçe:** collect() çağrısından sonra D.Add.forEach(System.out::print). **English:** E. Wrap the entire line in a System.out.print statement. **Türkçe:** E. Wrap bir System.out.print ifadesinde tüm satır. **English:** F. None of the above. The code already prints 12345. **Türkçe:** Yukarıdaki F. None. Kod zaten 12345'i basıyor. **English:** 12. Which is true of the following code? **Türkçe:** 12. Aşağıdaki kod için hangisi doğrudur?

```
Set<String> birds = Set.of("oriole", "flamingo");
Stream.concat(birds.stream(), birds.stream(), birds.stream())
    .sorted() // line X
    .distinct()
    .findAny()
    .ifPresent(System.out::println);
```

English: A. It is guaranteed to print flamingo as is and when line X is removed. **Türkçe:** A. It flamingoyu olduğu gibi ve X satırı çıkarıldığında yazdırmayı garanti eder. **English:** B. It is guaranteed to print oriole as is and when line X is removed. **Türkçe:** B. It olduğu gibi ve X satırı çıkarıldığında oriole yazdırmak için garanti edilir. **English:** C. It is guaranteed to print flamingo as is, but not when line X is removed. **Türkçe:** C. It flamingoyu olduğu gibi yazdırmayı garanti eder, ancak X satırı kaldırıldığında değil. **English:** D. It is guaranteed to print oriole as is, but not when line X is removed. **Türkçe:** D. It olduğu gibi oriole yazdırmak için garanti edilir, ancak X satırı kaldırıldığında değil. **English:** E. The output may vary as is. **Türkçe:** E. The çıktısı olduğu gibi değişebilir. **English:** F. The code does not compile. **Türkçe:** F. The kodu derlemez. **English:** G. It throws an exception because the same list is used as the source for multiple streams. **Türkçe:** G. It bir istisna atar çünkü aynı list çoklu streams için kaynak olarak kullanılır. **English:** 13. Which of the following is true? **Türkçe:** 13. Aşağıdakilerden hangisi doğrudur?

```
List<Integer> x1 = List.of(1, 2, 3);
List<Integer> x2 = List.of(4, 5, 6);
List<Integer> x3 = List.of();
Stream.of(x1, x2, x3).map(x -> x + 1)
    .flatMap(x -> x.stream())
    .forEach(System.out::print);
```

English: A. The code compiles and prints 123456. **Türkçe:** A. The kodu 123456'i derler ve yazdırır. **English:** B. The code compiles and prints 234567. **Türkçe:** B. The kodu 234567'yi derler ve yazdırır. **English:** C. The code compiles but does not print anything. **Türkçe:** C. The kodu derlenir ancak hiçbir şey yazdırmaz. **English:** D. The code compiles but prints stream references. **Türkçe:** D. The kodu derler ancak stream referanslarını yazdırır.

English: E. The code runs infinitely. **Türkçe:** E. Kod sonsuza kadar çalışır. **English:** F. The code does not compile. **Türkçe:** F. The kodu derlemez. **English:** G. The code throws an exception. **Türkçe:** G. The kodu bir istisna atar. **English:** 14. Which of the following are true? (Choose all that apply.) **Türkçe:** 14. Aşağıdakilerden hangisi doğrudur? (Tüm geçerli olanları seçin.)

```
Stream<Integer> s = Stream.of(1);
IntStream is = s.boxed();
DoubleStream ds = s.mapToDouble(x -> x);
Stream<Integer> s2 = ds.mapToInt(x -> x);
s2.forEach(System.out::print);
```

English: A. Line 4 causes a compiler error. **Türkçe:** A. Line 4 bir derleyici hatasına neden olur. **English:** B. Line 5 causes a compiler error. **Türkçe:** B. Line 5 bir derleyici hatasına neden olur. **English:** C. Line 6 causes a compiler error. **Türkçe:** C. Line 6 bir derleyici hatasına neden olur. **English:** D. Line 7 causes a compiler error. **Türkçe:** D. Line 7 bir derleyici hatasına neden olur. **English:** E. Line 8 causes a compiler error. **Türkçe:** E. Line 8 bir derleyici hatasına neden olur. **English:** F. The code compiles but throws an exception at runtime. **Türkçe:** F. The kodu derlenir ancak çalışma zamanında bir istisna atar. **English:** G. The code compiles and prints 1. **Türkçe:** G. The kodu derler ve yazdırır 1. **English:** 15. Given the generic type String, the partitioningBy() collector creates a Map<Boolean, List<String>> when passed to collect() by default. When a down-stream collector is passed to partitioningBy(), which return types can be created? **Türkçe:** 15. Genel String tipi göz önüne alındığında, partitioningBy() collector varsayılan olarak collect()'ye geçtiğinde Map<Boolean, List<String> oluşturur. Bir aşağı akış toplayıcı bölümlenme By()'ye geçirildiğinde, hangi geri dönüş türleri oluşturulabilir? **English:** (Choose all that apply.) **Türkçe:** (Tüm geçerli olanları seçin.) **English:** A. Map<boolean, List<String>> **Türkçe:** A. Map<boolean, List<String> **English:** B. Map<Boolean, List<String>> **Türkçe:** B. Map<Boolean, List<String> **English:** C. Map<Boolean, Map<String>> **Türkçe:** C. Map<Boolean, Map<String> **English:** D. Map<Boolean, Set<String>> **Türkçe:** D. Map<Boolean, Set<String> **English:** E. Map<Long, TreeSet<String>> **Türkçe:** E. Map<Long, TreeSet<String> **English:** F. None of the above **Türkçe:** F. Yukarıdakilerin hiçbirisi **English:** 16. Which of the following statements are true about this code? (Choose all that apply.) **Türkçe:** 16. Aşağıdaki ifadelerden hangisi bu kod hakkında doğrudur? (Tüm geçerli olanları seçin.)

```
Predicate<String> empty = String::isEmpty;
Predicate<String> notEmpty = empty.negate();
```

English: 22: **Türkçe:** 22:

```
var result = Stream.generate(() -> "")
    .limit(10)
    .filter(notEmpty)
    .collect(Collectors.groupingBy(k -> k))
    .entrySet()
    .stream()
    .map(Entry::getValue)
```

```
.flatMap(Collection::stream)
.collect(Collectors.partitioningBy(notEmpty));
System.out.println(result);
```

English: A. It outputs {}. **Türkçe:** A. çıktısı verir. **English:** B. It outputs {false=[], true=[]}. **Türkçe:** B. false=[], true=[] çıktısını alır. **English:** C.If we changed line 31 from partitioningBy(notEmpty) to **Türkçe:** C.If partitioningBy(notEmpty) satırından partitioningBy(notEmpty) satırına değiştirdik. **English:** groupingBy(n -> n), it would output {}. **Türkçe:** groupingBy(n -> n), çıktısını alırdı. **English:** D.If we changed line 31 from partitioningBy(notEmpty) to **Türkçe:** D.If partitioningBy(notEmpty) satırından partitioningBy(notEmpty) satırına değiştirdik. **English:** groupingBy(n -> n), it would output {false=[], true=[]}. **Türkçe:** groupingBy(n -> n), false=[], true=[] çıktısını alırdı. **English:** E.The code does not compile. **Türkçe:** E.The kodu derlemez. **English:** F.The code compiles but does not terminate at runtime. **Türkçe:** F.The kodu derlenir ancak çalışma zamanında sona ermez. **English:** 17.What is the result of the following? **Türkçe:** 17. Aşağıdakilerin sonucu nedir?

```
var s = DoubleStream.of(1.2, 2.4);
s.peek(System.out::println).filter(x -> x > 2).count();
```

English: A. 1 **Türkçe:** A. 1 **English:** B. 2 **Türkçe:** B. 2 **English:** C. 2.4 **Türkçe:** C. 2.4 **English:** D. 1.2 and 2.4 **Türkçe:** D. 1.2 ve 2.4 **English:** E. There is no output. **Türkçe:** E. Çıktısı yok. **English:** F.The code does not compile. **Türkçe:** F.The kodu derlemez. **English:** G. An exception is thrown. **Türkçe:** G. Bir istisna atılır. **English:** 18.What is the output of the following? **Türkçe:** 18. Aşağıdakilerin çıktısı nedir?

```
public class Paging {
    record Sesame(String name, boolean human) {
        @Override public String toString() {
            return name();
        }
    }
    record Page(List<Sesame> list, long count) {}
}
```

English: 18: **Türkçe:** 18:

```
public static void main(String[] args) {
    var monsters = Stream.of(new Sesame("Elmo", false));
    var people = Stream.of(new Sesame("Abby", true));
    printPage(monsters, people);
}
```

English: 24: **Türkçe:** 24:

English: 25: private static void printPage(Stream<Sesame> monsters, **Türkçe:** 25: özel statik void printPage(Stream<Sesame> canavarlar,

```
Stream<Sesame> people) {
    Page page = Stream.concat(monsters, people)
        .collect(Collectors.teeing(
            Collectors.filtering(s -> s.name().startsWith("E"),
                Collectors.toList()),
            Collectors.counting(),
```

English: 32: **Türkçe:** 32:

```
(l, c) -> new Page(l, c));
System.out.println(page);
} }
```

English: A. Page[list=[Abby], count=1] **Türkçe:** A. Sayfa[list=[Abby], sayım=1] **English:** B. Page[list=[Abby], count=2] **Türkçe:** B. Sayfa[list=[Abby], sayım=2] **English:** C. Page[list=[Elmo], count=1] **Türkçe:** C. Sayfa[list=[Elmo], sayım=1] **English:** D. Page[list=[Elmo], count=2] **Türkçe:** D. Sayfa[list=[Elmo], sayım=2] **English:** E. The code does not compile due to Stream.concat(). **Türkçe:** Stream.concat() nedeniyle E. The kodu derlemez. **English:** F. The code does not compile due to Collectors.teeing(). **Türkçe:** Collectors.teeing() nedeniyle F. The kodu derlemez. **English:** G. The code does not compile for another reason. **Türkçe:** G. The kodu başka bir nedenden dolayı derlemez. **English:** 19. What is the simplest way of rewriting this code? **Türkçe:** 19. Bu kodu yeniden yazmanın en basit yolu nedir?

```
List<Integer> x = IntStream.range(1, 6)
    .mapToObj(i -> i)
    .collect(Collectors.toList());
x.forEach(System.out::println);
```

English: A. **Türkçe:** A.

```
IntStream.range(1, 6);
```

English: B. **Türkçe:** B.

```
IntStream.range(1, 6)
    .forEach(System.out::println);
```

English: C. **Türkçe:** C.

```
IntStream.range(1, 6)
    .mapToObj(i -> i)
    .forEach(System.out::println);
```

English: D. None of the above is equivalent. **Türkçe:** Yukarıdaki D. None eşdeğerdir. **English:** E. The provided code does not compile. **Türkçe:** E. The sağlanan kod derlemez.

English: 20. Which of the following throw an exception when an Optional is empty? (Choose all that apply.) **Türkçe:** 20. Aşağıdakilerden hangisi bir Optional boş olduğunda bir istisna atar? (Tüm geçerli olanları seçin.) **English:** A. opt.orElse(""); **Türkçe:** A. opt.orElse(""); **English:** B. opt.orElseGet(() -> ""); **Türkçe:** B. opt.orElseGet(() -> ""); **English:** C. opt.orElseThrow(); **Türkçe:** C. opt.orElseThrow(); **English:** D. opt.orElseThrow(() -> throw new Exception()); **Türkçe:** D. opt.orElseThrow(() -> yeni Exception() atın; **English:** E. opt.orElseThrow(RuntimeException::new); **Türkçe:** E. opt.orElseThrow(RuntimeException::new); **English:** F. opt.get(); **Türkçe:** F. opt.get(); **English:** G. opt.get(""); **Türkçe:** G. opt.get(""); **English:** 21. What is the output of the following? **Türkçe:** 21. Aşağıdakilerin çıktısı nedir?

```
var spliterator = Stream.generate(() -> "x")
    .spliterator();
spliterator.tryAdvance(System.out::print);
var split = spliterator.trySplit();
split.tryAdvance(System.out::print);
```

English: A. x **Türkçe:** A. x **English:** B. xx **Türkçe:** B. xx **English:** C. A long list of x's **Türkçe:** C. A long list x'in **English:** D. There is no output. **Türkçe:** D. Çıktısı yok. **English:** E. The code does not compile. **Türkçe:** E. The kodu derlemez. **English:** F. The code compiles but does not terminate at runtime. **Türkçe:** F. The kodu derlenir ancak çalışma zamanında sona ermez.

Appendix · Official Review Question Answers / Resmî Cevaplar

Aşağıdaki cevaplar kaynak Appendix bölümündeki sıra ve gerekçeleri korur. Türkçe bloklar doğal teknik çeviridir.

Official Answer 1

English: 1.D. No terminal operation is called, so the stream never executes. The first line creates an infi-nite stream reference. If the stream were executed on the second line, it would get the first two elements from that infinite stream, "" and "1", and add an extra character, resulting in "2" and "12", respectively. Since the stream is not executed, the reference is printed instead, giving us option D.

Türkçe: 1.D. terminal operation olarak adlandırılmaz, bu nedenle stream asla çalıştırılmaz. İlk satır bir infi-nite stream referans oluşturur. stream ikinci satırda çalıştırılırsa, ilk iki elemanı o infinite stream, "" ve "1" den alır ve sırasıyla "2" ve "12" ile sonuçlanan ekstra bir karakter ekler. stream çalıştırılmadığından, referans bunun yerine basılır ve bize D seçeneği verilir.

Official Answer 2

English: 2.F. Both streams created in this code snippet are infinite streams. The variable b1 is set to true since anyMatch() terminates. Even though the stream is infinite, Java finds a match on the first element and stops looking. However, when allMatch() runs, it needs to keep going until the end of the stream since it keeps finding matches. Since all elements continue to match, the program hangs, making option F the answer. **Türkçe:** 2.F. Bu kod snippet'inde oluşturulan her iki streams de infinite streams'dir. anyMatch() sona erdiğinden beri b1 değişkeni set doğrudur. stream sonsuz olsa da, Java ilk elemanda bir eşleşme bulur ve aramayı durdurur. Bununla birlikte, allMatch() çalıştığında, eşleşmeleri bulmaya devam ettiği için stream sonuna kadar devam etmesi gerekir. Tüm elemanlar eşleşmeye devam ettiğinden, program asılır ve F seçeneği cevap olur.

Official Answer 3

English: 3.E. An infinite stream is generated where each element is twice as long as the previous one.

Türkçe: 3.E. Her bir elemanın bir öncekinden iki kat daha uzun olduğu sonsuz bir akış oluşur. **English:** While this code uses the three-parameter `iterate()` method, the condition is never false. **Türkçe:** Bu kod üç parametrelili `iterate()` metodunu kullanırken, durum asla yanlış değildir. **English:** The variable `b1` is set to false because Java finds an element that matches when it gets to the element of length 4. However, the next line tries to operate on the same stream. Since streams can be used only once, this throws an exception that the “stream has already been operated upon or closed” and making option E the answer. If two different streams were used, the result would be option B. **Türkçe:** B1 değişkeni set ile yanlıştır, çünkü Java uzunluk 4 ögesine geldiğinde eşleşen bir eleman bulur. Bununla birlikte, bir sonraki satır aynı stream üzerinde çalışmaya çalışır. streams yalnızca bir kez kullanılabileceğinden, bu, “stream'nin zaten üzerinde çalıştığı veya kapatıldığı” bir istisna atar ve E seçeneğini cevap yapar. Eğer iki farklı streams kullanılmış olsaydı, sonuç B seçeneği olurdu.

Official Answer 4

English: 4.A, B. Terminal operations are the final step in a stream pipeline. Exactly one is required, because it triggers the execution of the entire stream pipeline. Therefore, options A and B are correct. Option C is true of intermediate operations rather than terminal operations. Option D is incorrect because `peek()` is an intermediate operation. Finally, option E is incorrect because once a stream pipeline is run, the Stream is marked invalid. **Türkçe:** 4.A, B. Terminal operations bir stream pipeline içindeki son adımdır. Tam olarak bir tane gereklidir, çünkü tüm stream pipeline uygulamasının çalıştırılmasını tetikler. Bu nedenle, A ve B seçenekleri doğrudur. C seçeneği terminal operations yerine intermediate operations için geçerlidir. D seçeneği yanlış çünkü `peek()` bir intermediate operation. Son olarak, E seçeneği yanlıştır, çünkü bir stream pipeline çalıştırıldığında, Stream geçersiz olarak işaretlenir.

Official Answer 5

English: 5.C, F. Yes, we know this question is a lot of reading. Remember to look for the differences between options rather than studying each line. These options all have much in common. All of them start out with a `LongStream` and attempt to convert it to an `IntStream`. However, options B and E are incorrect because they do not cast the long to an int, resulting in a compiler error on the `mapToInt()` calls. **Türkçe:** 5.C, F. Evet, bu sorunun çok okunan bir soru olduğunu biliyoruz. Her satırı incelemek yerine seçenekler arasındaki farkları aramayı unutmayın. Bu seçeneklerin hepsinin birçok ortak noktası vardır. Hepsi bir `LongStream` ile başlar ve bir `IntStream`'ye dönüştürmeye çalışır. Ne olursa olsun, B ve E seçenekleri yanlıştır, çünkü long yi bir int dosyasına atmaz, bu da `mapToInt()` çağrılarında bir derleyici hatasına neden olur.

English: Next, we hit the second difference. Options A and D are incorrect because they are missing `boxed()` before the `collect()` call. Since `groupingBy()` is creating a `Collection`, we need a nonprimitive Stream. The final difference is that option F specifies the type of `Collection`. This is allowed, though, meaning both options C and F are correct. **Türkçe:** Daha sonra ikinci farka ulaştık. A ve D seçenekleri yanlış çünkü `collect()` çağrısından önce `boxed()` eksik. `groupingBy()` bir `Collection` oluşturduğundan, birincil olmayan Stream gerekiyor. Son fark, F seçeneğinin `Collection` türünü belirtmesidir. Buna izin verilir, ancak C ve F seçeneklerinin her ikisinin de doğru olduğu anlamına gelir.

Official Answer 6

English: 6.A. Options C and D do not compile because these methods do not take a Predicate parameter and do not return a boolean. When working with streams, it is important to remember the behavior of the underlying functional interfaces. Options B and E are incorrect. While the code compiles, it runs infinitely. The stream has no way to know that a match won't show up later. Option A is correct because it is safe to return false as soon as one element passes through the stream that doesn't match. **Türkçe:** 6.A. C ve D seçenekleri derlemez, çünkü bu yöntemler bir Predicate parametresi almaz ve bir boolean döndürmez. streams ile çalışırken, altta yatan functional interfaces davranışını hatırlamak önemlidir. B ve E seçenekleri incorrect'tir. Kod derlenirken, sonsuz çalışır. stream bir maçın daha sonra gelmeyeceğini bilmenin hiçbir yolu yoktur. A seçeneği doğrudur, çünkü bir eleman eşleşmeyen stream 'den geçer geçmez yanlış dönmek güvenlidir.

Official Answer 7

English: 7.F. There is no Stream<T> method called compare() or compareTo(), so options A through D can be eliminated. The sorted() method is correct to use in a stream pipeline to return a sorted Stream. The collect() method can be used to turn the stream into a List. The collect() method requires a collector be selected, making option E incorrect and option F correct. **Türkçe:** 7.F. compare() veya compareTo() olarak adlandırılan bir Stream<T> yöntemi yoktur, bu nedenle A ile D arasındaki seçenekler ortadan kaldırılabilir. sorted() yöntemi, sıralanmış bir Stream döndürmek için bir stream pipeline 'da kullanmak için doğrudur. collect() yöntemi stream'u List'a dönüştürmek için kullanılabilir. collect() yöntemi bir collector seçilmesini gerektirir, E seçeneği yanlış ve F seçeneği doğru olur.

Official Answer 8

English: 8.D, E. The average() method returns an OptionalDouble since averages of any type can result in a fraction. Therefore, options A and B are both incorrect. The findAny() method returns an OptionalInt because there might not be any elements to find. Therefore, option D is correct. The sum() method returns an int rather than an OptionalInt because the sum of an empty list is zero. Therefore, option E is correct. **Türkçe:** 8.D, E. average() yöntemi bir OptionalDouble döndürür, çünkü herhangi bir türün ortalamaları bir kesirle sonuçlanabilir. Bu nedenle, A ve B seçeneklerinin her ikisi de yanlıştır. findAny() yöntemi bir OptionalInt döndürür, çünkü bulunacak herhangi bir eleman olmayabilir. Bu nedenle D seçeneği doğrudur. sum() yöntemi OptionalInt yerine int döndürür çünkü boş list toplamı sıfırdır. Bu nedenle, E seçeneği doğrudur.

Official Answer 9

English: 9.B, D. Lines 4–6 compile and run without issue, making option F incorrect. Line 4 creates a stream of elements [1, 2, 3]. Line 5 maps the stream to a new stream with values [10, 20, 30]. Line 6 filters out all items not less than 5, which in this case results in an empty stream. For this reason, `findFirst()` returns an empty `Optional`. **Türkçe:** 9.B, D. 4 6 satırları sorunsuz bir şekilde derleyip çalıştırır, F seçeneğinin yanlış olmasını sağlar. Line 4, elemanların bir stream oluşturur [1, 2, 3]. Satır 5 maps stream değeri [10, 20, 30] olan yeni bir stream'e. Hat 6, 5'ten az olmayan tüm öğeleri filtreler, bu durumda boş bir stream ile sonuçlanır. Bu nedenle `findFirst()` boş bir `Optional` döndürür. **English:** Option A does not compile. It would work for a `Stream<T>` object, but we have a `LongStream` and therefore need to call `getAsLong()`. Option C also does not compile, as it is missing the `::` that would make it a method reference. Options B and D both compile and run without error, although neither produces any output at runtime since the stream is empty. **Türkçe:** A seçeneği derlenmiyor. Bir `Stream<T>` nesnesi için çalışırdı, ancak `LongStream` var ve bu nedenle `getAsLong()` aramamız gerekiyor. C seçeneği de derlemez, çünkü eksik olan `::` bu onu bir method reference yapar. B ve D seçenekleri, stream boş olduğu için çalışma zamanında herhangi bir çıktı üretmese de, hatasız olarak derlenir ve çalıştırılır.

Official Answer 10

English: 10.F. Only one of the method calls, `forEach()`, is a terminal operation, so any answer in which M is not the last line will not execute the pipeline. This eliminates all but options C, E, and F. Option C is incorrect because `filter()` is called before `limit()`. Since none of the elements of the stream meets the requirement for the `Predicate<String>`, the `filter()` **Türkçe:** 10.F. Yöntem çağrılarında sadece biri, `forEach()`, bir terminal operation'dir, bu nedenle M'nin son satır olmadığı herhangi bir cevap pipeline çalıştırmayacaktır. Bu, C, E ve F seçenekleri dışındaki tüm seçenekleri ortadan kaldırır. C seçeneği yanlıştır, çünkü `filter()` `limit()`'den önce çağrılır. stream öğelerinin hiçbirisi `Predicate<String>` gereksinimini karşılamadığından, `filter()` **English:** operation will run infinitely, never passing any elements to `limit()`. Option E is incorrect because there is no `limit()` operation, which means that the code would run infinitely. **Türkçe:** işlem sonsuza kadar çalışacak, hiçbir öğeyi `limit()`'a aktarmayacak. E seçeneği yanlıştır, çünkü `limit()` işlemi yoktur, bu da kodun sonsuza kadar çalışacağı anlamına gelir. **English:** Only option F is correct. It first limits the infinite stream to a finite stream of ten elements and then prints the result. **Türkçe:** Tek `Optional F` doğrudur. Önce infinite stream on elementin finite stream ile sınırlanır ve daha sonra sonucu yazdırır.

Official Answer 11

English: 11.B, C, E. As written, the code doesn't compile because the `Collectors.joining()` expects to get a `Stream<String>`. Option B fixes this, at which point nothing is output because the collector creates a `String` without outputting the result. Option E fixes this and causes the output to be 11111. Since the post-increment operator is used, the stream contains an infinite number of the character 1. Option C fixes this and causes the stream to contain increasing numbers. **Türkçe:** 11.B, C, E. Yazıldığı gibi, kod derlemez çünkü `Collectors.joining()` bir `Stream<String>` almayı bekler. B seçeneği bunu düzeltir, bu noktada hiçbir şey çıktı değildir, çünkü collector sonucu çıkarmadan bir `String` oluşturur. E seçeneği bunu düzeltir ve çıktının 11111 olmasına neden olur. Artış sonrası operatör kullanıldığından, stream sonsuz sayıda karakter içerir. C seçeneği bunu düzeltir ve stream sayısının artan sayılar içermesine neden olur.

Official Answer 12

English: 12.F. The code does not compile because `Stream.concat()` takes two parameters, not the three provided. This makes the answer option F. **Türkçe:** 12.F. Kod derlemez çünkü `Stream.concat()` verilen üç parametreyi değil, iki parametreyi alır. Bu, cevap seçeneğini F yapar.

Official Answer 13

English: 13.F. If the `map()` and `flatMap()` calls were reversed, option B would be correct. In this case, the Stream created from the source is of type `Stream<List>`. Trying to use the addition operator (+) on a List is not supported in Java. Therefore, the code does not compile, and option F is correct. **Türkçe:** 13.F. `map()` ve `flatMap()` çağrıları tersine çevrilseydi, B seçeneği doğru olurdu. Bu durumda, kaynaktan oluşturulan Stream `Stream<List>` tipindedir. List üzerinde (+) ek operatörünü kullanmaya çalışmak Java içinde desteklenmez. Bu nedenle, kod derlemez ve F seçeneği doğrudur.

Official Answer 14

English: 14.B, D. Line 4 creates a Stream and uses autoboxing to put the Integer wrapper of 1 inside. Line 5 does not compile because `boxed()` is available only on primitive streams like `IntStream`, not `Stream<Integer>`. This makes option B one answer. Line 6 converts to a double primitive, which works since Integer can be unboxed to a value that can be implicitly cast to a double. Line 7 does not compile for two reasons making option D the second answer. First, converting from a double to an int would require an explicit cast. **Türkçe:** 14.B, D. Line 4 bir Stream oluşturur ve Integer ambalajını 1 içine koymak için otomatik kutulama kullanır. `Stream<Integer>` değil, `IntStream` gibi `boxed()` yalnızca primitive streams üzerinde mevcut olduğu için satır 5 derlenmiyor. Bu Optional B bir cevap yapar. Line 6, Integer'den beri çalışan bir double ilkeline dönüşür ve double'e dolaylı olarak atılabilecek bir değere kutulanamaz. Line 7, D seçeneğini ikinci cevap yapan iki nedenden dolayı derlemez. İlk olarak, bir double'dan bir int'a dönüştürmek açık bir döküm gerektirecektir. **English:** Also, `mapToInt()` returns an `IntStream`, so the data type of `s2` is incorrect. The rest of the lines compile without issue. **Türkçe:** Ayrıca, `mapToInt()` bir `IntStream` döndürür, bu nedenle `s2`'nin veri türü yanlıştır. Çizgilerin geri kalanı sorunsuz bir şekilde derlenir.

Official Answer 15

English: 15.B, D. Options A and C do not compile because they are invalid generic declarations. Primitives are not allowed as generics, and Map must have two generic type parameters. Option E is incorrect because partitioning only gives a Boolean key. Options B and D are correct because they return a Map with a Boolean key and a value type that can be customized to any Collection. **Türkçe:** 15.B, D. A ve C seçenekleri, geçersiz jenerik beyanlar oldukları için derlemez. İlkellere jenerik olarak izin verilmez ve Map iki jenerik tip parametreye sahip olmalıdır. E seçeneği yanlıştır çünkü partitioning sadece Boolean anahtar verir. B ve D seçenekleri doğrudur, çünkü bir Map anahtarı ve herhangi bir Collection için özelleştirilebilir bir değer türü ile bir Boolean döndürürler.

Official Answer 16

English: 16.B, C. First, this mess of code does compile. While it starts with an infinite stream on line 23, it becomes finite on line 24 thanks to `limit()`, making option F incorrect. The pipeline preserves only nonempty elements on line 25. Since there aren't any of those, the pipeline is empty. Line 26 converts this to an empty map. **Türkçe:** 16.B, C. İlk olarak, bu kod karmaşası derlenir. 23. hatta bir infinite stream ile başlarken, `limit()` sayesinde 24. satırda sonlu hale gelir ve F seçeneğini yanlış yapar. pipeline, 25. satırda sadece boş olmayan elemanları korur. Bunlardan hiçbiri olmadığından, pipeline boştur. Satır 26, bunu boş bir map'e dönüştürür. **English:** Lines 27 and 28 create a Set with no elements and then another empty stream. Lines 29 and 30 convert the generic type of the Stream to `List<String>` and then `String`. Finally, line 31 gives us another `Map<Boolean, List<String>>`. **Türkçe:** 27 ve 28. satırlar elementsiz bir Set ve sonra başka bir boş stream oluşturur. Satır 29 ve 30, Stream genel türünü `List<String>` ve daha sonra `String` olarak dönüştürür. Son olarak, 31. satır bize başka bir `Map<Boolean, List<String>` verir. **English:** The `partitioningBy()` operation always returns a map with two Boolean keys, even if there are no corresponding values. Therefore, option B is correct if the code is kept as is. **Türkçe:** `partitioningBy()` işlemi her zaman karşılık gelen değerler olmasa bile iki Boolean tuşuyla bir map döndürür. Bu nedenle, kod olduğu gibi tutulursa B seçeneği doğrudur. **English:** By contrast, `groupingBy()` returns only keys that are actually needed, making option C correct if the code is modified on line 31. **Türkçe:** Buna karşılık, `groupingBy()` yalnızca gerçekten gerekli olan anahtarları döndürür ve kod 31. satırda değiştirilirse C seçeneğini düzeltir.

Official Answer 17

English: 17.D. The terminal operation is `count()`. Since there is a terminal operation, the intermediate operations run. The `peek()` operation comes before the `filter()`, so both numbers are printed, making option D the answer. After the `filter()`, the `count()` happens to be 1 since one of the numbers is filtered out. However, the result of the stream pipeline isn't stored in a variable or printed, and it is ignored. **Türkçe:** 17.D. terminal operation `count()` şeklindedir. terminal operation olduğundan, intermediate operations çalıştırılır. `peek()` işlemi `filter()`'den önce gelir, bu nedenle her iki sayı da basılır ve D seçeneği cevap olur. `filter()`'den sonra `count()`, sayılardan biri filtrelendiğinden 1 olur. Bununla birlikte, stream pipeline sonucu bir değişkende veya basılı olarak depolanmaz ve göz ardı edilir.

Official Answer 18

English: 18.D. This compiles, ruling out options E, F, and G. Since line 29 filters by names starting with E, that rules out options A and B. Finally, line 31 counts the entire list, which is of size 2, giving us option D as the answer. **Türkçe:** 18.D. Bu, E, F ve G seçeneklerini ele alarak derler. 29. satır E ile başlayan isimlere göre filtrelendiğinden, bu A ve B seçeneklerini dışlar. Son olarak, satır 31, 2 boyutundaki tüm list'u sayar ve bize cevap olarak D seçeneğini verir.

Official Answer 19

English: 19.B. Both lists and streams have `forEach()` methods. There is no reason to collect into a list just to loop through it. Option A is incorrect because it does not contain a terminal operation or print anything. Options B and C both work. However, the question asks about the simplest way, which is option B.

Türkçe: 19.B. Hem listeler hem de akışlar `forEach()` yöntemlerine sahiptir. Bir list içine sadece döngü içine toplamak için bir neden yoktur. A seçeneği yanlıştır, çünkü bir terminal operation içermiyor veya herhangi bir şey yazdırmıyor. B ve C seçenekleri her ikisi de çalışır. Bununla birlikte, soru en basit yolunu, yani B seçeneğini sorar.

Official Answer 20

English: 20.C, E, F. Options A and B compile and return an empty string without throwing an exception, using a `String` and `Supplier` parameter, respectively. Option G does not compile as the `get()` method does not take a parameter. Options C and F throw a `NoSuchElementException`. Option E throws a `RuntimeException`. Option D looks correct but will compile only if the `throw` is removed. Remember, the `orElseThrow()`

Türkçe: 20.C, E, F. Seçenekler A ve B sırasıyla bir `String` ve `Supplier` parametresi kullanarak bir istisna atmadan boş bir string derleyip döndürür. Option G, `get()` yöntemi parametre almadığı için derlemez. C ve F seçenekleri `NoSuchElementException` atar. Option E bir `RuntimeException` atar. Option D doğru görünüyor, ancak sadece atış kaldırılırsa derlenecek. Hatırlayın, `orElseThrow()` **English:** should get a lambda expression or method reference that returns an exception, not one that throws an exception. **Türkçe:** lambda ifadesi veya method reference bir istisnayı döndüren bir istisnayı değil, bir istisnayı atan bir ifadeyi almalıdır.

Official Answer 21

English: 21.B. We start with an infinite stream where each element is `x`. The `splitterator()`

Türkçe: 21.B. Her elemanın `x` olduğu bir infinite stream ile başlıyoruz. `splitterator()` **English:** method is a terminal operation since it returns a `Splitterator` rather than a `Stream`. **Türkçe:** Stream yerine `Splitterator` döndürdüğü için terminal operation yöntemidir. **English:** The `tryAdvance()` method gets the first element and prints a single `x`. The `trySplit()` **Türkçe:** `tryAdvance()` yöntemi ilk elemanı alır ve tek bir `x` yazdırır. `trySplit()` **English:** method takes a large number of elements from the stream. Since this is an infinite stream, it doesn't attempt to take half. Then `tryAdvance()` is called on the new split variable, and another `x` is printed. Since there are two values printed, option B is correct. **Türkçe:** Yöntem, stream 'den çok sayıda eleman alır. Bu bir infinite stream olduğundan, yarısını almaya çalışmaz. Daha sonra `tryAdvance()` yeni bölünmüş değişkene çağrılır ve başka bir `x` yazdırılır. Basılı iki değer olduğundan, B seçeneği doğrudur.

Coverage ledger

- Chapter body marker'ları: 531–590
- Appendix answer marker'ları: 942–945
- Resmî cevap hedefi: 1–21
- Kod blokları özgün dilinde tutulmuştur.
- Çeviri ayrıntıları ünite vocabulary ve grammar kaynaklarıyla desteklenir.